

JAVA INTRO

Friday, July 19, 2024 11:48 PM

ABOUT JVM ARCHITECTURE

CLASS LOADERS

Def : its a subsystem of JVM used to load class files,3 major functions are loading,linking & initialisation.

3 ttypes of built in class loaders - bootstrap, extension , system/application.

JVM MEMORY

Method Area : THE JVM area stores class structure,such as the meta data,the class name,the variable,the code for methods,and the constant runtime pool.

Only **one method area per JVM** is shared **among all classes**

Heap :

Most cmn mem type shared across multiple threads,stores all objects & their related instance variables, & arrays. There is also **one heap area per JVM** that is shared

JVM Language Stacks :

These store local variables along with their results and play an important role in method invocation and return

A new frame gets created on method invocation =,frame gets deleted after the process of method invocation completes . Each thread has its own JVM stack and its created simultaneously when the thread is created

PC Registers :

A PC Register stores the address of the JVM instruction which is currently being executed

Each thread has a separate PC Register

Native method stacks : JVM contains stacks that support native methods used in the application and **for every thread a separate native method stack is created ,**

Execution Engine

It reads and executes the byte code (.class code) line by line , it has a JIT , interpreter and a garbage collector.

Native method interface

Java native interface (JNI) interacts with the native method libraries written in another language as C , C++ ; it enables JVM to call C/C++ libs , which may be specific to hardware

Native method libraries : a set of native libs (C, C++) which are required by the execution engine

Working :

.java(source code) --> javac(compiled code) --> .class (byte code) , JVM (INTERPRETER FOR WINDOWS)

A class may depend on other classes for compilation and execution

The env var class path points to the folders of .class files for the JDK or JVM to refer to

Only a corresponding **JVM** can **understand** the **bytecode**

JVM interprets the bytecode to real machine instructions

Real machine executes instructions to run the program

BASIC JAVA PRGM STRUCTURE

Public class Myfirst

```
{
    Public static void main(String [] args)
    {
    }
}
```

Every java program has at least one class with main() method

The main() method's signature is predefined

Name of classes and methods follow camel casing, class name starts with a capital letter and method name starts with a small letter

Source file is generally named after the class name

Print and println take string as a parameter

JVM MAKES JAVA PLATFORM INDEPENDENT

USE OF POINTERS

OBJECT

Groups related states and behaviours and is often used to model real world objects.

Object is a key to understand object oriented tech,

State represents the object's data (value)

Behaviour : functionality of an object.

--> **Can be referred as a class instance**

Exposes behaviour through methods (functions) and stores the state in fields (variables).

Thus an object is an encapsulation of state and behaviour

Represents instance of a class and resides in a specific memory location, **EACH OBJECT NOT ONLY OCCUPIES A DISTINCT PORTION OF MEMORY BUT ALSO HAS A UNIQUE ADDRESS WHICH POINTS TO ITS LOCATION WITHIN THE MEMORY**

OBJECT'S STATE AND BEHAVIOUR CAN BE ACCESSED WITH DOT (.) OPERATOR

Classes

It is a logical entity which includes a group of objects sharing common behaviour and properties

It is a blueprint for creating/instantiating individual objects, it's a template that describes the behaviour and state that the object of its type can support

Class does not consume any space, uses class keyword to define a class, **new** keyword is used to create/instantiate objects of the class

DECLARING CLASSES :

Single public class (probably name of the file), caps begin,

If class contained in a package first package statement & next the imports

Package --> import --> class

Class can implement one or many interfaces as per the requirement

Declaring member variables :

Member / instance variables are associated with an object, accessible to all of its methods and member functions

Declared inside the class **BUT Outside of Methods, Constructors, or any block.**

It has access modifiers (public, private, protected, default) they have a name and type (obv)

Ref type : string, array, object.

DEFINING METHODS :

METHOD declaration has flwng components

Modifier

Return type

Camel casing with first letter in lower case

Parameter list

Methods accessible outside of class are public ,avoid private methods asmuchas poss

PACKAGES

COLELCTION OF CLASSES that provide access protection and naemspace management

Classes declare the package which they belong to

NAMING CONVENTION :

Package name follows dot notation with an inverted domain name

Example : com.deloitte.myproject.

Com.mywebsite.newproject

Pacakge 1st , import next (when u use a class from diff package),

Package makes easier to find clases and interfaces

Avoid naming conflicts (ex : java.lang,java.io,java.util)

All classes froma package can be imported using a wild card (ex : import com.deloitte.myprohect.*;)

Public Class Stdnt

```
{  
    Private int id;  
    Private  
    String name;  
}
```

P s v m (s ards)

```
{  
    Stdnt std = new Stdnt();  
    Std.id = 483;  
    Std.name = "Hemanth";  
}
```

NAMING CONVENTIONS AND STANDARDS :

Package Names	• Should be in lowercase, with a reversed domain name as a prefix • Example: com.example.project
Class Names	• Should be in UpperCamelCase • Example: MyClass
Interface Names	• Should be in UpperCamelCase, and typically should be an adjective • Example: Serializable
Method Names	• Should be in lowerCamelCase • Example: calculateInterest()
Variable Names	• Should be in lowerCamelCase • Example: numberOfStudents

Constants	• Should be in uppercase, with words separated by underscores • Example: MAX_VALUE
Indentation and Formatting	Use 4 spaces for indentation and follow a consistent style throughout the code
Comments	Use meaningful comments sparingly to explain complex algorithms, design decisions, or to provide context where necessary
Braces	Use the "Egyptian brackets" style, with the opening brace on the same line as the declaration
Naming Conventions for Packages, Classes, and Methods	• Follow meaningful and descriptive names that accurately reflect the purpose of the entity • For example, a class that represents a student in a school management system

Naming Conventions (Cont.)

```
package com.example.schoolmanagement;

public class Student {
    private String studentName;
    private int studentId;

    public Student(String studentName, int
studentId) {
        this.studentName = studentName;
        this.studentId = studentId;
    }

    public String getStudentName() {
        return studentName;
    }

    public void setStudentName(String
studentName) {
        this.studentName = studentName;
    }
}
```

```
public int getStudentId() {
    return studentId;
}

public void setStudentId(int studentId) {
    this.studentId = studentId;
}

public void displayStudentInfo() {
    System.out.println("Student Name: " +
studentName);
    System.out.println("Student ID: " +
studentId);
}
```

CODING STANDARDS:

Naming conventions , indentation and formatting , comments , braces .

The basic rule for creating JavaDoc comments is that they **begin with /** and end with */**

Use egyptian brck style

Imports	Organize import statements alphabetically and avoid using wildcard imports ('import java.util.*').
Code Duplication	Minimize code duplication by extracting common functionality into methods or classes.
Error Handling	Handle exceptions appropriately, either by catching them or declaring them in the method signature.
Use of Final	Use the 'final' keyword for constants, method parameters, and variables that should not be reassigned.
Use of Enums	Use enums for representing a fixed set of constants.

Code Duplication

Minimize code duplication by extracting common functionality into methods or classes.

Testing

Write unit tests to verify the correctness of your code.

Security

Follow best practices for security, such as avoiding hardcoded sensitive information and sanitizing inputs.

Summary

Here are the key learning points of the module:

- Java programming language is widely used in the distributed Internet environment.
- The key features of Java include garbage collection, robust and secure, garbage collection, static, portable, object-oriented, distributed, multithreaded, and platform-independent.
- JDK is a combination of JRE and other tools, including compilers, debuggers, and several libraries.
- JVM contains stacks that support native methods used in the application. For every thread, a separate native stack is created.
- Java coding standards help maintain consistency and readability across codebases.

JAVA LANGAUAGE PROGRAMMING CONSTRUCTS

Saturday, July 20, 2024 7:28 PM

Java Language Programming Constructs :

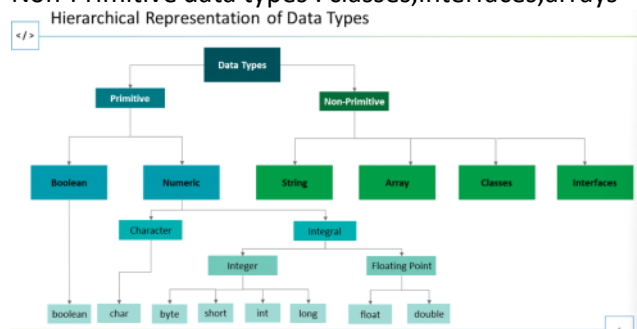
Topic	Descriptions	Duration
Java Language: Programming Constructs	Data Types and Variables and Their Scope, Java Return Types, Operators, Expressions, Blocks, and Statements, Control Flow Statements, and Keywords	12 hours
Control Flow Statements	If, if else, switch, for, while, do while, enhanced for, break, continue	
Modifiers	Access Modifiers, Non-access Modifiers, and Static keyword	

By the end of this session, you will be able to:

- Describe the basic constructs of Java programming language including data types, variables, operators, functions, and control flow statements
- Analyze the given problem-statement/use case, identify, and make use of appropriate Java language constructs for programming robust solutions in Eclipse
- Outline the use of access and non-access modifiers for defining/managing access mechanisms in Java

Prim : Boolean,char,int,short,long,float,double

Non-Primitive data types : classes,interfaces,arrays



Variables—Types and Scope

Variable Types	Descriptions
Local Variable	Declared inside the body of the method
Instance Variable (non-static fields)	Declared in the class but outside the method body
Class Variable (static fields)	Declared as static

Example:

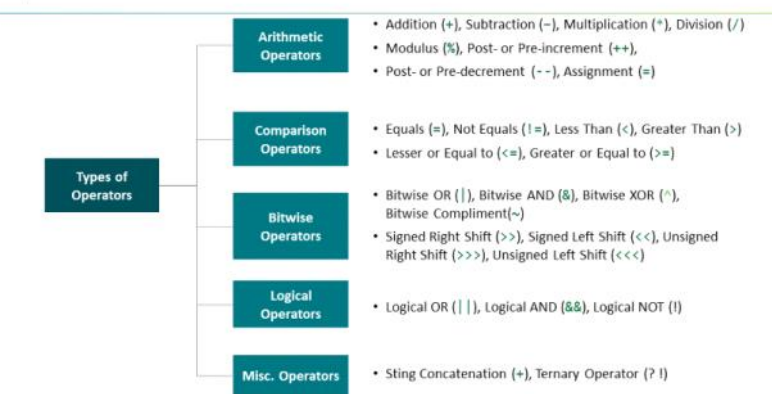
```
class A {
    int data = 50; // instance variable
    static int m = 100; // static variable
    void method() {
        int n = 90; // local variable
    }
} // end of class
```

Java Return Types

'return' is a keyword in Java that serves as an instruction for a program to exit a specific method. The exit may occur with the method either providing a value back to the part of the program that called it or without returning any value.

- It is mandatory for every method in Java to be declared with a return type.
- A method that does not return anything is declared as a return type (void).
- Methods are declared with the return type that corresponds to the type of value returned by a method.
- Methods may have return types such as:
 - primitive value (int, float, double)
 - class object
 - current or same-class object
 - variable

Operators



Expressions and Blocks

Expressions	Blocks
• A construct is made up of variables, operators, and method invocations. • The entire construct that evaluates to a single value is called an expression. Example: <pre>int numWheels = 4; System.out.println("No. of wheels in the car are " + numWheels); int area = length * breadth; if (value1 == value2) System.out.println("value1 == value2");</pre>	• Blocks are a group of statements between curly braces. • Blocks can consist of zero or more statements and be placed anywhere a single statement is allowed. Example: <pre>boolean condition = true; if (condition) { // Block 1 - Begins System.out.println("Condition is true."); } // Block 1 - Ends else { // Block 2 - Begins System.out.println("Condition is false."); } // Block 2 - Ends</pre>

Statements

Expression statement, declaration statement, control flow statement

Statements		
Expression Statement	Declaration Statement	Control Flow Statement
<pre>// assignment statement aValue = 8993.254; // increment statement aValue++; // method invocation statement System.out.println("Hello Java!"); // object creation statement Bicycle myBike = new Bicycle();</pre>	<pre>double aValue = 8993.254; int i = 400; long ssn = 456_33_41551;</pre>	<pre>if (expression){ } if (expression){ }else{ } switch(expression){- };</pre>

Statements

Expression Statement

Declaration Statement

Control Flow Statement

```
// assignment statement
aValue = 8993.254;
// increment statement
aValue++;
// method invocation statement
System.out.println("Hello Java!");
// object creation statement
Bicycle myBike = new Bicycle();
```

```
double aValue = 8993.254;
int i = 400;
long ssn = 456_33_4155L;
```

```
if (expression){
}
if (expression){
}else{
}
switch(expression){_
};
```

Copyright © 2024 Deloitte Development LLC. All rights reserved.

Control flow statements

Decision making statements, looping statements, branching statements,

Control Flow Statements

There are three kinds of control flow statements:

Decision-Making Statements

Based on the results of a condition, decision-making statements in Java allow you to make a decision.

- If-then
- If-then-else
- Switch

Looping Statements

A set of instructions repeatedly executed until a certain condition is filled is a loop in programming.

- While
- Do-While
- For

Branching Statements

When a certain condition is met a branching statement allows us to exit from a control statement.

- Break
- Continue
- Return

Switch is req,uk if and if-else

(first small)

```
Char grade = 'c';
```

```
Switch (grade) {
```

```
Case 'A' :
```

```
    Syso();
```

```
    Break;
```

```
Case 'B' :
```

```
    Syso();
```

```
    Brk;
```

```
    .
```

```
    .
```

```
Default :
```

```
    Sys0();
```

```
}
```

For,while,dowhile

The 'while' Statement:

Until the specified condition is true, 'while' is used to iterate a part of the program.

```
int count = 1;
while (count < 11) {
    System.out.println("code in while loop");
    count++;
}
```

The 'do-while' Statement:

Checks for the condition after executing the statements, like the while loop.

```
int count = 1;
do {
    System.out.println("code in do-while loop");
    count++;
} while (count < 11);
```

The 'for' Statement:

In Java, a 'for' statement, also known as a 'for' loop, is a control flow statement that allows a portion of code to be executed repeatedly based on a specified condition. This is particularly useful when we need to perform a task multiple times, such as iterating through an array or a collection.

```
for(int i=1; i<12; i++){
    System.out.println("Count is: " + i);
}
```

```
int[] numbers = {1,2,3,4,5,6,7,8,9,10};
for (int item : numbers) {
    System.out.println("Count is: " +
        item);
}
```

Breaks terminates the for,while,or dowhile loop,continue statement skips the crnt iteration of the for while and dow hile

Return exits from the crnt method and return the control flow to where it was invoked

MODIFIERS :

Access Modifiers and Rules

</>

- There are three access modifiers—public, protected, and private.
- There are four access levels—public, protected, default, and private.
 - **Public:** Public modifier access level can be accessed from within the class and package and outside the class and package. It is accessible everywhere.
 - **Private:** Private modifier access level is only within the class and cannot be accessed outside of the class.
 - **Default:** The default modifier access level is only within the package and cannot be accessed outside of the package. This is the default access level is you do not specify.
 - **Protected:** Protected modifier access level is within and outside the package through a child class. Without the child class it cannot be accessed from outside the package.

Illustration:

Modifier	Class	Package	Subclass
public	Y	Y	Y
private	Y	N	N
default	Y	Y	N
protected	Y	Y	Y

- Classes can have only public or default access.
- The class visibility revolves around whether the code in a class can:
 - Create an instance of another class;
 - Extend (or subclass) another class; and
 - Access methods and variables of another class.
- Some best practices that you must follow:
 - Make the classes public.
 - Make the fields (state) private.
 - Make the methods public.

Non-Access Modifiers and Rules

/>

In order to achieve many functionalities, Java provides several non-access modifiers, including:

- Static modifier: Creates class methods and variables
- Final modifier: Finalizes the implementations of classes, methods, and variables
- Abstract modifier: Creates abstract classes and methods
- Synchronized and Volatile modifiers: Are used for threads

Static variables : belong to class rather than any specific instance.this means all instances the same variable.

Static methods : can be called without creating an instance of a class,they can access static variables and other static methods.

Eg :	<pre> Public class Test{ static int x 100; static void increment(){ Counter++; } } </pre>
------	---

```

Final modifier : final int MAX = 100;
Public final void display() {
    System.out.println("Cannot be changed ");
}

```

Final classes cannot be subclassed

Abstract Modifier : **abstract classes** : cannot be **instantiated** and are **meant to be subclassed**, and abstract methods must be implemented by subclasses

Ex:

```

Abstract class Animal {
    Abstract void makeSound();
}

```

```

Class Dog extends Animal{
    Void makeSound(){
        System.out.println("Bark");
    }
}

```

Java Static Variable

- Also known as a class variable, is declared with the static keyword in a class, but outside a method, block, or constructor
- Has only one copy per class, irrespective of the number of objects that are created from it
- Is rarely used apart from being declared as a constant. A constant variable is declared as public/private, static, and final. The initial value of a constant variable never changes.
- Is stored in static memory which is also known as the method area
- Is created when the program starts and is destroyed once the program stops
- Has the same visibility as an instance variable. The visibility modifier (public, private, protected, or none) is chosen based on the specific requirements of the class and its interaction with other classes
- Is accessed using ClassName.VariableName

OOPS

Sunday, July 21, 2024 9:59 PM

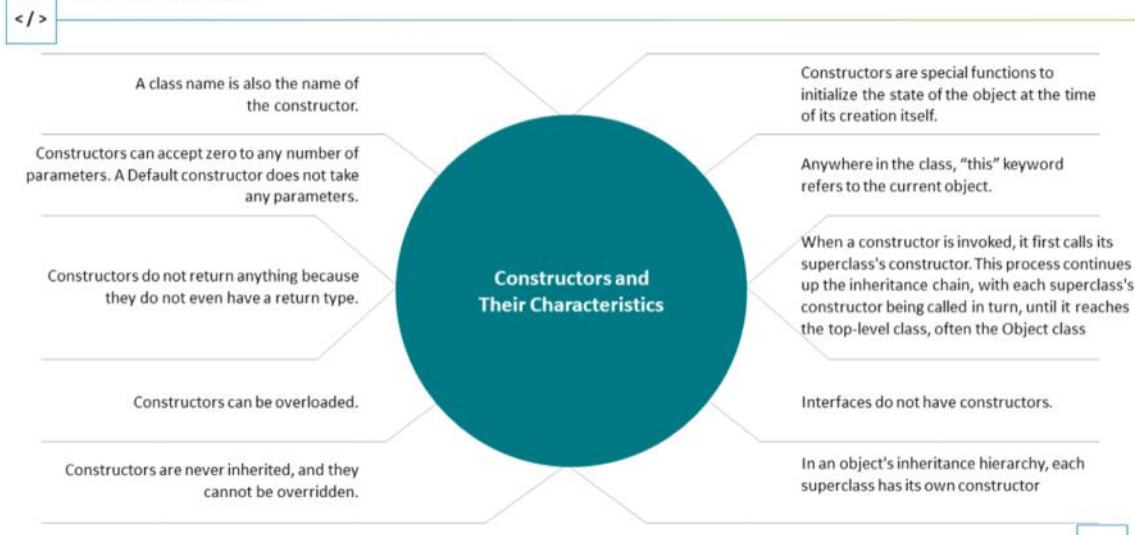
Topic	Descriptions	Duration
Java Constructors and this Keyword	Constructors, Constructor Chaining, and this Keyword and its Usage	8 hours
JAVA—OOPS Concepts	Object-Oriented Programming (OOP) Concepts in Java, Constructors, Polymorphism, Inheritance, Abstract Classes, and Encapsulation, Nested Classes, Abstract Classes, Interfaces	
Java Arrays	Java Arrays	

Learning Objectives

By the end of this session, you will be able to:

- Explain the fundamental concepts of constructors, this keyword, object-oriented programming including abstraction, encapsulation, polymorphism, and inheritance
- Describe nested classes, interfaces and abstract classes in Java
- Illustrate the use of constructors in Java along with this keyword
- Demonstrate the use of super keyword in Java inheritance
- Analyze the given problem-statement/use-case, identify, and make use of appropriate Java language constructs for programming robust solutions in Eclipse
- Discuss the concept of arrays for storing multiple values in a single Java variable along with their shortcomings
- Demonstrate the declaration, definition, and use of arrays in Java programs

Constructors



Constructor Types

In Java, constructors can be categorized into two main types:

Default Constructor:

This constructor is provided by Java if no other constructor is explicitly defined in the class. It initializes the object with default values.

Parameterized Constructor:

This constructor allows you to pass parameters when creating an object, enabling you to initialize the object with specific values.

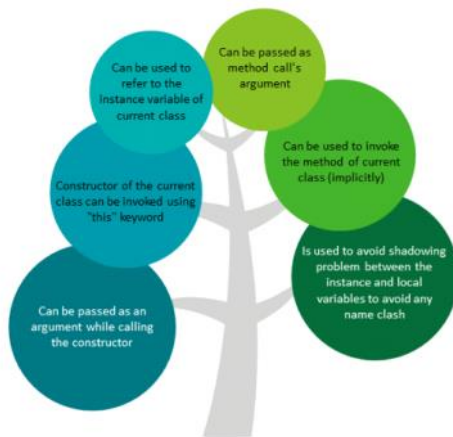
Constructor Chaining

- Refers to the process of one constructor calling another constructor in the same class or in its superclass
- Is typically done using the 'this()' or super() keyword

Using this()

- In a class, you can have multiple constructors with different parameter lists.
- You can use the this() keyword to invoke one constructor from another constructor in the same class.
- This allows you to reuse initialization code.

```
public class MyClass {  
    private int x;  
    private int y;  
    public MyClass() {  
        this(0, 0); // Calls the other constructor with default values  
    }  
    public MyClass(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
}
```



The "this" Keyword

Using super()

- If a subclass constructor does not explicitly call a superclass constructor using super(), the superclass's no-argument constructor is called implicitly.
- However, if the superclass does not have a no-argument constructor, the subclass constructor must explicitly call one of the superclass constructors using super().

```
public class SubClass extends SuperClass {  
    private int z;  
  
    public SubClass(int x, int y, int z) {  
        super(x, y); // Calls the superclass constructor with parameters  
        this.z = z;  
    }  
}
```

Constructor Chaining Example

```
// Superclass
class Vehicle {
    private String type;

    // Constructor of Vehicle
    public Vehicle(String type) {
        this.type = type;
        System.out.println("Vehicle type: " + type);
    }
}

// Subclass
class Car extends Vehicle {
    private String brand;
    private String model;

    // No-argument constructor
    public Car() {
        this("Unknown", "Unknown"); // Chaining to another constructor in the same class
    }
}
```

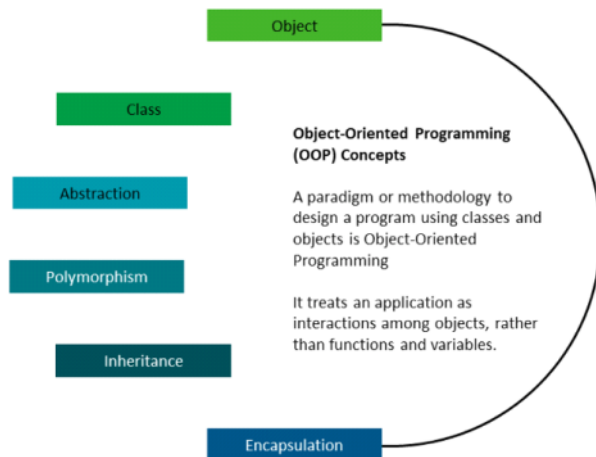
Constructor Chaining Example (Cont.)

```
// Constructor with one argument
public Car(String brand) {
    this(brand, "Unknown"); // Chaining to another constructor in the same class
}

// Constructor with two arguments
public Car(String brand, String model) {
    super("Car"); // Calling superclass constructor
    this.brand = brand;
    this.model = model;
    System.out.println("Car brand: " + brand + ", model: " + model);
}

public static void main(String[] args) {
    // Creating Car objects using different constructors
    Car car1 = new Car();
    Car car2 = new Car("Toyota");
    Car car3 = new Car("Honda", "Civic");
}
}
```

Object-Oriented Programming Concepts



OOP Principles

</>

	Definitions	How Do We Achieve It in Java?
Abstraction	- Hides internal details of an object from the user and shows only the relevant data - Example: Logging into an online bank account	- To achieve abstraction, use the abstract class and interface - Abstract means incomplete (The login verification process is abstracted from you).
Polymorphism	- Has the ability of a method to be used in different ways, which means it can be in different forms at different times (poly + morphos) - The two types of polymorphism are compile-time polymorphism and run-time polymorphism.	- Use method overloading and method overriding - The compile-time (static) polymorphism occurs when a method is overloaded whereas the run-time (dynamic) polymorphism occurs when the actual method itself is changed.
Inheritance	- Refers to when an object acquires (or inherits) the parent's object behaviors and properties - Allows the reuse of codes and improves reusability - Use the "extends" keyword to inherit a class.	- Use extends keywords to declare inheritance. - The parent class is called the base class or super class. - Derived class, Subclass, or Child class- Child class extending the base class.
Encapsulation	- Binds object state (fields) and behavior (methods) together - Protects the integrity of the data - Encapsulation is often confused with data abstraction.	- Use private variables to achieve encapsulation. - Java bean is the fully-encapsulated class because all the data members are private here.

Inheritance—Illustration

/>

Inheritance

```
class SuperClassA {
    public void foo(){
        System.out.println("SuperClassA");
    }
}

class SubClassB extends SuperClassA{
    public void bar(){
        System.out.println("SubClassB");
    }
}

public class Test {
    public static void main(String args[]){
        SubClassB a = new SubClassB();
        a.foo();
        a.bar();
    }
}
```

- Inheritance in Java can be implemented using the *extends* keyword.
- Superclass* is the object being inherited and the *subclass* is the object that inherits the *superclass*.
- A class can extend only one *superclass*.
- Java does not support multiple inheritance; however, a class can implement multiple interfaces.

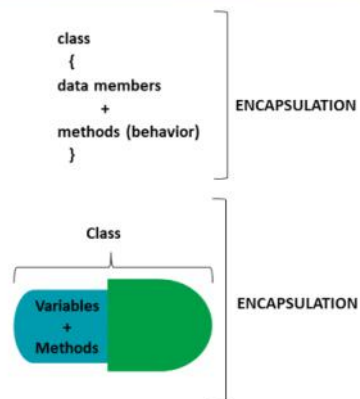
Encapsulation

</>

Encapsulation in Java is a way to wrap the data (variables) and the code acting on the data (methods) together—as a single unit.

Advantages:

- Data Hiding:** Hiding the implementation details in order to restrict the access of the data members.
- Increased Flexibility:** It enables you to make the variables of the class read-only or write-only, depending on the requirement.
- Reusability:** Encapsulation also improves the reusability and is easy to change with new requirements.
- Easy to Test:** Encapsulated code is easy to test for unit testing.



Polymorphism—Method Overloading and Method Overriding—Illustration

/>

Method Overloading

```
public class Circle {
    public void draw(){
        System.out.println("Circle with basic color");
    }

    public void draw(int diameter){
        System.out.println("Circle with basic color
and diameter");
    }

    public void draw(int diameter, String color){
        System.out.println("Circle with provided
parameter color and diameter");
    }
}
```

Method Overriding

```
interface Shape {
    public void draw();
}

class Circle implements Shape {
    public void draw() {
        System.out.println("Drawing circle");
    }
}

class Square implements Shape {
    public void draw() {
        System.out.println("Drawing Square");
    }
}

class Test{
    static Shape getShape() {
        return new Circle();
        // return new Square();
    }

    public static void main(String[] args) {
        Shape sh1 = getShape(); // logic to determine shape
        sh1.draw();
    }
}
```

Abstract Classes

An abstract class can have abstract methods, concrete methods and constructors. It can't be instantiated.

The "abstract" keyword is used to declare an abstract class that does not support multiple inheritance.

A class with abstract methods must be declared abstract.

Abstract classes have partial implementation, but they are meant for further extension by subclasses.

A concrete method has a body compared to an abstract method which does not have a body.

An abstract class can have final, instance, static variables.

Abstract
Class

Abstract Class—Illustration

/>

Abstract Class

```
abstract class Bike{
    abstract void run();
}
```

```
class Bullet extends Bike{
    void run(){System.out.println("running");}

    public static void main(String args[]){
        Bike obj = new Bullet();
        obj.run();
    }
}
```

Abstract vs. Final Classes

/>

Classes can be final and abstract, but they cannot be declared both final and abstract at a time.

Do you know that a final class cannot be subclassed, and an abstract class cannot be instantiated?

A single abstract method in a class implies that the whole class must be abstract.

The first concrete class to extend an abstract class must implement all the abstract methods.

Interfaces

A Java interface is used to achieve abstraction. However, they cannot be instantiated.

By default, all the methods() inside an interface are public abstract methods.

By default, all the variables declared inside an interface are public static final and must be initialized.

Interface cannot have a constructor.

By interfaces, we can achieve multiple inheritance.

Since Java 8, interfaces can have both default and static methods.

This feature is used to achieve loose coupling.

Interface vs. Abstract Classes

Interface

- Can be implemented by a class using the keyword "implements"
- Can extend multiple interfaces
- Has only static and final variables
- Contains only abstract methods (However, it can have both static and default and static methods starting Java 8)
- Cannot provide the implementation of the abstract class
- Members of a Java interface are public by default

```
public interface Drawable{  
    void draw();  
}
```

Abstract Class

- Can be extended using the keyword "abstract"
- Can extend another Java class and implement multiple Java interfaces
- Can have abstract and non-abstract methods
- Can have final, non-final, static and non-static variables
- Can provide the implementation of interface
- Can have class members, such as private and protected

```
public abstract class Shape{  
    public abstract void draw();  
}
```

Knowledge Check 2

What is the return type of constructors?

A. int

B. float

C. void

D. None of the above

Which method can be defined only once in a program?

A. The main method

B. The finalize method

C. The static method

D. The private method

ANSWERS :
NONE
MAIN METHOD
BOTH
SUPER
PRIVATE

Which of these options can be overloaded?

A. Methods

B. Constructors

C. Both

D. None of the above

Which of these keywords is used to refer to the member of a base class from a subclass?

A. Upper

B. Super

C. This

D. None of the above

What should all variables of a class be ideally declared as?

A. private

B. public

C. protected

D. default

Which of these keywords is used to prevent content of a variable from being modified?

A. final

B. last

C. constant

D. static

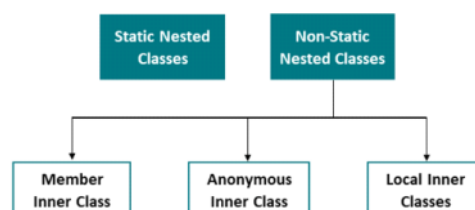
What Are Nested Classes?

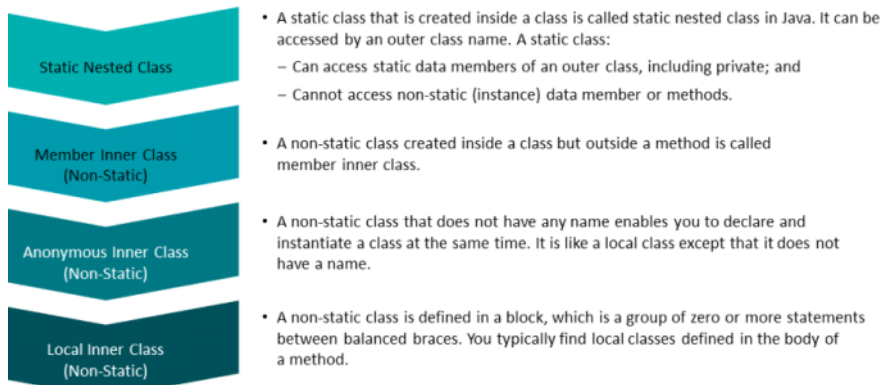
Is a class within another class or a logical grouping of classes that are only used in one place.

A nested class:

- Increases encapsulation
- Leads to more readable and maintainable code
- Non-static nested classes are also called inner classes

Types of Nested Classes





Arrays

- A Java array is an object that contains/stores elements of a similar data type in contiguous memory locations.
- A Java array can store only a fixed set of elements.
- The two types of arrays are:
 - Arrays
 - Multi-Dimensional Arrays

Arrays Excel At	Arrays Limitation
• Code Optimization: – It optimizes the code, making the retrieval or sorting of data efficient. • Random Access: – We can get any data located at an index position.	• Size Limit: • We can store only a fixed number of elements in the array. • It doesn't grow its size at the runtime. • To solve this problem, a collection framework is used in Java which grows automatically.

Arrays (Cont.)

Advantages	Disadvantages
Efficient storage and access	Fixed size
Simple and easy to use	No built-in methods for manipulation
Performance	No automatic resizing
Memory efficiency	Homogeneous elements
Multidimensional support	Static type

Declaration and Instantiation

- **Array—Declaration:**
 - To declare an array variable, you specify the data type of the elements followed by '[]' and then the variable name.
 - Example : `int[] myArray;`
- **Array—Instantiation:**
 - To initialize an array, you need to allocate memory for it using the new keyword, followed by the data type, and the size of the array in square brackets.
 - Example : `int[] myArray = new int[5];` // Creates an array of size 5
- **Array—Declaration and Instantiation Together:**
 - Can also declare and instantiate an array at the same time as follows.
 - Example: `int[] myArray = new int[5];`

Initialization

Assigning Values:

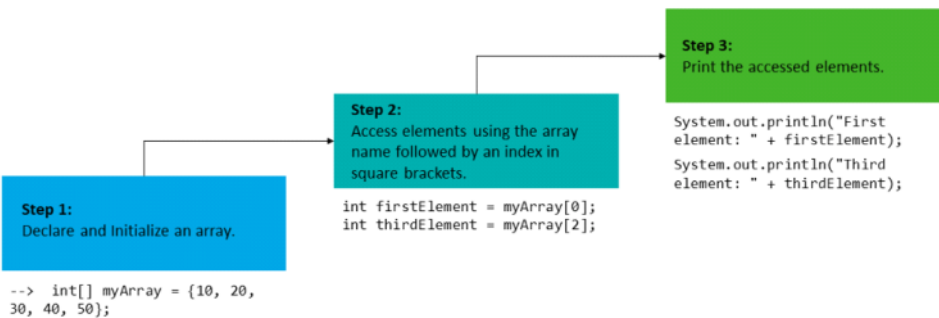
- Array is initialized by assigning values to the array elements.
- Initialization is done during declaration or later using the array index.
- Example:

```
// Declaration and Initialization by assigning values at same time
int[] myArray = {1, 2, 3, 4};

// Assigning values to the array later
myArray[0] = 1;
myArray[1] = 2;
myArray[2] = 3;
myArray[3] = 4;
```

Accessing Elements

- Each element in an array is assigned a unique index, which determines its position in the array.
- The indexing of an array starts from 0, meaning the first element of the array is at index 0, the second element is at index 1, and so on.



Note:

- Accessing an index that does not exist (an index > array length - 1) will throw `ArrayIndexOutOfBoundsException`.
- In the above example, accessing `myArray[5]` results in this exception as the array indices only go from 0 to 4.
- Array indexes—start from 0 and go up to the length of the array minus 1.
- So, the first element of the array is at index 0, and the last element is at index (array length - 1).

Array Length and Iterating Over Arrays

Array Length:

You can get the length of an array using the 'length' property.

```
int[] myArray = {10, 20, 30, 40, 50};
int length = myArray.length; // Length is 5
```

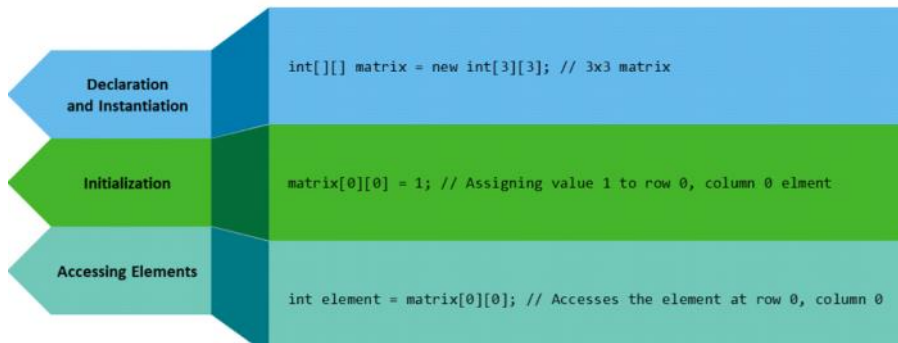
Iterating over Arrays:

You can iterate over arrays using loops like 'for' or enhanced 'for' loop.

```
int[] myArray = {10, 20, 30, 40, 50};
for (int i = 0; i < myArray.length; i++) {
    System.out.println(myArray[i]);
}
// Or using an enhanced for loop
for (int num : myArray) {
    System.out.println(num);
}
```

Multidimensional Arrays

Java supports multidimensional arrays, which are arrays of arrays.



Hands-On Activity 1

/>

Activity Details	
Problem Statement	Write a program that takes two inputs (integer/double/string) from the user and calls the method "add" (input1, input2). Create multiple methods (add) to accept any of the above datatype parameters. If both the inputs are numeric, then print the sum. If either of the inputs or both the inputs are of type string, then concatenate both the inputs and display.
Sample Input 1	5 4
Sample Input 2	1.1 2.2
Sample Input 3	Hello World

Hands-On Activity 2

>

Activity Details	
Problem Statement	Create a class "Car" with private properties "company," "model," "engineVersion," and "carType." Create three objects and populate the private fields using constructors: • Car(company) • Car(company, carType) • Car(engineVersion, CarType) Override the toString method and print the object.

Summary

Key learning points of the module:

- Java constructor chaining involves calling one constructor from another within the same class using 'this()' keyword, allowing initialization of objects with different parameters while avoiding code duplication. It's useful for reusing initialization logic and maintaining code conciseness but be cautious to avoid infinite loops or unintended side effects.
- The access modifiers in Java specifies the accessibility or scope of a field, method, constructor, or class.
- The main ideas behind Java's Object-Oriented Programming (OOP) concepts include abstraction, encapsulation, inheritance, and polymorphism.
- Arrays are efficient and simple data structures for storing collections of elements with fixed size and homogeneous data types. However, their lack of flexibility and dynamic resizing capabilities can be limiting in certain scenarios.

Object,String API,Wrappers,and Memory Management

Sunday, July 21, 2024 10:58 PM

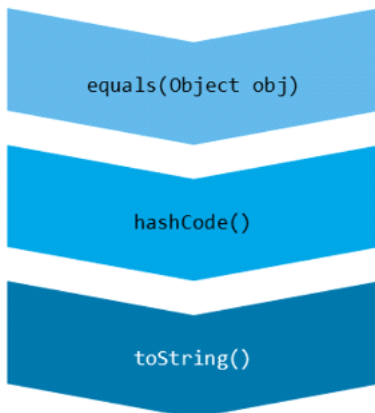
Topic	Descriptions
Object, String API and Wrapper Classes	Object Class and its Methods, String API, Wrapper Classes
Memory Management	Memory Collection in Java, Garbage Collection in Java

By the end of this session, you will be able to:

- Explain objects and use object methods
- Explain how strings may be used for representing a sequence of characters in the Java language
- Illustrate the use of string-related operations in Java programs
- Differentiate between String buffer and String builder
- Describe the concept of wrapper classes in Java
- Discuss Memory Management in Java

Object Class and its Methods

- In Java, the “Object” class is the root class for all Java classes.
- Every class in Java is a subclass of the “Object” class, either directly or indirectly.
- The “Object” class provides several methods that are available to all objects in Java.
- Here are some of the commonly used methods of the `Object` class:



- This method compares the current object with the specified object for equality.
- The default implementation in the “Object” class compares memory addresses, but it's often overridden in subclasses to provide custom equality checks.
- This method returns a hash code value for the object.
- It's used when storing objects in hash-based data structures like “HashMap”.
- It's recommended to override this method when the “equals” method is overridden.
- This method returns a string representation of the object.
- By default, it returns the class name followed by the memory address of the object.
- Subclasses often override this method to provide more meaningful string representations.

Object Class and its Methods (Cont.)

`getClass()`

- This method returns the runtime class of the object.
- It returns a "Class" object representing the class of the object.

`clone()`

- This method creates and returns a copy of the object.
- It's used to create a shallow copy of the object.
- To support cloning, the class must implement the "Cloneable" interface and override the "clone" method.

`finalize()`

- This method is called by the garbage collector before reclaiming the memory occupied by the object.
- Subclasses can override this method to perform cleanup operations before the object is garbage collected, but its use is discouraged in modern Java programming due to unpredictable behavior.

Object Class and its Methods (Cont.)

`wait(), notify(),
notifyAll()`

- These methods are used for inter-thread communication and synchronization.
- They are used in concurrent programming to coordinate the execution of threads.

- These are some of the key methods provided by the "Object" class in Java.
- Understanding these methods is essential for effective Java programming, as they are fundamental to the behavior of all objects in Java.

STRING API AND WRAPPER CLASSES

Strings and Their Operations

Strings

- Java strings are not primitive types.
- Java strings are objects and immutable.
- Two ways to create a string in Java:
 - Using a string literal (Example:
String name = "Java Language")
 - Using a new keyword (Example:
String name = new String ("Java Language")

Example:

Converting the case

- `name.toLowerCase()`
- `name.toUpperCase()`

Concatenating strings

- `String str = name + " is platform independent";`

Replacing parts of string

- `name.replace("Java", "Java Programming");`

Converting a string to array

- `name.getBytes()`
- `name.toCharArray()`

Converting primitives to strings

- `String.valueOf(123)`
- `String.valueOf(true)`

Parsing a string for primitive type

- `Integer.parseInt("123")`

String Buffer vs. String Builder

String Buffer and String Builder both are mutable in nature.

String Buffer

StringBuffer operations are thread-safe and synchronized, but lower in performance.

Example:

```
public class BufferTest{
    public static void main(String[]
    args){
        StringBuffer buffer=new
        StringBuffer("hello");
        buffer.append("world");
        System.out.println(buffer);
    }
}
```

String Buffer vs. String Builder

String Builder

StringBuilder operations are not thread-safe, but their performance is faster because there is no synchronization.

Example:

```
public class BuilderTest{
    public static void main(String[]
    args){
        StringBuilder builder=new
        StringBuilder("hello");
        builder.append("world");
        System.out.println(builder);
    }
}
```

Wrapper Classes/Autoboxing and Auto Unboxing

Wrapper classes

- Wrapper classes are used to convert Java primitives into reference types (objects).
- Since Java 5, the autoboxing and unboxing features automatically convert primitives into objects and objects into primitives.
- The automatic conversion of primitive into an object is known as autoboxing and vice-versa is auto unboxing.

Autoboxing and Auto Unboxing

- Constructing the object using a new operator is autoboxing.
- Extracting the value from the object is unboxing.

Example:

Primitive Data Type	Wrapper Class
char	Character
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
boolean	Boolean

Which method of class StringBuffer is used to concatenate the string representation to the end of invoking string?

concat()

B. append()

join()

D. concatenate()

Hands-On Activity 1

>

Activity Details	
Problem Statement	Write a program that prints if the given input string is a palindrome or not.
Sample Input 1	Malayalam
Sample Input 2	Hello

Hands-On Activity 2

>

Activity Details	
Problem Statement	Write a program to read a sentence and remove words that are used more than once, irrespective of the casing.
Sample Input	I can can I do this Do do this do do this this

Memory Allocation in Java

- Process of setting aside virtual memory sections in a program for storing the structures/classes instances and variables.
- However, it is just the reference that gets created at the time of declaration. There is no memory allocation that happens.
- The `new()` keyword is used to allocate heap memory to an object.
- The Java memory allocation is divided into the following sections for effective management:
 - **Heap:** Contains objects
 - **Stack:** Contains methods, local variables, and reference variables
 - **Code:** Contains your bytecode
 - **Static:** Contains static data/methods

Garbage Collection in Java

- Java programs perform automatic memory management through the process of garbage collection.
- Java programs that are compiled to bytecode can be run on a JVM for a short span of time.
- When running on the JVM, Java programs create objects on the heap, which dedicates a portion of memory to the program. Eventually, objects will not be needed.
- To free up memory, the garbage collector finds unused objects and deletes them.

Benefits of Java Garbage Collection

The garbage collector (part of the JVM) automatically removes unreferenced objects from the heap memory making Java more memory efficient.

Which of the following models is a garbage collection technique?

A. Cleanup Model

B. Mark and Sweep Model

C. Space Management Model

D. Sweep Model

Summary

Key learning points of the module:

- The 'Object' class in Java serves as the superclass for all classes, providing fundamental methods like 'equals', 'hashCode', and 'toString'. Subclasses inherit these methods and can override them to customize behavior such as equality comparisons and string representations.
- A wrapper class converts a primitive into an object and an object into a primitive. The automatic conversion of primitive into an object is known as autoboxing and vice-versa is unboxing.
- Process of setting aside virtual memory sections in a program for storing the structures/classes instances and variables.
- Java programs perform automatic memory management through the garbage collection process.

JAVA COLLECTION FRAMEWORK

Sunday, July 21, 2024 11:13 PM

Topic	Descriptions	Duration
Java Collection Framework	Collection Hierarchy, Advantages of Collection Framework, List, Set, Queue, Map, Iterator, ListIterator, Generics, Comparable, and Comparator	8 hours

Learning Objectives

By the end of this session, you will be able to:

- Explain the Java collections framework and its hierarchy
- Describe generics and their benefits
- Discuss collection framework algorithms
- Analyze and make use of appropriate collection(s) and related algorithm(s) to group, store, retrieve, and manipulate multiple elements/objects into a single unit in Java programs/applications

Collections framework , generics

What Are Collections?

- A collection, which is also known as a container, is simply an object that groups multiple elements into a single unit.
- Collections are used to store, retrieve, manipulate, and communicate aggregate data.
- Typically, they represent data items that form a natural group. Examples of collections include a poker hand (a collection of cards), a mail folder (a collection of letters), or a telephone directory (a mapping of names to phone numbers).

Collection Framework Overview

A collection framework is a unified architecture for representing and manipulating objects. It contains the following parts:

Interfaces:

- These are the abstract data types to represent the collection.
- `java.util.Collection` is the root interface of the Java collection framework.
- Examples include list, set, queue, and map.

Implementation Classes:

- These are the concrete implementation classes for collections.
- Some important Java collection classes are `ArrayList`, `LinkedList`, `HashMap`, `TreeMap`, `HashSet`, and `TreeSet`.

Algorithms:

- These are the methods that perform useful computations, such as searching, sorting, and shuffling.

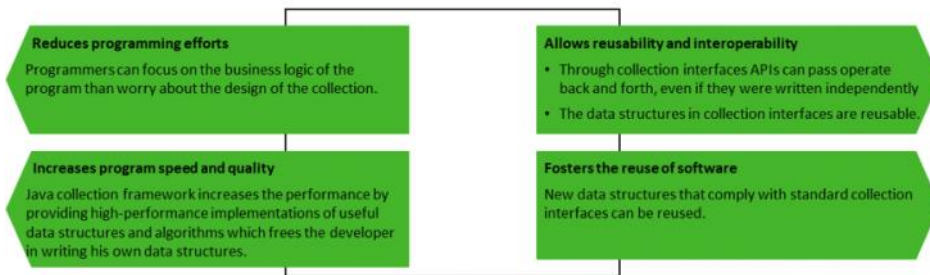
Java collection classes : `ArrayList` , `LinkedList` , `HashMap` , `TreesMap` , `HashSet` , `TreeSet`

Java Collection Framework

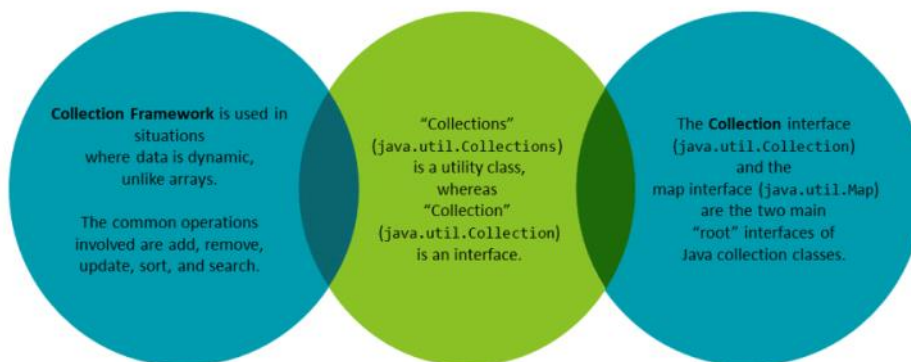


- Java Collection API provides a set of interfaces and classes to store data dynamically.
- It also allows quick data manipulation as compared to an array.

Benefits of Java Collection Framework



Collections vs. Collection

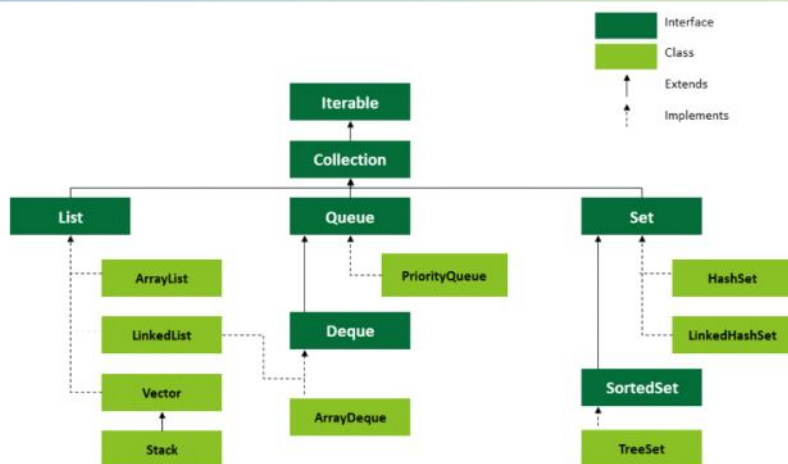


Collection framework is used in situations when data is dynamic unlike arrays. Add , remove , update , sort and search are the common operation

Collections is a utility class whereas collection is an interface

Collection interface and map interface are the two main root interfaces of java collection classes.

Collection Hierarchy



Which of these packages contains all the collection classes?

A. `java.lang`

B. `java.util`

C. `java.net`

D. `java.awt`

Which of these interfaces is NOT part of the Java Collection Framework?

A. `List`

B. `SortedList`

C. `SortedList`

D. `Set`

What does collection refer to in Java?

A. A group of objects

B. A group of interfaces

C. A group of classes

D. None of the above

List

Definition:

- The `List` interface is a core part of Java's Collection Framework.
- It is a child interface of the `Collection` interface and implements the `Iterable` interface.
- The `List` interface provides a way to store the ordered collection.

Key Characteristics

- **Order:**
 - Lists in Java maintain the order of elements based on the sequence in which they were added.
- **Index-based Methods:**
 - Lists provide index-based methods to insert, update, delete, and search the elements.
- **Duplicates:**
 - Lists may contain duplicate elements.
- **Nulls:**
 - Lists in Java allow null elements.

Sub-interfaces and Implementing Classes

- There are two main implementing classes of the List interface—ArrayList and LinkedList.
- ArrayList and LinkedList are widely used in Java programming due to their functionality and flexibility.

Usage

- Lists are commonly used data structures in Java. They can be used to store an ordered collection of items, which may include duplicates.
- They are particularly useful when you need constant-time access to individual elements in your collection, as they support positional access and insertion of elements.

Types of Lists

ArrayList

- ArrayList is the implementation of a List interface, where the elements can be dynamically added or removed from the list.
- The size of the list increases dynamically—it increases if more elements are added than the initial size.
- It is not thread-safe.
- Syntax: `ArrayList object = new ArrayList ();`

Linked List

- A linked list is a sequence of links containing items.
- Each link contains a connection to another link.
- Java Linked List classes use two types of lists to store elements:
 - Singly Linked List
 - Doubly Linked List
- It is an ideal choice to implement stacks—Last In First Out (LIFO) or queues—First In First Out (FIFO).
- Syntax: `LinkedList object = new LinkedList();`

Vector

- Vectors are like arrays, where the elements of the vector object can be accessed via an index into the vector.
- A Vector is the same as an ArrayList except with all its methods synchronized, where synchronization enables us to prevent multiple threads accessing the same instance at the same time.
- Syntax: `Vector object = new Vector(size, increment)`

Array List, Linked List, and Vector—Illustration

Array List

```
import java.util.*;
public class ArrayListExample {
    public static void main(String
args[]) {
        List<Integer> marks = new
ArrayList<> ();
        marks.add (56);
        marks.add (62);
        marks.add (79);
        System.out.println("Traversal
using for");
        for(Integer rank: marks)
            System.out.println(rank);
        System.out.println("Traversal
using iterator");
        Iterator<Integer> itr =
marks.iterator();
        while (itr.hasNext())
            System.out.println(itr.next());
    }
}
```

Linked List

```
import java.util.*;
public class LinkedListExample {
    public static void main(String
args[]) {
        List<Integer> marks = new
LinkedList<> ();
        marks.add (56);
        marks.add (62);
        marks.add (79);
        System.out.println("Traversal
using for");
        for(Integer rank: marks)
            System.out.println(rank);
        System.out.println("Traversal
using iterator");
        Iterator<Integer> itr =
marks.iterator();
        while (itr.hasNext())
            System.out.println(itr.next());
    }
}
```

Vector

```
import java.util.*;
public class VectorExample {
    public static void main(String
args[]) {
        List<Integer> marks = new
Vector<> ();
        marks.add (56);
        marks.add (62);
        marks.add (79);
        System.out.println("Traversal
using for");
        for(Integer rank: marks)
            System.out.println(rank);
        System.out.println("Traversal
using iterator");
        Iterator<Integer> itr =
marks.iterator();
        while (itr.hasNext())
            System.out.println(itr.next());
    }
}
```

LIST API AND ITERATORS

List API and Iterators

Index-Based Methods

- The List interface provides a variety of methods to insert, update, delete, and search elements based on their index.
- The index of elements in a List starts from 0 (just like an array).
- Some of the important index-based methods are:
 - `add(int index, E element)`
 - `get(int index)`
 - `set(int index, E element)`
 - `remove(int index)`

Iterator

- Iterator interface is a tool for traversing through collections in Java, such as List.
- It is a Universal Iterator as it can be applied to any Collection object.
- It can pass the Iterator object as a value parameter to a method and then use the Iterator methods to access the elements in the collection.
- It has methods like `hasNext()`, `next()`, and `remove()`.

List Iterator

- The List interface provides a special iterator, the `ListIterator`, that allows element iteration in both the forward and backward directions.
- The `listIterator()` method returns a `ListIterator` object which can be used to traverse the List in either direction.

`Add(index,element)` , `get(int index)` , `set(index , element)` , `remove(int index)` .

Iterator methods : hasNext() , next() , remove()

ListIterator methods : listIterator() method returns a ListIterator object which can be used to traverse the List in either direction.

Queue

Definition:

- It is used to hold the elements about to be processed in FIFO (First In First Out) order.
- It is an ordered list of objects with its use limited to insert elements at the end of the list and deleting elements from the start of the list, (i.e., it follows the FIFO or the First-In-First-Out principle).

Key Characteristics

- **FIFO (First-In-First-Out):**
 - Queues in Java are designed to follow the FIFO (First-In-First-Out) concept meaning that the element inserted first will be removed first.
- **Element Insertion:**
 - Elements are inserted at the tail using the add() or offer() methods.
- **Element Removal:**
 - Elements are removed from the head using the remove() or poll() methods.
- **Element Retrieval:**
 - element() or peek() methods are used to retrieve the head of the Queue without removing it.

Add(), offer() **tail** , remove(), poll() **head**, element() or peek() methods ARE used to retrieve the head of the queue without removing it

Queue (Cont.)

Sub-interfaces and Implementing Classes

Some of the classes that implement the Queue interface include LinkedList, PriorityQueue, and ArrayDeque.

Usage

- Queues are often used in multi-threaded programs to synchronize tasks.
- In order processing and similar scenarios, as orders come in, get added to the Queue and processed in the arrival order.
- In resource management, for example, in a print spooler where multiple print requests are sent concurrently, a queue can manage these requests in the order they were received.

LL,PQ,AD

Stack vs. Queue

- A **stack** follows the LIFO principle, which means the element inserted at the last is the first element to come out.
- The stack is the subclass of Vector, and supports methods like push(), pop(), where:
 - Insert operation is called push; and
 - Delete operation is called pop.
- In stack we always keep track of the last element present in the list with a pointer called top.
- A stack is simpler than a queue.

Stack
vs.
Queue

- A **queue** interface maintains the FIFO order.
 - Supports methods like peek(), poll() and remove()
- The following classes implement the queue interface:
 - PriorityQueue class: Holds the elements or objects which are to be processed by their priorities; and implements the queue interface
 - Deque: Stands for a double-ended queue which enables us to perform the operations at both the ends like the deck (as it is pronounced) of cards; and extends the queue interface
 - ArrayDeque: Implements the deque interface. Unlike queue, we can add or delete the elements from both ends

Stack, Queue, and Deque—Illustration

/>

Stack

```
import java.util.*;
public class StackExample {
    public static void main(String
args[]) {
        Stack<Integer> marks = new
Stack<> ();
        marks.push (56);
        marks.push (62);
        marks.push (79);
        System.out.println("Traversal
using for");
        for(Integer rank: marks)
            System.out.println(rank);
        System.out.println("Traversal
using iterator");
        Iterator<Integer> itr =
marks.iterator();
        while (itr.hasNext())
            System.out.println(itr.next());
        System.out.println("Popping the
elements");
        while (!marks.isEmpty())
            System.out.println(marks.pop());
    }
}
```

Queue

```
import java.util.*;
public class QueueExample {
    public static void main(String args[]) {
        Queue<Integer> marks = new
PriorityQueue<>();
        marks.add (56);
        marks.add (62);
        marks.add (79);
        System.out.println("Traversal
using for");
        for(Integer rank: marks)
            System.out.println(rank);
        System.out.println("Traversal
using iterator");
        Iterator<Integer> itr =
marks.iterator();
        while (itr.hasNext())
            System.out.println(itr.next());
        System.out.println("Polling the
elements");
        while (!marks.isEmpty())
            System.out.println(marks.poll());
    }
}
```

Deque

```
import java.util.*;
public class DequeExample {
    public static void main(String
args[]) {
        Deque<String> deque = new
ArrayDeque<String>();
        deque.add ("abc");
        deque.add ("def");
        deque.add ("ghi");
        deque.addFirst ("First
element");
        deque.addLast ("Last");
        System.out.println("Traversal
using for");
        for(String str: deque)
            System.out.println(str);
    }
}
```

Set

Definition:

- The Set interface in Java represents a collection that does not contain duplicate elements.
- This interface models the mathematical set abstraction and is used to manipulate sets of elements.
- It extends the Collection interface.

Key Characteristics

- **No Duplicate Elements:**
 - Set does not support storing duplicate values.
 - This is the primary distinction between a Set and other forms of the Collection like List and Map.
- **One Null Element:**
 - In a Set, null element can be stored, but you can insert it only once.
- **Unordered and Unindexed:**
 - A Set in Java does not maintain insertion order and also not indexed. So, they do not offer positional access.

Set (Cont.)

Implementing Classes

The Set interface has various Implementation classes available like HashSet, LinkedHashSet, EnumSet, TreeSet, etc.,

Usage

- It is used for storing a set of unique values like IDs or usernames where each ID or username must not be repeated.
- It supports mathematical set operations like union, intersection, and subset, where you might have a set of employees in one department and another set in a different department, and you want to find employees who are in both (intersection), or all employees (union), etc.

HashSet, TreeSet, EnumSet, LinkedHashSet.

It support operations like union, interaction, subset

Set - Implementations

</>

Types of Set Implementation



HashSet

- HashSet only contains unique elements.
- It inherits the AbstractSet class and implements the set interface.
- It uses a mechanism known as hashing to store the elements.



TreeSet

- Java TreeSet class implements the set interface which uses a tree for storage.
- The objects of this class are stored in ascending order and use binary search.
- It inherits AbstractSet class and implements the NavigableSet interface.
- It contains unique elements such as HashSet.



LinkedHashSet

- Java LinkedHashSet class is a hash table and linked list implementation of the set interface.
- It contains unique elements such as HashSet and maintains the insertion order.

HashSet : unique elements ,it inherits abstract class and implement set interface , hashing mechanism to store elements

TreeSet : ascending order , use binary search , inherits abstract class and implements Navigable Set interface , contains unique elements as the HashSet

Linked HashSet : LinkedHashSet class is a hash table and linked list implementation of the set interface , order - insertion order

HashSet, LinkedHashSet, and TreeSet—Illustration

</>

HashSet

```
import java.util.*;
public class HashSetDemo {
    public static void main(String
args[]) {
    // Creating HashSet and adding
elements
    HashSet<String> hs = new
HashSet<String>();
    hs.add("Java");
    hs.add("Collections");
    hs.add("Is");
    hs.add("Very");
    hs.add("Helpful");
    // Traversing elements
    Iterator<String> itr =
hs.iterator();
    while (itr.hasNext())
        System.out.println(itr.next());
    }
}
```

LinkedHashSet

```
import java.util.*;
public class LinkedHashSetDemo {
    public static void main(String
args[]) {
    // Creating LinkedHashSet and adding
elements
    HashSet<String> lhs = new
LinkedHashSet<String>();
    lhs.add("Java");
    lhs.add("Collections");
    lhs.add("Is");
    lhs.add("Very");
    lhs.add("Helpful");
    // Traversing elements
    Iterator<String> itr =
lhs.iterator();
    while (itr.hasNext())
        System.out.println(itr.next());
    }
}
```

TreeSet

```
import java.util.*;
public class TreeSetDemo {
    public static void main(String
args[]) {
    // Creating TreeSet and adding
elements
    Set<String> ts = new
TreeSet<String>();
    ts.add("Java");
    ts.add("Collections");
    ts.add("Is");
    ts.add("Very");
    ts.add("Helpful");
    // Traversing elements
    Iterator<String> itr =
ts.iterator();
    while (itr.hasNext())
        System.out.println(itr.next());
    }
}
```

Do you Know?

- **HashSet** implements Set (no duplicates, no order).
- **LinkedHashSet** implements Set (no duplicates but ordered).
- **TreeSet** implements NavigableSet (no duplicates, sorted in the natural order).

Set	Allows unique values but no duplication
List	Cares about which position each object is in
Queue	Arranges in the order of processing (for example, to-do list)



MAP

Map

Definition:

- Map interface in the Java Collections Framework represents a mapping between a set of keys and a set of values.
- Each key is linked to a specific value.
- Once stored in a Map, you can access a value using its corresponding key.

Key Characteristics

- **Key-Value Pairs:**
 - Map stores data in key-value pairs, also known as entries.
 - Each entry consists of a unique key and a corresponding value associated with that key.
- **Unique Keys:**
 - In a Map, each key must be unique. However, values can be duplicated.
- **Null Keys and Values:**
 - Depending on the specific implementation, a Map may allow one null key and multiple null values.
- **No Intrinsic Order:**
 - Maps don't guarantee any specific order of entries (except few sub-classes).

Keys unique, values duplicate, one null key and multiple null values allowed, no intrinsic order except for few subclasses

Map (Cont.)

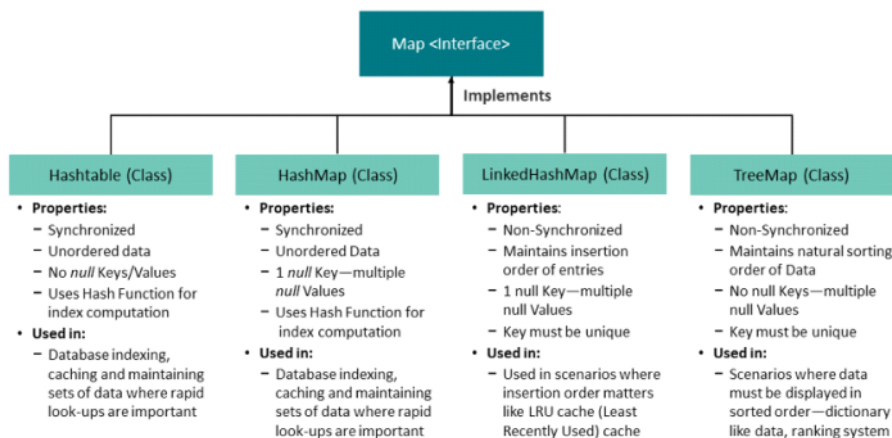
Implementing Classes

Classes that implement the Map interface include HashMap, TreeMap, and LinkedHashMap.

Usage

- Map is used to create dictionary-like data structures, where keys are words and values are their definitions.
- For quick Data Searches, e.g., a database of students where the key is student id and value is the student object.
- Map is used to count the frequency of elements in a collection.
- For instance, in a list of names, you could use a Map to store the frequency of each name where keys are the names and values are their frequencies.

Map—Implementations (Diagrammatic View)



HashMap, TreeMap, LinkedHashMap

To count the frequency and for data searches.

HashTable - no null keys/values,

HashMap - 1 null key, multiple null values

Both HT and HM, Use Hash Function for index computation

LinkedHashMap : 1 null key, multiple null values

TreeMap : no null key, multiple null values.

Keys must be unique for both T, M, LHM.

HashMap and Hashtable—Illustration

/>

HashMap

```
import java.util.*;
public class HashMapDemo {
    public static void main(String args[]) {
        // Creating HashMap and adding elements
        Map<Integer, String> hm = new
HashMap<>();
        hm.put(1, "Java");
        hm.put(2, "Servlets");
        hm.put(3, "JSP");
        // Finding the value of a key
        System.out.println("Value for 1 is " +
hm.get(1));
        // Traversing through the HashMap
        for (Map.Entry<Integer, String> e :
hm.entrySet())
            System.out.println(e.getKey() + " " +
e.getValue());
    }
}
```

Hashtable

```
import java.util.*;
public class HashtableDemo {
    public static void main(String args[]) {
        // Creating Hashtable and adding elements
        Map<Integer, String> ht = new
Hashtable<>();
        ht.put(1, "Java");
        ht.put(2, "Servlets");
        ht.put(3, "JSP");
        // Finding the value of a key
        System.out.println("Value for 1 is " +
ht.get(1));
        // Traversing through the Hashtable
        for (Map.Entry<Integer, String> e :
ht.entrySet())
            System.out.println(e.getKey() + " " +
e.getValue());
    }
}
```

HashMap vs. Hashtable

HashMap

- Is non-synchronized, which means it is not thread-safe
- Allows only one null key and multiple null values
- Is fast
- Is traversed by iterator
- Inherits AbstractMap class

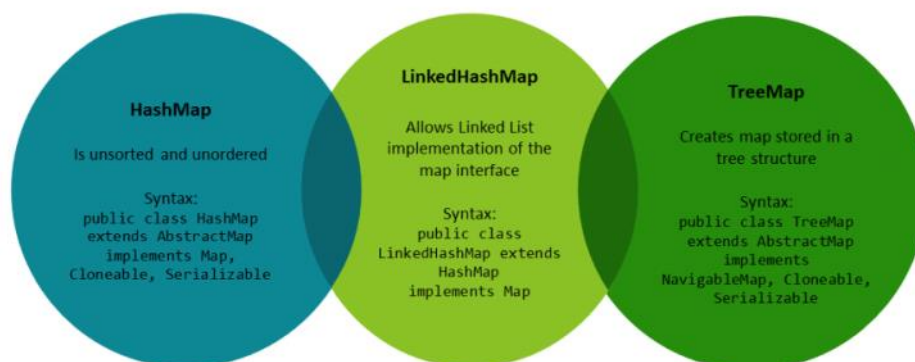
Hashtable

- Is synchronized and can be shared with many threads
- Does not allow null key or its value
- Is slow
- Is traversed by enumerator and iterator
- Inherits the Dictionary class

HashMap is generally preferred over Hashtable, if thread synchronization is not needed.

A hash table has a descending order of storing elements **maybe**

HashMap vs. LinkedHashMap vs. TreeMap



HashMap vs LinkedHashMap vs TreeMap—Illustration

</>

HashMap

```
import java.util.*;
class HashMapDemo
{
    // This function prints ordering of
    // all elements
    static void
    insertAndPrint(AbstractMap<Integer,
    String> map)
    {
        int[] array= {1, -1, 0, 2,-2};
        for (int x: array)
            map.put(x,
            Integer.toString(x));
        for (int k: map.keySet())
            System.out.print(k + ", ");
    }
    // Driver method to test above method
    public static void main (String[]
    args)
    {
        HashMap<Integer, String> map =
        new HashMap<Integer, String>();
        insertAndPrint(map);
    }
}
```

LinkedHashMap

```
import java.util.*;
class LinkedHashMapDemo
{
    // This function prints ordering of
    // all elements
    static void
    insertAndPrint(AbstractMap<Integer,
    String> map)
    {
        int[] array= {1, -1, 0, 2,-2};
        for (int x: array)
            map.put(x,
            Integer.toString(x));
        for (int k: map.keySet())
            System.out.print(k + ", ");
    }
    // Driver method to test above method
    public static void main (String[]
    args)
    {
        LinkedHashMap<Integer, String>
        map = new LinkedHashMap<Integer,
        String>();
        insertAndPrint(map);
    }
}
```

TreeMap

```
import java.util.*;
class TreeMapDemo
{
    // This function prints ordering of
    // all elements
    static void
    insertAndPrint(AbstractMap<Integer,
    String> map)
    {
        int[] array= {1, -1, 0, 2,-2};
        for (int x: array)
            map.put(x,
            Integer.toString(x));
        for (int k: map.keySet())
            System.out.print(k + ", ");
    }
    // Driver method to test above method
    public static void main (String[]
    args)
    {
        TreeMap<Integer, String> map =
        new TreeMap<Integer, String>();
        insertAndPrint(map);
    }
}
```

34

List vs. Set vs. Queue vs. Map

>

Property	List	Set	Queue	Map
Ordering	Maintains insertion order	Does not maintain any order	Maintains order (FIFO or priority based)	Does not maintain any order (except some implementations like LinkedHashMap or TreeMap)
Duplicate Elements	Allows duplicates	Does not allow duplicates	Allows duplicates	Does not allow duplicate keys, allows duplicate values
Null Elements	Allows null elements	Allows null elements (only once in a Set)	Allows null elements	Allows null values, and one null key
Data Structure	Linear data structure	Mathematical set	Linear data structure	Key-Value pairs
Use Case	When order matters and duplicates are allowed	When you want to store unique elements	When elements need to be processed in a particular order (FIFO or priority)	When you want to map keys to values
Main Implementations	ArrayList, LinkedList	HashSet, TreeSet	PriorityQueue, ArrayDeque	HashMap, TreeMap

Iterator and Iterable Interfaces

Iterator Interface (java.util.iterator)

- An interface that iterates the elements
- Used to modify the elements and traverse the list
- Has three methods:
 - **public boolean hasNext()**—Returns true if the iterator has more elements
 - **public Object next()**—Returns the element and moves the cursor pointer to the next element
 - **public void remove()**—Removes the last elements returned by the iterator
- The Iterator interface is also known as Universal Java Cursor as it is applicable for all the classes of the collection framework.

Iterable Interface (java.lang.iterable)

- The Iterable interface provides only one method which returns an iterator.
- The Iterator iterator() returns an iterator.

An interface that iterates the elements. Used to modify and traverse the list, has three methods: `boolean hasNext()`, `Object next()`: returns the element and moves the cursor to the next point, `void remove()`: removes the last elements returned by the iterator

Iterable interface provides only one method which returns an iterator, the `Iterator iterator()` returns an iterator

Iterator vs. ListIterator Interfaces

/>

Both Iterator and ListIterator are utilized for traversing or iterating over the elements of collection classes.

Property	Iterator	ListIterator
Type	Interface	Interface and it extends 'Iterator'
Direction of Traversal	Can only traverse in forward direction	Can traverse in both forward and backward directions
Add Elements	Does not have the ability to add elements to the collection	Can add elements to the collection while traversing
Replace Elements	Does not have the ability to replace elements in the collection	Can replace the last element returned by next or previous methods
Indexing	Does not support indexing	Supports indexing—can return the index of the element that would be returned by a subsequent call to the next or previous
Usage	Can be used with all Collection Framework classes	Can only be used with List and its implementations
Cursor Movement	Only one way cursor movement	Two-way cursor movement (forward and backward)
Methods	Supports with methods like 'hasNext()' and 'next()'	Provides additional methods like 'hasPrevious()', 'previous()', 'add()', 'set()', and 'remove()'

hasPrevious(), previous(), add(), set(), remove() ..

Iterator vs. ListIterator Example

```
public class IteratorExample {
    public static void main(String[] args) {
        // Creating a list
        List<String> myList = new ArrayList<>();
        myList.add("Java");
        myList.add("Python");
        myList.add("JavaScript");

        // Using Iterator
        Iterator<String> iterator = myList.iterator();
        while (iterator.hasNext()) {
            String language = iterator.next();
            System.out.println(language);
        }
    }
}
```

```
// Using ListIterator for bidirectional traversal
ListIterator<String> listIterator =
myList.listIterator(myList.size());
System.out.println("Printing list in reverse:");
while (listIterator.hasPrevious()) {
    String language = listIterator.previous();
    System.out.println(language);
}
}
```

Which of these interfaces must contain a unique element?

A. Set

B. List

C. Array

D. Collection

Which of these methods can convert an object into a list?

A. SetList()

B. ConvertList()

C. SingletonList()

D. CopyList()

SingletonList()

A map implements the collection interface.

Yes

No

No

What should you use to sort elements of ArrayList?

A. `Collection.sort(listObj);`

B. `listObj.sort();`

C. `Collections.sort(listObj);`

D. None of the above

What is the return type of the hasNext() method of an iterator?

A. Integer

B. Double

C. Boolean

D. Collections Object

Is the PriorityQueue thread-safe?

Yes

No

Which of these interfaces must contain a unique element?

A. Set

B. List

C. Array

D. Collection

Which of these methods can convert an object into a list?

A. SetList()

B. ConvertList()

C. SingletonList()

D. CopyList()

Which of these interfaces handle sequences?

A. Set

B. Comparator

C. List

D. Collection

Which of these options is a static variable defined in collections?

A. EMPTY_SET

B. EMPTY_LIST

C. EMPTY_MAP

D. All of the above

Generics—Overview

- In a nutshell, generics enable types (classes and interfaces) to be the parameters when classes, interfaces, and methods are defined.
- Before generics, we could store any type of object in the collection (non-generic).
- Now generics force Java programmers to store a specific type of object.
- T: Type
- E: Element (Examples: ArrayList and Set)
- K: Key (Used in Map)
- N: Number
- V: Value (Used in Map)

What Is Generics?

Type Parameters Naming Convention

Why Generics?

- Provides type safety
- Does not require type casting
- Performs compile-time checking
- Avoids ClassCastException
- Reuses codes
- Implements generic algorithms

Types of Generics

- Generics Class
- Generic Interface
- Generic Type
- Generic Method

Generics—Benefits



Benefits of Generics

Provides type safety

- Generics force type safety in the Java language.
- We can hold only a single type of object in generics. It does not allow the storage of other objects.
- Without generics, we can store any type of objects.

Does not require type casting

- There is no need to type cast the object. Before generics, we need to type cast.
- If we already know that our list only holds string data, then we need not type cast it every time.

Performs compile-time checking

- Generics make errors appear at the compile time instead of the run time.
- Generics add stability to your code by making more of your bugs detectable at the compile time.
- Generics were added in Java 5 to provide compile-time type checking and remove the risk of `ClassCastException` that was common with collection classes.

Generics Using ArrayList and Map—Illustration

Generics Using ArrayList

```
import java.util.*;

class GenericsUsingArrayList {
    public static void main(String args[]) {
        ArrayList<String> list=new ArrayList<String>();
        list.add("abc");//works fine
        list.add("def");
        //list.add(32);//compile time error
        String s=list.get(1);//type casting is not required
        System.out.println("element is: "+s);
        Iterator<String> itr=list.iterator();
        while(itr.hasNext()) {
            System.out.println(itr.next());
        }
    }
}
```

Generics Using Map

```
import java.util.*;

class GenericsUsingMap {
    public static void main(String args[]) {
        Map<Integer,String> map=new HashMap<Integer,String>();
        map.put(1,"abc");
        map.put(4,"def");
        map.put(2,"gjl");
        //Now use Map.Entry for Set and Iterator
        Set<Map.Entry<Integer,String>> set=map.entrySet();
        Iterator<Map.Entry<Integer,String>> itr=set.iterator();
        while(itr.hasNext()){
            Map.Entry e=itr.next();//no need to typecast
            System.out.println(e.getKey()+" "+e.getValue());
        }
    }
}
```

Generic Class—Illustration

Java Generic Class

```
// We use < > to specify Parameter type
class Test<T> {
    // An object of type T is declared
    T obj;
    Test(T obj) {
        this.obj = obj; // constructor
    }
    public T getObject() {
        return this.obj;
    }
}

class GenericsClassExample1 {
    public static void main (String[] args) {
        // instance of Integer type
        Test<Integer> iObj = new Test<Integer>(25);
        System.out.println(iObj.getObject());
        // instance of String type
        Test<String> sObj = new Test<String>("Java Generics");
        System.out.println(sObj.getObject());
    }
}
```

Java Generic Class (Multiple Type Parameters)

```
// We use < > to specify Parameter type
class Test<T, U> {
    T obj1; // An object of type T
    U obj2; // An object of type U
    Test(T obj1, U obj2) {
        this.obj1 = obj1; // constructor
        this.obj2 = obj2;
    }
    // To print objects of T and U
    public void print() {
        System.out.println(obj1);
        System.out.println(obj2);
    }
}

class GenericsClassExample2 {
    public static void main (String[] args) {
        Test<String, Integer> obj = new Test<String, Integer>("Generics", 3);
        obj.print();
    }
}
```

Generic Method and Generic Interface—Illustration

Java Generic Method—Example

```
public class GenericsMethodExample {  
    public static < E > void printArray(E[] elements) {  
        for ( E element : elements){  
            System.out.println(element );  
        }  
        System.out.println();  
    }  
  
    public static void main( String args[] ) {  
        Integer[] intArray = { 10, 15, 20, 25, 30 };  
        Character[] charArray = {'A','B','C'};  
        System.out.println( "Printing Integer Array"  
    );  
        printArray( intArray );  
        System.out.println( "Printing Character Array"  
    );  
        printArray( charArray );  
    }  
}
```

Java Generic Interface

"Comparable interface is a great example of Generics in interfaces"

```
package java.lang;  
import java.util.*;  
  
public interface Comparable<T> {  
  
    public int compareTo(T o);  
}
```

Comparable vs. Comparator

Comparable Interface

- Provides a single sorting sequence, which means it can be sorted based on a single element such as ID or name
- Compares an object with other objects of the same type
- Is used to sort the objects with a natural ordering
- Compares "this" reference with the object specified
- Affects the original class, which means that the actual class is modified
- Provides only one method to sort elements:
 - int compareTo()
- Is present in the java.lang package
- Can sort the list elements of the comparable type by using the Collections.sort(List) method

java.lang.Comparable
and
java.util.Comparator
are powerful interfaces
that can be used to sort
the collection elements.

Comparator Interface

- Provides multiple sorting sequences, which means it can be sorted based on multiple elements
- Is used to order the objects of a specific class
- Is used to sort attributes of different objects
- Compares two different class objects provided
- Doesn't affect the original class, which means that the actual class is not modified
- Provides two methods to sort elements:
 - compare(Object obj1, Object obj2)
 - equals(Object element)
- Is present in the java.util package
- Can sort the list elements of Comparator type by the Collections.sort(List, Comparator) method

Implementation of Comparable and Comparator Interfaces

```
// Example class representing a book  
class Book implements Comparable<Book> {  
    private String title;  
    private int pageCount;  
  
    public Book(String title, int pageCount) {  
        this.title = title;  
        this.pageCount = pageCount;  
    }  
  
    public String getTitle() {  
        return title;  
    }  
  
    public int getPageCount() {  
        return pageCount;  
    }  
}
```

```
// Implementing compareTo method for natural ordering  
@Override  
public int compareTo(Book other) {  
    return this.pageCount - other.pageCount;  
}  
  
@Override  
public String toString() {  
    return "Book(" +  
        "title=" + title + ' ' +  
        ", pageCount=" + pageCount +  
        ')';  
}
```

Implementation of Comparable and Comparator Interfaces (Cont.)

```
public class Main {
    public static void main(String[] args) {
        // Creating a list of books
        List<Book> books = new ArrayList<>();
        books.add(new Book("Java in Action", 500));
        books.add(new Book("Introduction to Algorithms", 1000));
        books.add(new Book("Clean Code", 400));

        // Sorting books using Comparable (natural ordering)
        Collections.sort(books);
        System.out.println("Sorted books by page count (natural ordering):");
        for (Book book : books) {
            System.out.println(book);
        }

        // Sorting books using Comparator (title)
        Collections.sort(books, new TitleComparator());
        System.out.println("\nSorted books by title:");
        for (Book book : books) {
            System.out.println(book);
        }
    }
}
```

```
// Comparator implementation for sorting books by title
class TitleComparator implements Comparator<Book> {
    @Override
    public int compare(Book book1, Book book2) {
        return
            book1.getTitle().compareTo(book2.getTitle());
    }
}
```

Knowledge Check 14

Which of the following methods cannot be type parameterized?

A. Overloaded Methods

B. Generic Methods

C. Class Methods

D. Overriding Methods

Which of these type parameters is used for a generic method to return and accept any type of object?

A. K

B. N

C. T

D. W

Which of these data types cannot be parameterized?

A. Array

B. List

C. Map

D. Set

Which of these types cannot be used to initiate a generic type?

A. Integer Classes

B. Primitive Types

C. Float Classes

D. Collections

Which of these options is a wildcard symbol?

A. ?

B. !

C. %

D. &

Which of these keywords is used to upper bound a wildcard?

A. stop

B. bound

C. extends

D. implements

Which of the following keywords is used for lower bounding a wildcard?

A. Extends

B. Super

C. Lower

D. Class

What is the use of wildcards?

A. It is used in cases when the type that is being operated upon is not known.

B. It is used to make codes more readable.

C. It is used to access members of super classes.

D. It is used for type argument of the generic method.

Activity Details	
Problem Statement	Write a code for a map that associates the employees with the department. Let the keys of maps be an employee object and the associated value be a string.
Sample Input	N/A

Summary

Here are the key learning points of the module:

- The Java Collections Framework simplifies the management and manipulation of data structures in Java, offering a wide range of classes and interfaces for different collection types and providing algorithms and utilities for common collection operations. It is an essential part of Java programming and widely used in various applications and libraries
- The hierarchy of the entire collection framework consists of four core interfaces, such as Collection, List, Set, Map, and specialized interfaces.
- Collection: The root interface that defines the common behavior and operations for all collection types.
- List: An ordered collection that allows duplicate elements. Elements can be accessed by their index.
- Set: A collection that does not allow duplicate elements. There is no defined order for elements.
- Queue: A collection designed for holding elements prior to processing. It follows the FIFO (First-In-First-Out) principle.
- Map: An object that maps keys to values. Each key is unique, and the values can be retrieved using the associated key.
- if you need to iterate over elements in a list and require bidirectional traversal or the ability to modify the list while iterating, 'ListIterator' would be the better choice. Otherwise, if you just need to iterate forward over any collection, 'Iterator' suffices.

Summary (Cont.)

Here are the key learning points of the module (cont.):

- Generics enable types (classes and interfaces) to be the parameters when classes, interfaces, and methods are defined. It has several benefits, which include providing type safety, doesn't require casting and performing compile time checking.
- Thread Safety: The Java Collections Framework provides several synchronized implementations of collection classes, allowing multiple threads to safely access and modify collections concurrently.

GIT BASICS

Monday, July 22, 2024

10:35 PM

Learning Objectives

By the end of this session, you will have gained the ability to:

- Understand Git's role as a Version Control System (VCS)
- Master the terminology associated with Git
- Comprehend and apply the Git Workflow
- Grasp the basics of Git, including installation and command line usage
- Manage files effectively within Git
- Understand and utilize key Git processes
- Become familiar with Git Clients, with a particular emphasis on GitHub and its functionality.

What is Version Control?



Version control helps keep track of our work and helps easily explore the changes made to data, coding scripts, notes, etc.



Saving multiple files after each change like `main_script_1` or `main_script_v1` or `mainScript_Final`, etc., are types of version control.



Tens or hundreds of similar files will be left when you use the version control, making it cumbersome to directly compare different versions and share with the collaborators.



Version control becomes much smoother and easier to implement with help from software like GIT.

What is GIT?



Git is an open-source version control system or protocol used to track changes in files



Distributed version control systems like GIT give flexibility for version control, sharing, collaboration, and managing code in a consistent manner.

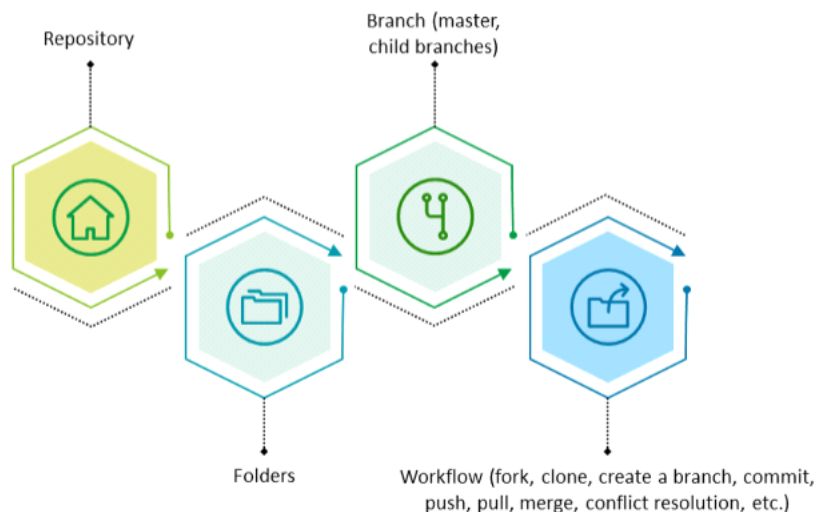


It allows collaboration among multiple developers working on the same project. Team members can publish, share, review, and iterate on code changes through GIT branches shared with others.

Key Concepts of GIT

- **Version Control:** Git allows you to keep track of the changes made to your code over time. Each change is recorded, along with who made the change and when.
- **Distributed:** Unlike centralized version control systems, each developer's working copy of the code is also a repository that can contain the full history of changes.
- **Snapshots:** Git thinks of data more like a series of snapshots of a miniature filesystem. With Git, every time you commit, or save the state of your project, Git basically takes a picture of what all your files look like at that moment and stores a reference to that snapshot.
- **Branching:** Branches in Git are lightweight and easy to create. They allow you to work on different features or bug fixes independently without affecting the main codebase.
- **Merging:** Git allows you to merge changes from one branch into another. This is useful when you have finished working on a feature branch and want to incorporate those changes into the main branch.

Basic Git Workflow



Git Workflow (Contd..)

1. **Initializing a Repository:** You start by creating a new Git repository or cloning an existing one.

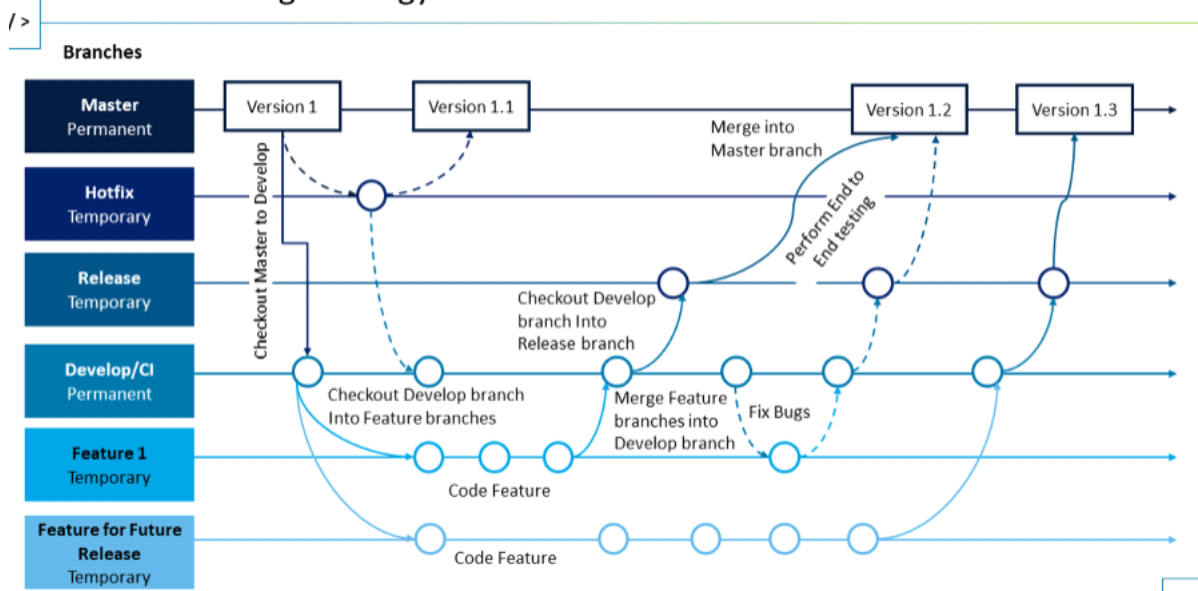
Git Workflow (Contd..)

1. Initializing a Repository: You start by creating a new Git repository or cloning an existing one.
2. Adding Changes: After making changes to your files, you stage them using `git add` to prepare them for committing.
3. Committing Changes: Once your changes are staged, you commit them to the repository using `git commit`. Each commit represents a snapshot of your code at a particular point in time.
4. Reviewing History: You can view the commit history using `git log` to see the changes made by you and others.
5. Branching and Merging: Create branches to work on new features or bug fixes. Use `git branch` to create branches and `git merge` to incorporate changes from one branch into another.
6. Working with Remotes: Git allows you to collaborate with others by pushing your changes to a remote repository (`git push`) and pulling changes from a remote repository (`git pull`).

Benefits of Git

- Collaboration: Git enables multiple developers to work on the same project simultaneously.
- Versioning: Git keeps track of all changes made to your code, allowing you to revert to previous versions if needed.
- Branching: Branches provide a way to isolate changes and work on them independently.
- Backup and Restore: Git repositories serve as a backup of your code, and you can restore previous versions easily.

GIT—Branching Strategy



Git command -help

Git help --all

Help commands

Git commit -help

Usage : git commit [] [--] ...

-q, --quiet suppress summary after successful commit

Git help -a-v, --verbose show diff in commit message template

Q exit and shift + g to jump at the end of the list.

Git Verification

To verify that Git has been installed and configured correctly, you can run the following command to see the version of Git installed on your system.

```
git --version
```

GitHub

GitHub is a popular platform for hosting and collaborating on Git repositories. Here's a brief overview of GitHub:

- **Repository Hosting:** GitHub allows users to host Git repositories online. You can create public or private repositories to store your code, documentation, and other project files.
- **Collaboration Tools:** GitHub provides tools for collaboration, such as issues, pull requests, and project boards. These features enable teams to track and manage tasks, discuss changes, and review code.
- **Version Control:** GitHub uses Git for version control, allowing you to track changes to your code over time. You can commit changes, create branches, merge code, and revert to previous versions as needed.
- **Community and Open Source:** GitHub has a large community of developers and hosts millions of open-source projects. You can discover projects, contribute to others' code, and build upon existing work.
- **Integration:** GitHub integrates with various tools and services, such as CI/CD platforms, code editors, project management tools, and more. This makes it easy to incorporate GitHub into your development workflow.

Git Configuration

Once Git is installed, you need to configure it with your username and email address. Open a terminal or command prompt and run the following commands, replacing 'Your Name' and 'your.email@example.com' with your actual name and email:

<pre>git config --global user.name "[your_name]"</pre>	In the current repository, an author's name is added to all the commits
<pre>git config --global user.email "[email_address]"</pre>	An email address is added to all the commits by the current user
<pre>git config --global alias.[alias_name] [git_command]</pre>	To create an alias or shortcut for commonly used Git command
<pre>git config --global color.ui auto</pre>	To enable Git's automatic command line coloring to easily track repository changes

These commands set your name and email address globally, so Git can associate your commits with your identity. If you want to override these settings for a specific project, you can run the commands without the `--global` flag within the project directory.

Initializing Repository

To start setting up GIT, we first need to initialize our repository.

Initializing Repository:

- Start using the GIT Bash terminal window to navigate to your project's directory, to initialize a GIT repository in an existing directory:

```
cd [directory path]
```
- Once you go to the project directory, initialize a GIT repository as below:

```
git init
```
- Initializing the repository creates a subdirectory called `.git` which has the files GIT needs to start tracking the changes made to the project files.
- Once we commit the changes in GIT, the repository starts tracking project versions.

Clone a Repository:

`git clone` command clones a current repository and copies the repository to your system.

```
git clone [url] [directory]
```

<pre>[url]</pre>	The URL of the Git repository we want to clone.
<pre>[directory]</pre>	The directory name we want to clone the repository into.

[url]	The URL of the Git repository we want to clone.
[directory]	The directory name we want to clone the repository into.

GIT—Commands

/>

git add <file/directory>	Stage all changes for the file/directory for the next commit
git add	Stage all changes for the next commit
git commit -m "Commit Message"	Commits the staged snapshot
git status	Lists which files are staged, unstaged, and untracked
git log --oneline	Using the default format, it displays the entire commit history
git revert <commit>	Creates a new commit which undoes all the change made in <commit>, then apply it to the current branch
git reset <commit>	Remove <file> from the staging area but leaves the working directory unchanged. This will unstage a file without overwriting any changes

GIT—Commands (Cont.)

/>

git branch	List the branches in the repo. Add a <branch> argument which will create a new branch with the name <branch>
git checkout -b <branch_name>	A new branch named <branch> create and check-out. The -b flag dropped will checkout an existing branch
git merge <branch>	Merge <branch> into the current branch
git fetch <remote> <branch>	Fetches a specific <branch> from the repository
git push origin <branch>	When the branch is pushed to <remote>, along with necessary commits and objects. A named branch is created in the remote repository if it doesn't exist
git pull <remote>	The current branch's specified remote copy is fetched and immediately merged into the local copy

Java Util And Time Packages

Tuesday, July 30, 2024 9:43 PM

Java Util Packages : Collection Class,Array Class, String Tokenizer

Java Time Package : Time Package Overview

Learning Objective :

Important components of java util package

Explain the usage of String Tokenizer

Java util package :

Provides multiple utility classes and APIs

Some of them to name are collections,arrays,String Tokenizers

Collection Algorithms :

These are defined in a class called **collections** ,that's defined in the java util package ,these algos are efficient and make coding simpler.

Sorting , searching , shuffling

Collection Algorithms

- Collection algorithms in Java are defined in a class called **Collections** that is defined in the `java.util` package.
- These algorithms are efficient and make coding simpler.



Sorting

Follows a natural order (ascending) by default, where the natural order is defined by the comparable interface



Searching

Provides iterative-based binary searching



Shuffling

Provides a random permutation of elements in the list

Collection ALGOs features and overview

Collection Algorithms—Overview and Features

Overview
- The Collections class is a useful utility class to work with the collection framework API. - The Collections class contains polymorphic algorithms that operate on the collection and the "wrapper" objects. - The Java Object class is the parent class in Java that marks the top level in the Java class hierarchy. - Every Java API class extends the Object class by default. - The Object class is beneficial especially when we are not sure of the type of object in which we want to store data.

Features Provided in Collections Class
Sort a collection framework class Collections.sort() method is used to sort any collection framework class. - In case a user-defined collection class is used, the user-defined class needs to implement a Comparable/Comparator interface. Provide a Binary Search Collections.binarySearch() method is used to search for an element using the binary search technique. - The searching mechanism is very fast as compared to linear/search by a linked list. Fill a value Collections.fill() method is used to replace all the elements of the specified list with the specified element.

Collection Algorithms :

```

import java.util.*
public class CollectionAlgorithmsExample {
    public static void main(String[] args) {
        ArrayList list = new ArrayList();
        PriorityQueue queue = new PriorityQueue();
        HashSet set = new HashSet();
        HashMap map = new HashMap();

        Random num = new Random();
        for(int i = 0; i < 5; i++) {
            list.add(num.nextInt(100));
            queue.add(num.nextInt(100));
            set.add(num.nextInt(100));
            map.put(i, num.nextInt(100));
        }

        System.out.println("List => " + list);
        System.out.println("Queue => " + queue);
        System.out.println("Set => " + set);
        System.out.println("Map => " + map);
        System.out.println("-----");
    }
}

```

```

Collections.sort(list);
System.out.println("List in ascending order => " + list);
System.out.println("Largest element in set => " + Collections.max(set));

System.out.println("Smallest element in queue => " + Collections.min(queue));

Collections.reverse(list);
System.out.println("List in reverse order => " + list);

Collections.shuffle(list);
System.out.println("List after shuffle => " + list);
    }
}

```

Arrays

Overview:

- The java.util.Arrays class is a part of the Java Collection Framework.
- This class provides static methods to dynamically manipulate arrays.
- The methods include various operations such as sorting, searching, copying, filling, etc.

Advantages

- **Efficiency:**
 - The java.util.Arrays class provides efficient, pre-built methods for manipulating arrays, which can save considerable development time.
- **Ease of use:**
 - The methods in this class are static, so they can be called directly on the class without creating an instance.
- **Versatility:**
 - The class provides methods for both single-dimensional and multi-dimensional arrays.
- **Integration:**
 - The class is part of the standard Java library, so it integrates well with other Java classes and interfaces.

Arrays (Cont.)

Important Methods:

<code>sort()</code>	Sorts the specified array into ascending numerical order
<code>binarySearch()</code>	Searches the specified array for the specified object using the binary search algorithm
<code>equals()</code>	Returns true if the two specified arrays are equal to one another
<code>fill()</code>	Assigns the specified data type value to each element of the specified range of the specified array
<code>copyOf()</code>	Copies the specified array, truncating or padding with zeros (if necessary) so the copy has the specified length

Arrays—Illustrations

</>

Sort an array

```
int[] array = {1, 5, 2, 6, 3, 7, 4, 8};  
Arrays.sort(array);
```

Search an array

```
int index = Arrays.binarySearch(array, 5);
```

Check if two arrays are equal

```
int[] array1 = {1, 2, 3, 4, 5};  
int[] array2 = {1, 2, 3, 4, 5};  
boolean equal = Arrays.equals(array1, array2);
```

Fill an array

```
int[] array = new int[5];  
Arrays.fill(array, 10);
```

Copy an array

```
int[] original = {1, 2, 3, 4, 5};  
int[] copy = Arrays.copyOf(original,  
original.length);
```

Knowledge Check 1

Which of these methods of Arrays class is used to sort an array or its subset?

A. `binarysort()`

B. `bubblesort()`

C. `sort()`

D. `insert()`

Knowledge Check 2

Which of these methods can randomize all elements in a list?

A. `rand`

B. `randomize()`

C. `shuffle()`

D. `ambiguous()`

Knowledge Check 3

Which of these options is an incorrect form of using the `max()` method to obtain a maximum element?

A. `max(Collection c)`

B. `max(List c)`

C. `max(Collection c, Comparator comp)`

D. `max(Comparator comp)`

Knowledge Check 4

Which of the following sorting algorithm is of the priority queue sorting type?

A. Bubble Sort

B. Insertion Sort

C. Merge Sort

D. Selection Sort

Knowledge Check 5

The binary search algorithm cannot be applied to ...

A. Sorted Linked List

B. Sorted Binary Trees

C. Sorted Linear Array

D. Pointer Array

Hands-On Activity 1

/>

Activity Details

Problem Statement Write a program to sort `ArrayList` in descending order?

Sample Input N/A

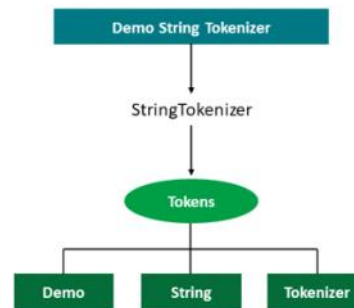
String Tokenizer—Overview



- The `StringTokenizer` class in Java is used to break a string into tokens.
- It is a legacy class retained for compatibility reasons and its use is discouraged in new code.
- The recommended alternative is to use the `split()` method of the `String` class or the `java.util.regex` package.
- `StringTokenizer` object internally maintains a current position with the string to be tokenized.
- Some operations that advance the current position past the character are tokenized.
- It is useful for simple cases where you need to break a string into tokens, but for more complex string processing, other methods are recommended.

String Tokenizer

Overview
• To break a string into tokens use the <code>java.util.StringTokenizer</code> • The string to be tokenized internally maintains its current position. • To return a token, take a substring of the string that was used to create the <code>StringTokenizer</code> object.
Constructors
• <code>public StringTokenizer(String str)</code> – <code>str</code> is the string that is to be tokenized. • <code>public StringTokenizer(String str, String delim)</code> – <code>delim</code> is the set of delimiters that are used to tokenize the string. • <code>public StringTokenizer(String str, String delim, and Boolean flag)</code> – If the flag is true, the delimiter characters are considered to be tokens. – If the flag is false, the delimiter serves to separate tokens.



java.time Package—Overview

Overview:

- The `java.time` package is part of the Java SE 8 Date and Time API, which was developed to address the shortcomings of the older `java.util.Date` and `java.util.Calendar` classes.
- The `java.time` package provides comprehensive, immutable, and thread-safe classes for dates, times, instants, and durations.

Advantages
• Immutability: – The core classes in the <code>java.time</code> package are immutable, which makes them safe to use in multithreaded environments. • Thread Safety: – Unlike <code>java.util.Date</code> and <code>java.util.Calendar</code>, the classes in the <code>java.time</code> package are inherently thread-safe due to their immutability. • Ease of Use: – The <code>java.time</code> package provides clear, straightforward methods for common tasks related to date and time. • Versatility: – It provides classes for different use cases - local dates and times (used when timezone is not important), zoned dates and times (used when timezone is important), periods, durations, and more.

java.time Package—Overview (Cont.)

Important Classes and Methods
• LocalDate: – Represents a date without a time or time-zone. Important methods include <code>now()</code>, <code>plusDays()</code>, <code>minusDays()</code>, etc. • LocalTime: – Represents a time without a date or time-zone. Important methods include <code>now()</code>, <code>plusHours()</code>, <code>minusHours()</code>, etc. • LocalDateTime: – Represents a date and time without a time-zone. Important methods include <code>now()</code>, <code>plusDays()</code>, <code>minusDays()</code>, etc. • ZonedDateTime: – Represents a date and time with a time-zone. Important methods include <code>now()</code>, <code>plusDays()</code>, <code>minusDays()</code>, etc. • Period: – Represents a quantity of time in terms of years, months, and days • Duration: – Represents a quantity of time in terms of seconds and nanoseconds

LocalDate, LocalTime, LocalDateTime

- 'LocalDate', 'LocalTime', and 'LocalDateTime' are classes provided by the Java 8 Date and Time API ('java.time' package) for representing date, time, and combined date-time values respectively.
- They are part of the modern date and time API introduced in Java 8 to address the limitations of the old 'java.util.Date' and 'java.util.Calendar' classes.

LocalDate

- It represents a date without a time zone, typically in the format 'yyyy-MM-dd'.
- It stores the year, month, and day-of-month.

```
LocalDate date = LocalDate.now(); // Current date
LocalDate specificDate = LocalDate.of(2024, 5, 7); // 7th May 2024
```

LocalTime

- It represents a time without a time zone, typically in the format 'HH:mm:ss.SSS'.
- It stores the hour, minute, second, and fraction of second.

```
LocalTime time = LocalTime.now(); // Current time
LocalTime specificTime = LocalTime.of(15, 30, 0); // 3:30 PM
```

LocalDate, LocalTime, LocalDateTime (Cont.)

LocalDateTime

- It represents a date-time without a time zone, typically in the format 'yyyy-MM-ddTHH:mm:ss.SSS'.
- It combines date and time.

```
LocalDateTime dateTime = LocalDateTime.now(); // Current date and time
LocalDateTime specificDateTime = LocalDateTime.of(2024, 5, 7, 15, 30, 0); // 7th May 2024, 3:30 PM
```

DateTimeFormatter

- 'DateTimeFormatter' is a class used for formatting and parsing date-time objects in Java.
- It allows you to define custom patterns for formatting dates and times.

```
LocalDateTime dateTime = LocalDateTime.now();
DateTimeFormatter formatter = DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");

String formattedDateTime = dateTime.format(formatter);
System.out.println("Formatted DateTime: " + formattedDateTime); // Output: Formatted DateTime: 2024-05-07 15:30:00

String dateStr = "2024-05-07 15:30:00";
LocalDateTime parsedDateTime = LocalDateTime.parse(dateStr, formatter);
System.out.println("Parsed DateTime: " + parsedDateTime); // Output: Parsed DateTime: 2024-05-07T15:30
```

Knowledge Check 6

Which of these options contain date information?

A. java.sql.Timestamp

B. java.sql.Time

C. java.io.Time

D. java.io.Timestamp

Summary

The key learning points of the module:

- Collection algorithms in Java are defined in a class called `Collections`, which are defined in the `java.util` package. These algorithms are efficient and make coding simple.
- Java Util packages provide multiple utility classes and APIs including `StringTokenizer`, `Date`, and `Time` under the `java.util` package.
- `LocalDate`, `LocalTime`, and `LocalDateTime` are Java 8 classes for handling dates, times, and combined date-time values without time zones. They provide more flexibility and precision than the older `java.util.Date` and `java.util.Calendar` classes. `DateTimeFormatter` is used to format and parse date-time objects according to custom patterns. Together, these classes offer robust date and time functionality in Java programming.

Java exception handling

Thursday, August 1, 2024 10:01 PM

Topic	Descriptions
Exception Handling	Overview of Exception Handling, Keywords, Exception Types, Exception Hierarchy, Exception Classification, Exception versus Error, throw vs throws, Catch Block Scenarios, Exception Object and methods illustrations, Exception Stack Trace, Understanding and Importance of Stack Trace, Custom Exception

Learning Objectives

By the end of this module, you will be able to:

- Explain Exception Handling in Java programming language
- State the exception handling keywords
- Describe and Contrast types of exceptions
- Illustrate the process of throwing and handling exceptions in Java programs
- Compare the use of throw and throws keywords in Java programs
- Explain annotations and types of annotations

Exception Handling Overview

1. An exception indicates a problem that occurred in the program execution flow. Exceptions may arise due to an invalid input, inconsistent data or database, or file errors.
2. An exception object is created when an error occurs while executing a statement. Then the normal flow of the program halts, and Java Runtime Environment (JRE) tries to find someone that can handle the raised exception.
3. When the exception occurs in a method, the process of creating the exception object and handing it over to runtime environment is called "throwing the exception."
4. Exception handler is the block of code that can process the exception object and is said to be "catching the exception."
5. If there is no appropriate exception handler found, then the program terminates printing information about the exception. The core advantage of exception handling is maintaining the application's normal flow.

Note that Java Exception Handling is a framework used to handle only runtime errors (exceptions), such as `ClassNotFoundException`, `IOException`, `NullPointerException` and `SQLException`.

try

- Where we should place the exception code a specified block is used
- Must be followed by either `catch` or `finally`
- Means that we cannot use the block alone

catch

- To handle the exceptions, it is placed at the end of the try block.
- Allows multiple catch blocks with a try, and a try-catch block can also be nested
- Requires a parameter that should be of type exception

finally

- Can only be used with the try-catch block and is optional
- Gets executed always, whether the exception had occurred or not
- Can be used to close the open sources

throw

- Is used to throw exceptions to the runtime to handle it (for example, in a user authentication program, we should throw exception to client if the password is null)

throws

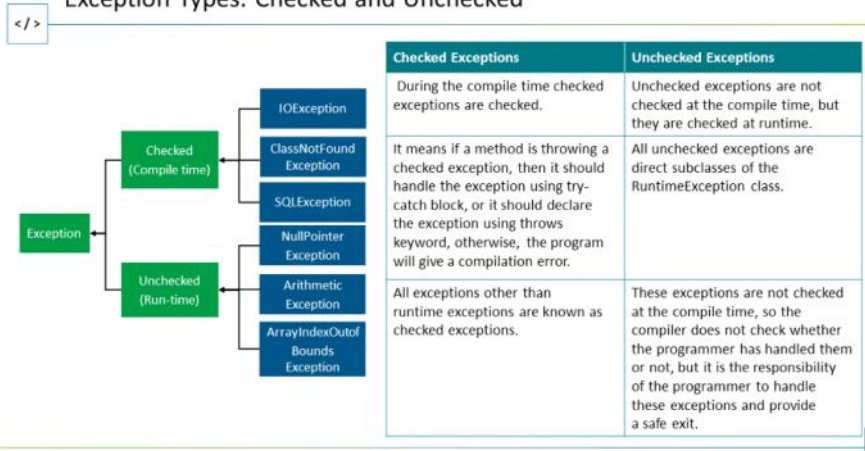
- Is used to declare exceptions but doesn't throw an exception
- Specifies that there may occur an exception in the method
- Is always used with method of signature

Exception :

checked-`IOException`,`ClassNotFoundException`,`SQLException`

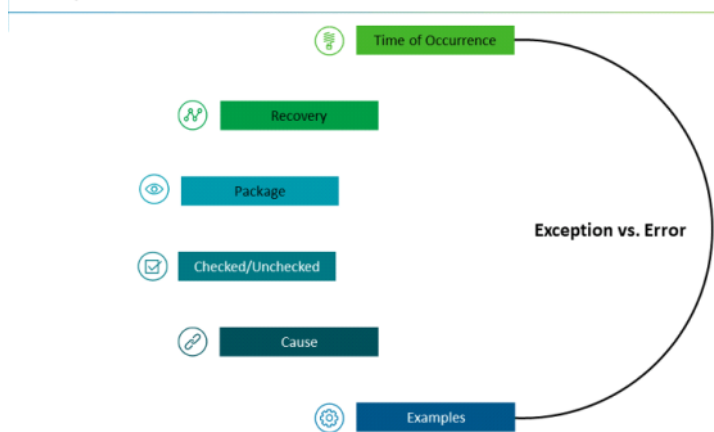
unchecked - `NullPointerException`, `ArithmeticException` , `ArrayIndexOutOfBoundsException`

Exception Types: Checked and Unchecked



Exception vs error

Exception vs. Error



Time of occurrence : Exception occurs at compile time or runtime whereas errors only during runtime

Recovery : try-catch block allows u to recover from exception, whereas programs are irrecoverable from errors once they occur.

Package : errors are defined in java.lang.Error and exceptions are defined in java.lang.Exception

Checked or unchecked : errors -unchecked type, exceptions - both

Cause : application itself mainly causes exceptions , whereas errors caused by the environment in which the program is running

Examples : checked exceptions : io,sql

Unchecked : null,classnotfound

Errors : java.lang.OutOfMemoryError, java.lang.StackOverflowError.

Throwing Exceptions

- The throw and throws keywords are for exception handling. The throws keyword is used in the signature of the method, but the throw keyword throws the exception explicitly from a block of code or method.
- The method that may throw a checked exception must declare it in the signature itself using the **throws** keyword.
- The unchecked exceptions need not be declared upfront.
- Exceptions are thrown using the **throw** keyword.
- A method may throw multiple exceptions.
- Invoker of the method must handle all the checked exceptions.

The unchecked exceptions need not be declared upfront. A method may throw multiple exceptions , invoker of the method must handle all the checked exceptions

Throwing Exceptions illustration :

```
// Java program to illustrate throws
class test
{
    public static void main(String[] args)throws InterruptedException
    {
        Thread.sleep(10000);

        System.out.println("Method Interrupted");
    }
}
```

Difference Between Throw and Throws

Throw	Throws
• A single exception explicitly thrown from any constructor or method uses a throw keyword • throw is followed by an instance of an exception class • The method and constructor use throw within • We can throw only single exceptions using the throw • Propagated to the caller method is a checked exception • Without using break keyword a throw keyword can break a loop or switch statement.	• Denoting which exception can possibly be thrown by this method, a throws keyword is used in the method and constructor declaration • throws is followed by the exception class name • throws is used with the constructor and method signature • You can state multiple exceptions using throws one of which may or may not throw by a method. • Unchecked exceptions are not propagated and thus may not require explicit exception handling • We can not use break keyword in throws

Handling Exceptions—Illustration

- Invoker (Calling method):
 - Must handle the checked exceptions if any. Otherwise, it results in a compilation error.
 - May or may not handle the unchecked exceptions
- The try-catch keywords and their respective blocks are used for handling exceptions.
- The printStackTrace() method on exceptions is useful for debugging.
- Any common code that needs to be executed with or without exceptions can be placed in the finally block.
- In all cases the finally block is guaranteed to be executed.

```
String name = null;
long phoneNumber = 0L;

DirectoryService service = new DirectoryService();
try {
    service.addEmployee(name, phoneNumber);
} catch (InvalidEmployeeException e) {
    e.printStackTrace();
} finally {
    System.out.println("Thankyou for using the service");
}
```

PRINTSTACKTRACE() method is used for debugging Exception Object

- An Exception Object in Java is an instance of a class that extends the Throwable class.
- It represents problems that arise during the execution of the program.
- When such an issue occurs, an object representing that problem is created and thrown in the method that caused the error.
- That method may choose to handle the exception itself or pass it on.
- Either way, at some point, the exception is caught and processed.
- **Exception Methods in Java**
 - Java provides several methods to handle exceptions. Below are a few commonly used ones:

public String getMessage()	This method returns the detail message string of the throwable.
public Throwable getCause()	This method returns the cause of the exception as represented by a Throwable object.
public String toString()	This method returns a short description of the throwable.
public void printStackTrace()	This method prints the throwable and its backtrace to the standard error stream.

Void printStackTrace() , String toString(), Throwable getCause() , String getMessage() , getMessage()

getMessage():
This method returns the detail message string of the Throwable.

```
try {
    int divideByZero = 5 / 0;
} catch (ArithmeticException e) {
    System.out.println("Caught an exception: " + e.getMessage());
}
```

In this example, the getMessage() method is called on the exception object 'e', which returns the detail message string of the ArithmeticException.

getCause()

getCause():
This method returns the cause of the exception, or null if the cause is nonexistent or unknown.

```
try {
    Throwable cause = new ArithmeticException("This is the cause.");
    throw new Exception("This is the Exception", cause);
} catch (Exception e) {
    System.out.println("Caught an exception: " + e.getMessage());
    System.out.println("Cause of the exception: " + e.getCause().getMessage());
}
```

- In this example, an ArithmeticException object is created and used as the cause for a new Exception object.
- The getCause() method is then used to retrieve the original cause of the Exception.

toString()

toString():

This method returns a short description of the Throwable.

```
try {
    int divideByZero = 5 / 0;
} catch (ArithmeticException e) {
    System.out.println(e.toString());
}
```

In this example, the toString() method is called on the exception object 'e', which returns a short description of the ArithmeticException.

printStackTrace()

printStackTrace():

This method prints the Throwable and its backtrace to the standard error stream.

```
try {
    int divideByZero = 5 / 0;
} catch (ArithmeticException e) {
    e.printStackTrace();
}
```

In this example, the printStackTrace() method is called on the exception object 'e', which prints the Throwable and its backtrace to the standard error stream.

Exception Stack Trace

- An Exception Stack Trace is a report that provides detailed information about the sequence of method calls that the application was in the middle of when an Exception was thrown.
- The Anatomy of a Stack Trace:
 - Stack traces usually start with the method where the exception was originally thrown and trace their way up through the method call hierarchy to the first method call.
 - Here's a simple breakdown of a typical stack trace:

Exception Type	• This is the type of exception that was thrown. • It's usually the first line of the stack trace.
Exception Message	• This is a message describing the exception. • It's usually on the same line as the exception type.
Stack Trace Elements	• These are the method calls that were in progress when the exception was thrown. • They are listed in order of most recent to least recent. • Each element includes the fully qualified name of the class, the method name, and the line number where the exception occurred.

Exception Stack Trace (Cont.)

- For example, consider the following code:

```
public class Main {
    public static void main(String[] args)
    {
        method1();
    }

    static void method1() {
        method2();
    }

    static void method2() {
        throw new NullPointerException
            ("This is a NullPointerException.");
    }
}
```

- Running this code would produce a stack trace like this:

```
Exception in thread "main"
java.lang.NullPointerException: This is a
NullPointerException.
    at Main.method2(Main.java:12)
    at Main.method1(Main.java:8)
    at Main.main(Main.java:4)
```

Understanding a Stack Trace

Steps to Understand a Stack Trace

1. Identify the type of Exception from the first line of the stack trace. In our example, it's a NullPointerException.
2. Read the Exception message which follows the Exception type. In our example, the message is "This is a NullPointerException."
3. Look at the stack trace elements that follow. The first element usually points to the exact line of code that threw the Exception. Each subsequent line represents the previous method call that led up to the exception.

Importance of Stack Trace

- A stack trace is crucial for debugging because it helps you trace the path that your code took leading up to the exception.
- By understanding the sequence of method calls and the line numbers involved, you can pinpoint where and why the error occurred.
- This information can help you identify and fix the issue more efficiently.

Catch Block—Scenarios

• Single try block with a single catch block:

- This is the simplest scenario where there is one block of code that might generate an exception (the try block) and one block of code to handle any exceptions that occur (the catch block).

```
try {
    int divideByZero = 5 / 0;
} catch (ArithmeticException e) {
    System.out.println("ArithmeticException => " + e.getMessage());
}
```

• Single try block with multiple catch blocks:

- In this scenario, we have a single try block that can generate multiple types of exceptions.
- Each type of exception has its own catch block.

```
try {
    String str = null;
    System.out.println(str.length());
    int divideByZero = 5 / 0;
} catch (ArithmeticException e) {
    System.out.println("ArithmeticException => " + e.getMessage());
} catch (NullPointerException e) {
    System.out.println("NullPointerException => " + e.getMessage());
}
```

Catch Block—Scenarios (Cont.)

• Nested try-catch blocks:

- Here, we have a try-catch block inside another try-catch block. Each try block can have its own catch blocks.

```
try {
    try {
        int divideByZero = 5 / 0;
    } catch (ArithmeticException e) {
        System.out.println("Inner ArithmeticException => " + e.getMessage());
    }
    String str = null;
    System.out.println(str.length());
} catch (NullPointerException e) {
    System.out.println("Outer NullPointerException => " + e.getMessage());
}
```

• Try-catch block with a finally block:

- In this scenario, there is a finally block that executes regardless of whether an exception occurs or not.

```
try {
    int divideByZero = 5 / 0;
} catch (ArithmeticException e) {
    System.out.println("ArithmeticException => " + e.getMessage());
} finally {
    System.out.println("This is the final block.");
}
```

Catch Block—Scenarios (Cont.)

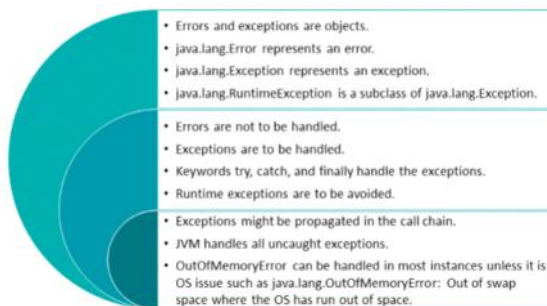
• Catching multiple exceptions in a single catch block:

- This is a feature available from Java 7 and later.
- It allows us to catch multiple exceptions in a single catch block.

```
try {
    String str = null;
    System.out.println(str.length());
    int divideByZero = 5 / 0;
} catch (ArithmeticException | NullPointerException e) {
    System.out.println("Exception => " + e.getMessage());
}
```

JVM HANDLES ALL THE UNCAUGHT EXCEPTIONS

Important Points to Remember!



Custom Exceptions

- Custom exceptions, also known as user-defined exceptions, are created by users to meet specific needs.
- It helps us to create more specific exception types that can add more meaning and handle unique scenarios.

Advantages

- **Improve readability:**
 - By creating exceptions specific to your application, you can make your code easier to understand.
- **Increase maintainability:**
 - Custom exceptions can encapsulate complex error handling logic, making your code easier to maintain.

Steps to Create a Custom Exception

- Create a new class whose name should end with "Exception" to make it clear that it's an exception.
- This new class should be a subclass of one of the pre-existing exception classes, like Exception or RuntimeException.
- Add any necessary constructors to the exception class.

Custom Exception Example

```
// Step 1: Define your custom exception
class CustomException extends Exception {
    public CustomException(String message) {
        super(message);
    }
}

// Step 2: Use your custom exception
public class Main {
    public static void main(String[] args) {
        try {
            checkAge(15);
        } catch (CustomException e) {
            System.out.println(e.getMessage());
        }
    }

    static void checkAge(int age) throws CustomException {
        if (age < 18) {
            throw new CustomException("Access denied - You must have at least 18 years.");
        } else {
            System.out.println("Access granted - You are old enough!");
        }
    }
}
```

Throw keyword is used to monitor exceptions

Throw is used to manually throw an exception

Catch keyword is used by calling function to handle exceptions thrown by the called function

Exception class is related to all exceptions that can be caught by using catch

Java runtime system handles the exception when no catch is used

Which operator is used to generate an instance of an exception that can be thrown by using throw? - new

Which of these clauses will be executed even if no exceptions are found? - Finally

Throwable is a superclass of the exception class

Activity Details

Problem Statement	Develop Java programs to handle most frequently encountered Checked and Unchecked exception scenarios, as per business use cases.
Sample Input	N/A

Summary

The key learning points of the module:

- Exception handling in Java allows developers to handle and recover from exceptional situations that can occur during program execution. It provides a mechanism to catch and handle errors, exceptions, and unexpected conditions, ensuring the stability and robustness of Java applications.
- Exceptions are objects that represent exceptional conditions or errors that can occur during program execution, divided into two categories, checked exceptions and unchecked exceptions.
- Checked Exceptions are exceptions that the compiler requires the programmer to handle explicitly using try-catch blocks or declare them in the method signature using the throws keyword. Examples include IOException, SQLException, etc.
- Unchecked Exceptions, also known as runtime exceptions, do not require explicit handling or declaration. They are typically caused by programming errors, such as null pointer access, array index out of bounds, etc. Examples include NullPointerException, ArrayIndexOutOfBoundsException, etc.
- The try-catch block is used to catch and handle exceptions. The code that might throw an exception is placed within the try block, and if an exception occurs, it is caught by an appropriate catch block. Multiple catch blocks can be used to handle different types of exceptions.
- The finally block is optional and follows the try-catch block. It is used to specify code that should be executed regardless of whether an exception occurs or not and is commonly used to release resources or perform cleanup tasks.

Summary (Cont.)

Here are the key learning points of the module (cont.):

- **Throwing Exceptions:** The `throw` keyword is used to explicitly throw an exception from within a method. It is used to indicate that an exceptional condition has occurred and needs to be handled by the caller or a higher-level method.
- **Exception Hierarchy:** Java has a hierarchy of exception classes, with the root class being `Throwable`. `Throwable` has two main subclasses: `Error` (indicates serious system errors) and `Exception` (represents exceptional conditions in program execution). The `Exception` class further subclasses into checked exceptions and unchecked exceptions.
- **Custom Exceptions:** Developers can create their own custom exception classes by extending the `Exception` class or one of its subclasses. Custom exceptions allow for more specific exception handling and provide meaningful error messages.
- **Exception Propagation:** When an exception is thrown in a method, it can be caught and handled within that method or propagated to the calling method. If an exception is not handled, it continues to propagate up the call stack until it is caught, or the program terminates.
- **Exception Handling Best Practices:** Proper exception handling practices include providing meaningful error messages, handling exceptions at the appropriate level, avoiding overly broad catch blocks, and releasing resources in the `finally` block.
- **Java's exception handling mechanism** allows developers to write code that gracefully handles exceptional situations, improves the reliability of applications, and provides better error reporting and recovery mechanisms. By understanding and effectively using exception handling, developers can create more robust and fault-tolerant Java programs.

JSTL TAGS AND EXPRESSION LANGUAGE(EL)

Wednesday, July 24, 2024 9:03 AM

To print something on the server we can use EL

JSTL CORE TAGS

To use jstl u need to use a jstl1.2 or any latest version jar file, but I don't know whether it's the case with maven

It should be inside webapp

If u keep session true, it will take a new session if there is no previous session then the username cannot be retrieved that's why we keep session false;

Generally when u forward a req dispatcher from doPost, it will go to doPost in the next servlet

When u do a sendRedirect it will go to the doGet

FORMAT OF THE EXPENSE PROJECT

MVC : MODEL VIEW CONTROLLER PATTERN

Model

User - Javabean

Expense - username - Javabean

UserController - validateuser / addUser

LogoutController - logout

ExpenseController - addexpense/getAllExpense

IUserService

validateUser(); - login

registerUser(); - register

IExpenseService

addExpense();

getAllExpenseByUserName();

IUserDAO

IExpenseDAO

HOW TO WORK WITH DATE IN JAVA

ExpenseBean -- private Date expenseDate

2022-04-24 -- String Date

String should be converted to date

Import DateFormat, Simple
DateFormat

import DateFormat, SimpleDateFormat, java.util.Date

```
Dateformat df=new SimpleDateFormat("yyyy-MM-ddHH:mm:ss");  
Date df=df.parse(request.getParameter("date"))  
setExpenseDate(df);
```

convert util.date to java.sql.Date

```
psmt.setDate(1,new java.sql.Date(bean.getExpenseDate().getTime()));
```

```
<%@page language="java" contentType="text/html; charset=UTF-8"  
pageEncoding="UTF-8"%>
```

```
<%@taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core"%>  
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<meta charset="UTF-8">
```

```
<title>Expense page </title>
```

```
</head>
```

```
<body>
```

```
<table>
```

```
<tr>
```

```
<th>Username</th>
```

```
<th>Expense Date</th>
```

```
<th>Expense Type</th>
```

```
<th>Expense Description</th>
```

```
<th>Expense Amount </th>
```

```
</tr>
```

```
<tr>
```

```
<c:forEach items="${ListOfExpenses}" var="s">
```

```
<td><%s.getu %>Hemanth</td>
```

```
<td>24-07-2024</td>
```

```
<td>Travel</td>
```

```
<td>Bus Fare</td>
```

```
<td>1500</td>
```

```
<c:forEach>
```

```
</table>
```

```
</body>
```

```
</html>
```

CODING FROM SCRATCH

Thursday, July 25, 2024 7:04 PM

IDEA : BATTER/BIKE/CAR

CAR TYPES -SUV,SEDAN,HACHBACK

Servlet

Thursday, August 1, 2024 8:23 PM

Servlet isn't a server

Server receives the http request, creates an empty response object, it forwards it to the servlet and the servlet gives the relevant response object from the database and return it back to the web container, the web container sends the response back to the browser

building a response, we use `Writer` which sends the contents out of the browser via a response body, we're going to append a content to the response body and the content we append is a parameter that comes with the request, so that parameter name will be reflected back to the browser.

```
response.getWriter().append(request.getParameter("name"));
```

From <<https://www.linkedin.com/learning/java-ee-8-essential-training/building-dynamic-responses?autoSkip=true&resume=false&u=44872036>>

JAVA 8 FEATURES

Saturday, August 3, 2024 8:06 PM

Java 8 Features

Functional Interfaces, Lambda Expressions, Stream API, Method References, Default Methods, and the `forEach()` Method

Learning Objectives

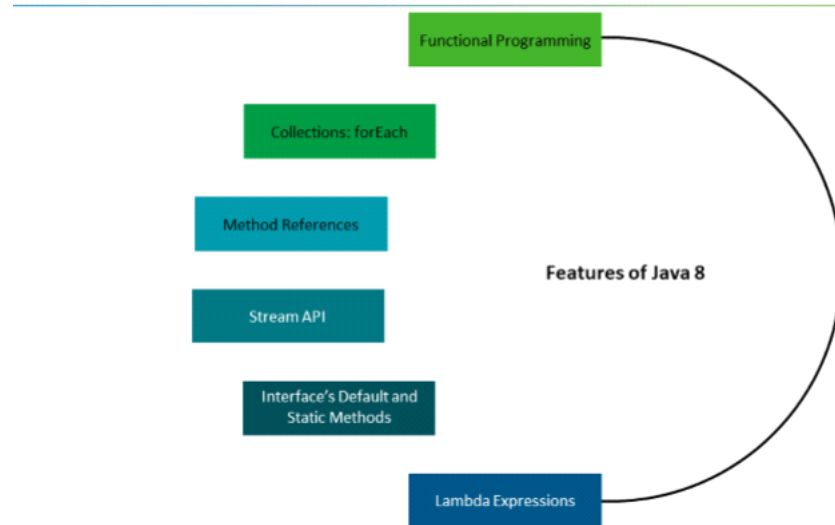
By the end of this session, you will be able to:

- Discuss and Explain the key features in Java 8
- Experiment with the real-time implementation of Java 8 features in programs/applications
- Demonstrate the use of lambda function for implementing functional interfaces in Java programs

About Java 8

- Java 8 is a significant update to the programming language and includes a major upgrade to functional programming called the lambda expressions.
- Java 8 also got a new and improved date/time API, an enhanced JavaScript engine, and a new streaming API.

Features of Java 8



Java 8—Functional Programming

>

Object-Oriented Programming

A programming standard based on the concept of objects that combines data (fields or attributes) and code (procedures or methods).

In OOP, we follow this procedure:

1. Define classes
2. Create instances of classes
3. Invoke methods on the instances

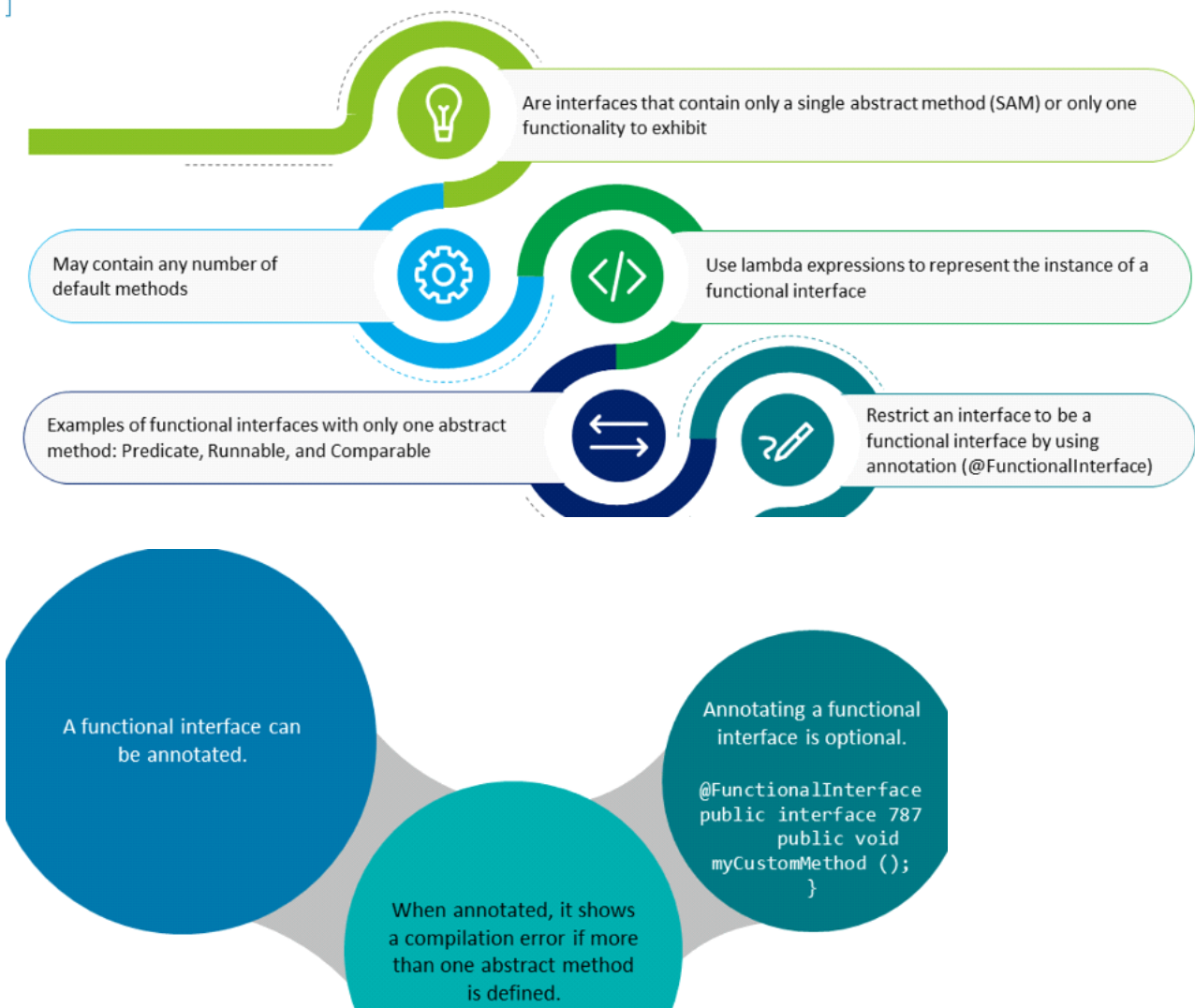
Functional Programming

A style of building the structure and elements of programs that treats computation as the evaluation of mathematical functions and avoids changing-state and mutable data.

In FP:

- There is a complete separation of data and behavior (functions).
- The data (objects) are immutable.
- The shared state is avoided so that there is no inheritance.

Functional Interfaces—Overview



Lambda Expression

Overview

A lambda expression is a block of code that represents an anonymous function with a list of formal parameters and a body.

Lambda Expression

Overview

- Is an unnamed block of code (or an unnamed function) with a list of formal parameters and a body
- Is an anonymous function, which means it is a function with no name and without being bound to an identifier
- Represents an instance of an anonymous class implementing a functional interface with the unnamed block representing the implementation of a single abstract method of that functional interface
- Is often called “lambda”

Basic Syntax:

```
(parameters) -> expression (Example: (x, y) -> x + y)
Or, (parameters) -> { statements; }
Or, (Type1 parameter1, Type2 parameter2) -> { statements; }
Or, () -> expression
```

Rules for Writing Lambda Expressions

- The number of parameters in a lambda expression can be zero, one, or more.
- Parameter types can be explicitly declared or inferred from the context.
- Multiple parameters are presented within mandatory parentheses and separated by commas.
- Empty parentheses are used to indicate empty set of parameters.
- If there is only single parameter, and its type is inferred, it is not mandatory to use parentheses. (Example: a -> return a*a)
- The number of statements in the body of a lambda expression can be zero, one, or more.
- If the body of lambda expression has a single statement, then the curly brackets are not mandatory, and the return type of anonymous function is the same as that of body expression. When there are more than one statements in the body, they must be enclosed in curly brackets.

Anonymous Class to Lambda Expression Illustration

Where to Use Lambda?

- Lambda expressions can be used at any place where an instance of any functional interface is required.

Anonymous Class:

```
Runnable r1 = new Runnable() {
    @Override
    public void run() {
        System.out.println("Hello World");
    }
};
```

Equivalent Lambda Expression:

```
Runnable r2 = () -> system.out.println ("HelloWorld");
```

Anonymous Class to Lambda Expression Illustration (Cont.)

Iterator:

```
List<Integer> numberList = Arrays.asList(1,2,3,4,5);
```

Before Java 8:

```
for (Iterator ite = numberList.iterator(); ite.hasNext();) {  
    Integer int = (Integer) ite.next();  
    System.out.println(int);  
}
```

In Java 8:

```
numberList.forEach((integer) -> System.out.println(int));
```

Use of Lambda Expressions

Why Use Lambda?

- Since lambda represents an instance of an anonymous class, it can be used to replace the usage of all functional interfaces in libraries.
- It is implemented via a new JVM instruction “invokedynamic.”
- Using lambda gives performance improvement over using the traditional anonymous inner class, which uses the “invokevirtual” instruction.
- Many of the standard JDK classes are refactored to use lambda.
- The refactoring to use lambda is one of the major contributing factors in the improved performance of JDK 8.

Lambda Expressions—Examples

/>

```
package com.example.java8;  
  
import java.util.ArrayList;  
import java.util.List;  
  
public class LambdaExample {  
  
    public static void main(String args[]) {  
        example1();  
        example2();  
        example3();  
    }  
  
    //Example 1: Java Lambda Expression with no parameter  
  
    static void example1(){  
        MyFunctionalInterface msg = () -> {return  
        "Hello";};  
        System.out.println(msg.sayHello());  
    }  
}
```

```
//Example 2: lambda expression with single parameter  
  
static void example2(){  
    MyFunctionalInterface2 f = (num) -> num+5;  
    System.out.println(f.incrementByFive(22));  
}  
  
//Example 3: Iterating collections using foreach loop  
  
static void example3(){  
    List<String> list=new ArrayList<String>();  
    list.add("abc");  
    list.add("def");  
    list.add("ghi");  
    list.add("jkl");  
    list.add("mno");  
    list.forEach((names)->  
        System.out.println(names));  
}
```

In-built Functional Interfaces and Implementations

/>

Functional Interface	Overview	SAM (Single Abstract Method)	Example
Consumer<T>	Represents an operation that accepts a single input argument and returns no result	void accept(T t);	<pre>Consumer<String> print = x -> System.out.println(x); print.accept("Hello World!");</pre>
Supplier<T>	Represents a supplier of results, taking no input	T get();	<pre>Supplier<String> helloWorldSupplier = () -> "Hello World"; System.out.println(helloWorldSupplier.get());</pre>
Predicate<T>	Represents a predicate (boolean-valued function) of one argument	boolean test(T t);	<pre>Predicate<String> lengthCheck = x -> x.length() > 5; System.out.println(lengthCheck.test("Hello World!"));</pre>
Function<T,R>	Represents a function that accepts one argument and produces a result	R apply(T t);	<pre>Function<String, Integer> lengthFunc = x -> x.length(); System.out.println(lengthFunc.apply("Hello World!"));</pre>

In-built Functional Interfaces and Implementations (Cont.)

Note:

- These FIs are designed to be used with lambda expressions and method references, which are key features of functional programming in Java 8.
- They allow for more streamlined and readable code by reducing the need for boilerplate code.



Hands-On Activity 1

>

Activity Details	
Problem Statement	Write a code to demonstrate Java Lambda expression with No Parameter
Sample Input	N/A

Default and Static Methods of an Interface

When does a class implement an interface that has a default method?

- A class may choose not to override default methods in which case it inherits all default methods of interfaces it extends.
- A class can override the default methods similar to the other methods we override in a subclass.
- An abstract class can redeclare the default method as abstract again which forces subclasses to override it again.

When must a class explicitly resolve conflicts?

- Conflicts can occur:
 - When the same method is defined in an implementing interface and an extending class.
 - When same method is defined in two implementing interfaces and at least one of them is marked as default.
- A class needs to explicitly provide implementation of such methods to resolve conflict.
 - It can refer to any specific interface implementation using `Interface1.super.methodName()`.

Where do we use static methods?

- Static methods can be used to provide the companion utility methods relevant to your interface.
- There is no need to create separate utility classes, which unnecessarily increases the number of classes in libraries.
- Unlike the default methods, static methods do not face conflicts as they cannot be overridden.

Detailed explanation :

Default and Static Methods in Interfaces—Implementation

Although Java classes can implement multiple interfaces, they can extend only one superclass.

```
public interface NewInterface {
    void foo();
    // abstract public (instance) (used pre-JDK 8)
    // Default methods are marked by keyword "default"

    default void bar() {
        // public (instance) (used post-JDK 8)
        System.out.println("My Java8 Interface With Default run
        default bar()");
    }
    //default void bar1();
    // error: missing method body, or declare abstract
}

public interface NewInterface2 {
    // Same signature (but different implementation) as the
    // default method
    default void bar() { // public (instance) (post-JDK 8)
        System.out.println("NewInterface2 runs default bar()
        too!");
    }
}
```

```
public class StaticAndDefaultMethodExample implements
NewInterface, NewInterface2 {
    @Override
    public void foo() {
        System.out.println("StaticAndDefaultMethodExample
        runs foo()");
    }
    @Override
    public void bar() {
        System.out.println("StaticAndDefaultMethodExample
        runs overridden bar()");
    }
    public static void main(String[] args) {
        StaticAndDefaultMethodExample c = new
        StaticAndDefaultMethodExample();

        c.foo();
        c.bar();
    }
}
```

Stream API and Pipeline

Stream API	Pipeline
What Is a Stream? - Represents a sequence of objects from a source that support aggregate operations - A new abstract layer (introduced in Java 8) - Processes data in a declarative way similar to SQL statements - Supports aggregate operations (of functional programming) Characteristics of Streams: - Sequence of elements - Source - Aggregate operations - Pipelining - Automatic iterations Limitations of Streams - Once a stream is consumed, it can't be used later. - There are a lot of stream API methods.	What Is a Pipeline? Is a sequence of operations on a collection or array that composes: A source, which is a collection or an array (for example, <code>plst</code>, which is a list of person objects) Stream() to produce a stream, which is a sequence of elements to be carried from the source into the pipeline Some intermediate operations (for example, <code>filter(Predicate)</code>, which creates a new stream consisting of elements that match the predicate) A terminal operation or the reduction operation, such as <code>forEach()</code>, which produces the desired result Generating Streams <code>stream()</code>: Returns a sequential stream by considering the collection as its source <code>parallelStream()</code>: Returns a parallel stream by considering the collection as its source

Stream API and Pipeline (Cont.)

forEach	<pre>Stream<Integer> numbers2 = Stream.of(1,2,3,4,5); numbers2.forEach(i -> System.out.println(i+""));</pre>
Map	<pre>Stream<String> names = Stream.of("abc", "def", "ijk"); System.out.println(names.map(s -> {return s.toUpperCase();}).collect(Collections.toList()));</pre>
Filter	<pre>List<String> strings = Array.asList("abc", "", "def", "", "ijk"); int count = strings.stream().filter(string -> string.isEmpty()).count();</pre>
Limit	<pre>Random = new Random(); random.ints().limit(10).forEach(System.out.println);</pre>
Sorted	<pre>Random = new Random(); random.ints().limit(10).sorted().forEach(System.out.println);</pre>
Match	<pre>Stream<Integer> numbers2 = Stream.of(1,2,3,4,5); System.out.println("Stream contains 4? " +numbers2.anyMatch(i -> i==4));</pre>

Popular Ways To Create Stream

- **From a Collection:**

- Every collection class has a stream() method that returns a sequential stream of elements in the collection.
- Example:

```
List<String> list = Arrays.asList("a", "b", "c");
Stream<String> stream = list.stream();
```

- **From an Array:**

- You can create a stream from an array using the Stream.of() method or Arrays.stream() method.
- Example:

```
String[] array = {"a", "b", "c"};
Stream<String> stream1 = Arrays.stream(array);
Stream<String> stream2 = Stream.of(array);
```

- **Note:**

- The Stream.of method in Java 8 is a static method used to create a new Stream object from an existing array or specific values.
- The elements of the Stream object are the elements of the specified array or the specified values.

forEach Method

forEach to Iterate and Display Each Key and Value Pair of HashMap

```
Private Static Void ExampleCollection() {
    System.out.println("Example Collection -");
    Map<Integer, String> hmap = new HashMap<Integer, String>();
    hmap.put(1, "abc");
    hmap.put(2, "def");
    hmap.put(3, "ijk");
    hmap.put(4, "lmn");
    /* forEach to iterate and display each key and value pair of HashMap. */
    hmap.forEach((key,value)->System.out.println(key+" - "+value));
    /* forEach to iterate a Map and display the value of a particular key */
    hmap.forEach((key,value)->{
        if(key == 4){
            System.out.println("Value associated with key 4 is: "+value);
        }
    });
    /* forEach to iterate a Map and display key associated with particular value */
    hmap.forEach((key,value)->{
        if("Sachin".equals(value)){
            System.out.println("Key associated with Value ijk is: "+key);
        }
    });
}
```

Result

Example Collection -

1 - abc
2 - def
3 - ijk
4 - lmn

Value associated with key 4 is: lmn
Key associated with Value ijk is: 3

Java Method References

A new feature called method reference is offered in Java 8. Method references:

- Are used to refer to the method of functional interface
- Are the compact and easy form of lambda expressions
- Are intended to be used instead of lambda expressions to prevent unnecessary boilerplate
- Require a target type context that consists of a compactible functional interface

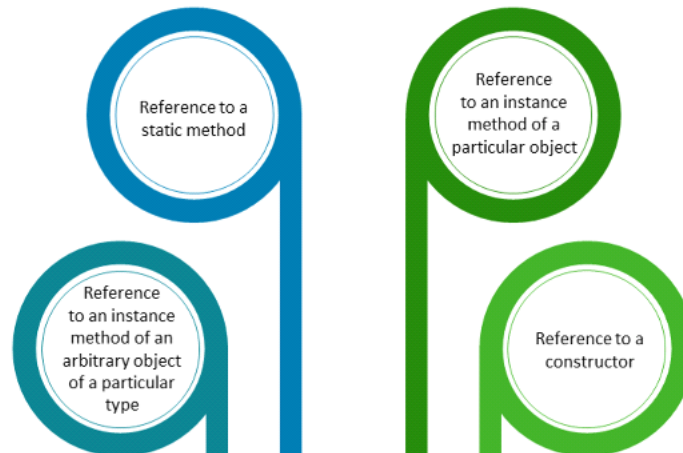
The lambda parameter list and return type must match the signature of the methods.

Method References as Lambda Expressions:

Syntax	Example	As Lambda
ClassName::new	String::new	() -> new String()
Class::staticMethodName	String::valueOf	(s) -> String.valueOf(s)
object::instanceMethodName	x::toString	() -> x.toString()
Class::instanceMethodName	String::compareToIgnoreCase	(s1,s2) -> s1.compareToIgnoreCase(s2)

Java Method References (Contd.)

There are four types of method references.



Java Method References (Contd.)

:/>

Reference to a Static Method

```
interface Sayable{
    void say();
}
public class MethodReference {
    public static void saySomething(){
        System.out.println("This is static method.");
    }
    public static void main(String[] args) {
        // Referring static method
        Sayable sayable = MethodReference::saySomething;
        // Calling interface method
        sayable.say();
    }
}
```

Reference to an Instance Method

```
interface Sayable{
    void say();
}
public class InstanceMethodReference {
    public void saySomething(){
        System.out.println("Non-static method.");
    }
    public static void main(String[] args) {
        InstanceMethodReference methodReference = new InstanceMethodReference();
        Sayable sayable = methodReference::saySomething;
        sayable.say();
        Sayable sayable2 = new InstanceMethodReference();
        sayable2.say();
    }
}
```

Reference to a Constructor

```
interface Messageable{
    Message getMessage(String msg);
}
class Message{
    Message(String msg){
        System.out.print(msg);
    }
}
public class ConstructorReference {
    public static void main(String[] args) {
        Messageable hello = Message::new;
        hello.getMessage("Hello");
    }
}
```

Java Method References (Contd.)

Reference to a Static Method

```
public static void example(){
    BiFunction<Integer, Integer, Integer> product =
    MethodReferenceExample::multiply;
    int pr = product.apply(11, 5);
    System.out.println("Product of given number is:
    "+pr);
}
```

Reference to an Instance Method of a Particular Object

```
public static void example(){
    MethodReferenceExample obj = new
    MethodReferenceExample();
    MyFunctionalInterface3 ref =
    obj::myMethod1;
    ref.display();
}
```

Reference to an Instance Method of an Arbitrary Object of a Particular Type

```
public static void example() {
    String[] stringArray = { "abc", "def", "gji", "jkl",
    "mno", "pqr", "stu"};
    Arrays.sort(stringArray,
    String::compareToIgnoreCase);
    for(String str: stringArray) {
    System.out.println(str); }
}
```

Reference to a Constructor

```
public static void example() {
    MyFunctionalInterface4 ref =
    MethodReferenceExample::new;
    ref.display("Hello World!");
}
```

Using lambda expressions, you can refer to either final variable or effectively final variable (which is assigned only once).

Yes

No

Most of the stream operations return stream itself so that their result can be pipelined.

True

False

Which of the following statements is correct about Java 8 lambda expression?

- A. Using the lambda expression, you can refer to final variable or effectively final variable.
- B. Lambda expressions throw a compilation error, if a variable is assigned a value the second time.
- C. Both of the above
- D. None of the above

Which of the following features is not introduced with Java 8?

- A. Stream API
- B. Serialization
- C. Spliterator
- D. Lambda Expressions

What is the return type of lambda expressions?

- A. String
- B. Object
- C. Void
- D. Function

Which new method is introduced in Java 8 to iterate over a collection?

- A. `for (String i : StringList)`
- B. `foreach (String i : StringList)`
- C. `StringList.forEach()`
- D. `list.for()`

Activity Details

Problem Statement

- a. Create a student class that contains three private variables—Student ID, Name, and Age.
- b. Generate Getter/Setters
- c. Use `Collections.sort()` method on `ArrayList` to sort the list based on age.

Sample Input

N/A

Summary

The key learning points of the module:

- Java 8 is a major update to the programming language and introduces a significant upgrade to functional programming called lambda expressions.
- Java 8 has several features, such as functional programming, collections, method references, default method and static methods, stream API, and lambda expressions.
- Functional Programming (FP) is a programming paradigm, which is a style of building the structure and elements of computer programs, that treats computation as the evaluation of mathematical functions and avoids changing-state and mutable data.
- Lambda expression is an unnamed block of code (or an unnamed function) with a list of formal parameters and a body.
- The Stream API provides a new way to process collections of data in a functional and declarative style. Streams allow for parallel and sequential processing, enabling efficient and concise operations such as filtering, mapping, reducing, and sorting.
- Java provides a new feature called method reference in Java 8, which is used to refer to the method of functional interface.

Summary (Cont.)

Here are the key learning points of the module (cont.):

- Default methods allow interfaces to have concrete method implementations. This feature enables adding new methods to existing interfaces without breaking backward compatibility. Default methods are useful for adding optional functionality to interfaces or providing sensible default implementations. Java 8 allows adding default methods directly in interfaces. This feature enables interfaces to provide method implementations, reducing the need for abstract classes in some scenarios.
- Java 8 brought significant improvements to the language, focusing on functional programming, stream processing, and overall developer productivity. These features have had a profound impact on Java development and continue to be widely used in modern Java applications.

JDBC

Sunday, August 4, 2024 2:23 PM

JDBC : architecture, types of drivers , jdbc classes and interfaces , DB tables , constraints , Implementation of JDBC API, CRUD Operations
Describe the differne tways to runs the queries while interacting with the database
Exmaine and use appropate jdbc classes and intefaces to perform crud operationd o database tables

What, Why and How ?

What : it's a java api to execute and connect the query with the database, enables java application to connect to relationsal databases like mysql,postgresql etc , has a library that incldues APIs for each task that are commonly associated with database usage

Why : odbc was the only one before and it was written in c so to provide a pure java solutions we use jdbc

What are the diff components of the jdbc program :

DriverManager : A class that manages a list of databse drivers

Driver : An interface that handles the communication with the database server

Connection : A class that reprs database connection

Statement : An interface used to execute SQL statements against a relational database

ResultSet : After an SQL query is executed it holds the data retrived from a database

SQLException : A class that handles any errors that occur ina database appplication

Types of jdbc drivers :

Jdbc-odbc bridge jdbc driver

Java + native code jdbc driver

All Java + middleware translation jdbc driver

All java jdbc driver

Types of JDBC Drivers

Type 1: JDBC-ODBC Bridge JDBC Driver	Type 2: Java + Native Code JDBC Driver
- The JDBC-ODBC bridge driver uses the ODBC driver to connect to the database. - The JDBC-ODBC bridge driver converts JDBC method calls into the ODBC function calls. Note: Oracle does not support the JDBC-ODBC bridge from Java 8.	- This type is also known as Native-API driver (or a partially Java driver). - The Native API driver uses the client-side libraries of the database. - The driver converts JDBC method calls into native calls of the database API. - It is not written entirely in Java.
Type 3: All Java + Middleware Translation JDBC Driver	Type 4: All Java JDBC Driver
- This type is also known as the Network Protocol driver (or a fully Java driver). - The Network Protocol driver uses middleware (application server) that converts JDBC calls directly or indirectly into the vendor-specific database protocol. - It is fully written in Java.	- This type is also known as thin driver (or a fully Java driver). - The thin driver converts JDBC calls directly into the vendor-specific database protocol, and that is why it is known as the thin driver. - It is fully written in Java.

FIVE STEPS TO CONNECT TO THE DATABASE

1 Register the driver

forName()

2 Create a connection

getConnection()

3 Create statements

createStatement()

4 Execute Queries

executeQuery()

5 Close the connections

Close()

Five Steps to Connect to a Database

1. Register the driver.	2. Create connections.	3. Create statements.	4. Execute queries.	5. Close connections.
- The forName() method of the class is used to register the driver class. - This method is used to dynamically load the driver class. Syntax <pre>public static void forName(String className) throws ClassNotFoundException</pre> Example <pre>Class.forName("oracle.jdbc.driver.OracleDriver");</pre>	The getConnection() method of the DriverManager class is used to establish a connection with the database. Syntax <pre>public static Connection getConnection(String url, String user, String password) throws SQLException</pre> Example: <pre>Connection con = DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:xe", "system", "password");</pre>	- To create a statement using the connection interface the createStatement() method is used. - To execute queries with the database the object of the statement is responsible. Syntax <pre>public Statement createStatement() throws SQLException</pre> Example <pre>Statement stmt = con.createStatement();</pre>	- To execute queries to the database the executeQuery() method of the statement interface is used. - To get all the records of a table this method is used to return the object of ResultSet. Syntax <pre>public ResultSet executeQuery(String sql) throws SQLException</pre> Example: <pre>ResultSet rs = stmt.executeQuery("select * from emp"); while(rs.next()){ System.out.println(rs.getInt(1) + " " + rs.getString(2)); }</pre>	- ResultSet and object statement will be closed automatically by closing the connection. - To close the connection the close() method of the connection interface is used. Syntax <pre>public void close() throws SQLException</pre> Example <pre>con.close();</pre>

Connection

/>

- A Connection is the session between the Java application and database.
- The Connection interface is a factory of Statement, PreparedStatement, and Database Metadata.
- The Connection interface provide many methods for transaction management, including commit() and rollback().
- You need to open a connection to the database before you can read or write data from and to a database via JDBC.

Opening the JDBC Connection

```
• With only URL:
Connection connection =
DriverManager.getConnection(url);

• With URL and credentials:
Connection connection =
DriverManager.getConnection(url, user, password);

• With URL and properties:
Connection connection =
DriverManager.getConnection(url, properties);
```

Closing the JDBC Connection

```
• Command
connection.close();

• Via try-with-resources
try(Connection connection
=DriverManager.getConnection(url,
user, password)) {
//use the JDBC Connection inhere
}
```

Statement

>

- To execute the SQL commands the Statement interface contains the essential functions.
- They also define methods that help bridge data type differences between Java and SQL data types used in a database.
- Creating a Statement object:
Statement stmt = con.createStatement();
- The three ways to execute SQL instructions are:

executeQuery()	Is used for the select instructions
executeUpdate()	Is used for updating the data or the database structure
execute()	Can be used for both cases above when the result is unknown

Example for execute():

```
String tableSql = "CREATE TABLE IF NOT EXISTS
employees"
+ "(emp_id int PRIMARY KEY AUTO_INCREMENT,
name varchar(30),"
+ "position varchar(30), salary double)";
stmt.execute(tableSql);
```

Example for executeQuery():

```
String selectSql = "SELECT * FROM employees";

ResultSet resultSet =
stmt.executeQuery(selectSql);
```

Example for executeUpdate():

```
String insertSql = "INSERT INTO employees(name,
position, salary)" + " VALUES('abc', 'developer',
2000)";
stmt.executeUpdate(insertSql);
```

PreparedStatement

>

- Precompiled SQL sequences are contained in PreparedStatement objects
- One or more parameters can be denoted by a question mark.
- The PreparedStatement interface is a sub-interface of Statement.

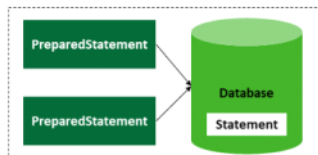
Steps to Create a PreparedStatement

1. Create a PreparedStatement which updates records based on the given parameters to the employees' table.

```
String updatePositionSql = "UPDATE employees SET
position=? WHERE emp_id=?";
PreparedStatement pstmt =
con.prepareStatement(updatePositionSql);
```
2. Add parameters to the PreparedStatement.

```
pstmt.setString(1, "Developer");
stmt.setInt(2, 123);
```
3. Execute the PreparedStatement.

```
int rowsAffected = pstmt.executeUpdate();
```



CallableStatement

</>

- The CallableStatement interface allows calling stored procedures and functions.
- Some databases with heavy operations may benefit performance-wise from being executed inside the same memory space as the database server as a stored procedure.
- By using stored procedures and functions, we can have business logic on the database.

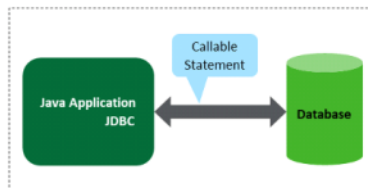
Steps to Create a CallableStatement

1. Create a CallableStatement.

```
String preparedSql = "{call
InsertEmployee(?, ?, ?)}";
CallableStatement cstmt =
con.prepareCall(preparedSql);
```
2. Set up the parameter values.

```
cstmt.setString(2, "abc");
cstmt.setString(3, "tester");
cstmt.setDouble(4, 2222)
```
3. Execute the CallableStatement.

```
cstmt.execute();
```



Statement vs PreparedStatement vs CallableStatement

Statement	PreparedStatement	CallableStatement
- It is used to execute normal SQL queries. - When a particular SQL query is to be executed only once, this interface is preferred. - You cannot pass the parameters to the SQL query using this interface. - DDL statements, such as CREATE, ALTER, DROP mainly use this interface. - This interface performance is very low.	- To execute dynamic or parameterized SQL queries this interface is used. - When a particular query is to be executed multiple times, this is the preferred interface. - At run time you can pass the parameters to SQL query using this interface. - For any kind of SQL query that is to be executed multiple times this interface is used. - When used for multiple executions of the same query the performance of this interface is better than Statement.	- To call the stored procedures this interface is used. - When executing stored procedures, this is the preferred interface. - You can pass three types of parameters-IN, OUT, and IN OUT using this interface. - It is used to execute functions and stored procedures. - This interface has high performance.

Performance : callable > prepared > statement

ResultSet

- By executing database queries ResultSet generates a table of data.
- To maintain a cursor pointing to a row of a table is the object of ResultSet.
- Initially, the cursor points before the first row.
- By using the stored procedures and functions we can have the business logic on the database.

Name	Age	Gender
abc	27	Male
xyz	21	Male
pqr	31	Female

Generating a ResultSet

```
PreparedStatement pstmt =
dbConnection.prepareStatement("select * from people");
ResultSet rs = pstmt.executeQuery();
while(rs.next()) {
    String name = rs.getString("name");
    Integer age = rs.getInt("age");
    System.out.println("Name:"+name+", Age:"+age);
}
```

Retrieving MetaData from the ResultSet

```
ResultSetMetaData metaData = rs.getMetaData();
Integer columnCount = metaData.getColumnCount();
```

Hands-On Activity 1

Activity Details	
Problem Statement	Write a Java program to perform the following operations using PreparedStatement in JDBC- - Drop a column from a DB table. Please note that the column should be empty. - Add a PK to the existing DB table. Use appropriate DDL commands for the same.
Sample Input	N/A

Types Class

java.sql.Types:

- The JDBC types are the class that defines the constants that are used to identify generic SQL types.
- This class is never instantiated.
- These types have been designed to represent the most-commonly used SQL types.

CHAR, VARCHAR, and LONGVARCHAR

- CHAR represents a small, fixed-length character string.
- VARCHAR represents a small, variable length character string.
- LONGVARCHAR represents a large, variable-length character string.

DATE, TIME, and TIMESTAMP

- DATE represents a date consisting of day, month, and year.
- TIME represents a time consisting of hours, minutes, and seconds.
- TIMESTAMP represents DATE plus TIME plus a nanosecond field.

DECIMAL and NUMERIC

- They both represent fixed-precision decimal values.
- These SQL types take scale and precision parameters.
- The recommended Java mapping for the NUMERIC and DECIMAL types is java.math.BigDecimal.

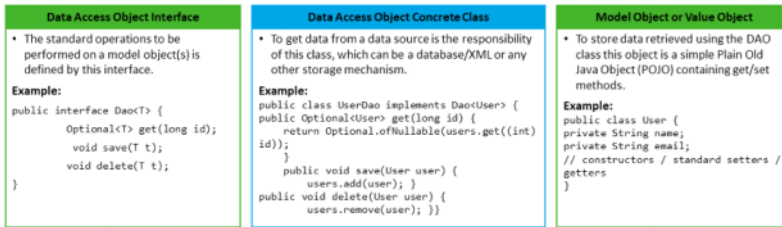
BINARY, VARBINARY, and LONGVARBINARY

- BINARY represents a small, fixed-length binary value.
- VARBINARY represents a small, variable-length binary value.
- LONGVARBINARY represents a large, variable-length binary value.

DAO PATTERN

Data Access Object(DAO) Pattern

- Using an abstract API the Data Access Object (DAO) pattern is a structural pattern that allows us to isolate the business/application layer from the persistence layer. The functionality of this API is to hide from the application all the complexities involved in performing Create, Read, Update, and Delete (CRUD) operations in the underlying storage mechanism.
- This allows both layers to evolve separately without knowing anything about one another.
- Following are the participants in the Data Access Object pattern:

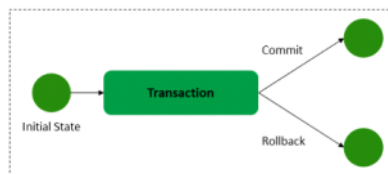


Transaction Management

- A transaction is a group of related actions that need to be performed as a single action.
- A logical unit of work whose effect is visible outside the transaction either in its entirety or not at all, is a transaction.

In the ACID properties,

- Atomicity (A)** means either all successful or none;
- Consistency (C)** ensures bringing the database from one consistent state to another consistent state;
- Isolation (I)** ensures that the transaction is isolated from other transactions; and
- Durability (D)** means that once a transaction has been committed, it will remain so, even in the event of errors or power loss.

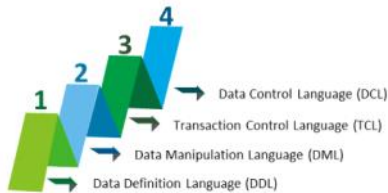


Methods in the Connection interface to manage transaction:

- void **setAutoCommit**(boolean status): Commits each transaction by default
- void **commit**(): commits the transaction
- void **rollback**(): cancels the transaction

DBMS

- A Database Management System (DBMS) is a software system that allows users to interact with databases.
- It provides functionalities for creating, manipulating, and managing databases efficiently.
- Structured Query Language (SQL) is used for handling databases. SQL commands are essential for managing data within a relational database management system (RDBMS) and controlling access to data.
- Here's an overview of SQL commands categorized into four main types:



- Each category of SQL commands serves a specific purpose in the database management process, ensuring data integrity, security, and efficiency.

ROW - RECORD , COLUMN - FIELD

Database Terminologies

Database	A collection of organized data stored electronically in a computer system.
Table	- A collection of related data organized in rows and columns. - Each row represents a record, and each column represents a field.
Row	A single record in a table containing data for each column.
Column	A vertical entity in a table that contains all information associated with a specific field in a table.
Primary Key	A unique identifier for each record in a table. It ensures that each row in a table is uniquely identifiable.
Foreign Key	A field in one table that uniquely identifies a row of another table or establishes a relationship between two tables.

Database Terminologies (Cont.)

>

Index	A data structure that improves the speed of data retrieval operations on a database table at the cost of additional space and decreased performance during INSERT, DELETE, and UPDATE operations.
Query	A request for data or information from a database table or combination of tables, usually in the form of a SQL statement.
Transaction	A sequence of operations performed as a single logical unit of work that must be either fully completed or fully rolled back.
Normalization	The process of organizing data in a database efficiently, reducing data redundancy and improving data integrity.
Denormalization	The process of intentionally introducing redundancy into a database table to improve performance by eliminating joins and simplifying queries.
Stored Procedure	A precompiled collection of SQL statements and procedural logic stored in the database and executed as a single unit.

Database Terminologies (Cont.)

>

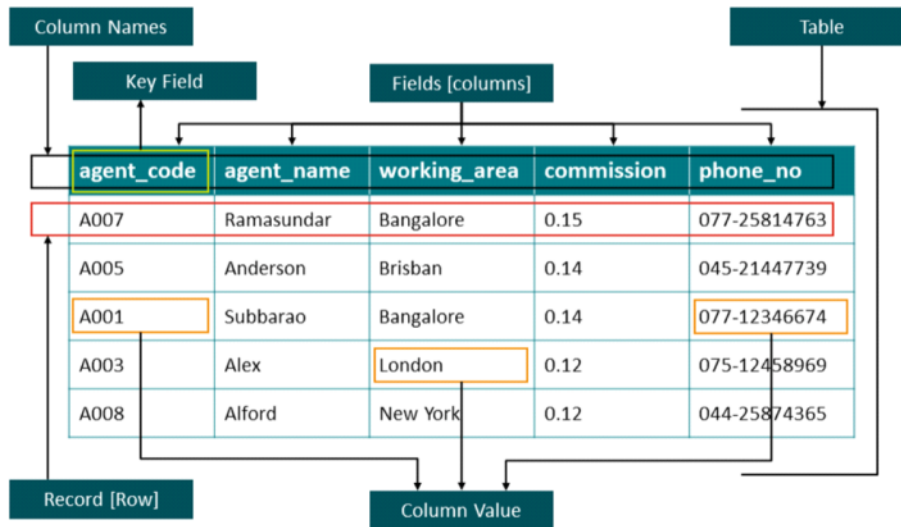
Trigger	A database object that automatically executes a specified set of SQL statements in response to certain events like INSERT, UPDATE, or DELETE operations on a table.
View	A virtual table generated by a query that can be used like a table in SQL statements but does not store data on its own.
Schema	A logical container for database objects such as tables, views, indexes, etc., which helps in organizing and managing database objects.

DB tables

/>

- All the data in a database is contained in database objects called tables.
- Similar to a spreadsheet, data is logically organized in a row-and-column format in a table. A field in the record is represented in each column, and each row represents a unique record.
- Every column must have a data type and name, which brings the table into existence.
- These columns store data according to the defined data types, and these data records are called rows.
- Every relational table has one primary key.
- The primary key's purpose is to identify each row in the database uniquely. Note that no two rows can have the same primary key value.

A Typical SQL DB Table Structure



SQL Commands

• Data Definition Language (DDL):

CREATE	Used to create database objects like tables, indexes, views, etc.
ALTER	Modifies the structure of existing database objects
DROP	Deletes database objects like tables, indexes, views, etc.
TRUNCATE	Deletes all records from a table but keeps its structure intact
RENAME	Renames an existing database object

• Data Manipulation Language (DML):

SELECT	Retrieves data from one or more tables
INSERT	Adds new records into a table
UPDATE	Modifies existing records in a table
DELETE	Removes records from a table

SQL Commands (Cont.)

• Transaction Control Language (TCL):

COMMIT	Saves all transactions to the database
ROLLBACK	Reverts the database to its previous state since the last COMMIT
SAVEPOINT	Sets a point within a transaction to which you can later roll back
SET TRANSACTION	Specifies transaction characteristics like isolation level and access mode

• Data Control Language (DCL):

GRANT	Provides specific privileges to database users
REVOKE	Removes previously granted privileges from database users
DENY	Prevents a user or group from performing actions on a database object

Constraints

- Constraints are the restrictions that help maintain the data integrity inside a database table.
- Constraints can be divided at the column level to limit only column data or at the table level to limit whole data in a database table.

Commonly Used Constraints	
Not Null	Restricts a column from the null value
Unique	Restricts a column to having different values, which means that a column cannot have a duplicate value
Primary Key	In a database table, each record is uniquely identified; ensures that the column values are unique and not null
Foreign Key	Uniquely identifies each record in a different table
Check	Restricts a column value, if it does not satisfy the given condition
Default	When the user doesn't specify a value it will set a default value to the column.

Singleton Design Pattern

- The Singleton pattern is a design pattern that ensures a class has only one instance and provides a global point of access to that instance.
- This is achieved by making the class responsible for its own instantiation and ensuring that no other instance can be created.
- It's commonly used in scenarios where only one instance of a class is needed, such as managing resources, configurations, or caches.
- In the context of JDBC, Using this Singleton pattern ensures that there's only one database connection object shared across the application, which helps in efficient resource management and avoids connection overhead.

Implementing a Singleton Pattern for JDBC

- In the context of JDBC (Java Database Connectivity), using the Singleton pattern can be beneficial for managing database connections efficiently.
- By creating a Singleton class to handle database connections, you ensure that there's only one connection object throughout the application, preventing resource wastage and simplifying connection management.

```
public class DatabaseSingleton {
    private static DatabaseSingleton instance;
    private Connection connection;

    private DatabaseSingleton() {
        try {
            // Initialize the database connection
            connection =
                DriverManager.getConnection("jdbc:mysql://localhost:3306/
                mydatabase", "username", "password");
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

```
public static DatabaseSingleton getInstance() {
    if (instance == null) {
        instance = new DatabaseSingleton();
    }
    return instance;
}

public Connection getConnection() {
    return connection;
}
}
```

Which of the following types of JDBC driver uses database native protocol?

- A. JDBC-ODBC Bridge + ODBC Driver
- B. Native API, which is a partly Java driver
- C. JDBC-Net, which is a pure Java driver
- D. Native-protocol, which is a pure Java driver

NATIVE API, WHICH IS A PARTLY JAVA DRIVER

Which JDBC driver types can be used in either applet or servlet codes?

- A. Both Type 1 and Type 2
- B. Both Type 1 and Type 3
- C. Both Type 3 and Type 4
- D. Type 4 only

TYPE 3 AND 4

Knowledge Check 3

The JDBC API has always supported persistent storage of objects defined in the Java programming language through the getObject and setObject methods.

Yes

No

NO

Which of the following methods are needed for loading a database driver in JDBC?

- A. registerDriver() method
- B. Class.forName()
- C. Both A and B
- D. getConnection()

Class.forName()

BLOB, CLOB, ARRAY, and REF type of columns can be updated in _____.

- A. JDBC 1.0
- B. JDBC 4.0
- C. JDBC 2.0
- D. JDBC 3.0

JDBC 2.0 this version introduced support for large objects including BLOB and CLOB, as well as support for SQL3 data types such as ARRAY and REF

Activity Details	
Problem Statement	Write a Java program to perform CRUD operations on a DB table using JDBC statements.
Sample Input	N/A

Summary

The key learning points of the module:

- JDBC stands for Java Database Connectivity. It is an API (Application Programming Interface) that provides a standardized way for Java applications to interact with databases.
- JDBC allows Java programs to connect to and interact with relational databases such as MySQL, Oracle, SQL Server, etc.
- JDBC follows a layered architecture consisting of various components, including the DriverManager, Connection, Statement, ResultSet, and Driver.
- JDBC drivers are required to establish a connection between Java applications and databases. There are four types of JDBC drivers: Type 1 (JDBC-ODBC bridge), Type 2 (native API), Type 3 (network protocol), and Type 4 (pure Java).
- JDBC provides the Connection interface to establish a connection to a database. The connection can be obtained using the DriverManager class by providing the appropriate database URL, username, and password.
- JDBC offers two types of statements, Statement and PreparedStatement. Statements are used for executing SQL queries, while PreparedStatements allow parameterized queries and offer better performance and security.
- JDBC ResultSets are used to retrieve data from a database after executing a query. It provides methods to navigate through the results and access individual data elements.
- JDBC supports transaction management, which allows multiple database operations to be treated as a single unit of work. Transactions ensure consistency and integrity of data.

Summary (Cont.)

Here are the key learning points of the module (cont.):

- JDBC provides specific exceptions that help in handling errors and exceptions that may occur during database operations, such as SQLException, etc.
- Overall, JDBC is a powerful and widely used API that simplifies database connectivity and interaction for Java applications, enabling them to perform database operations efficiently and securely.

Servlets

Sunday, August 4, 2024 4:12 PM

Agenda

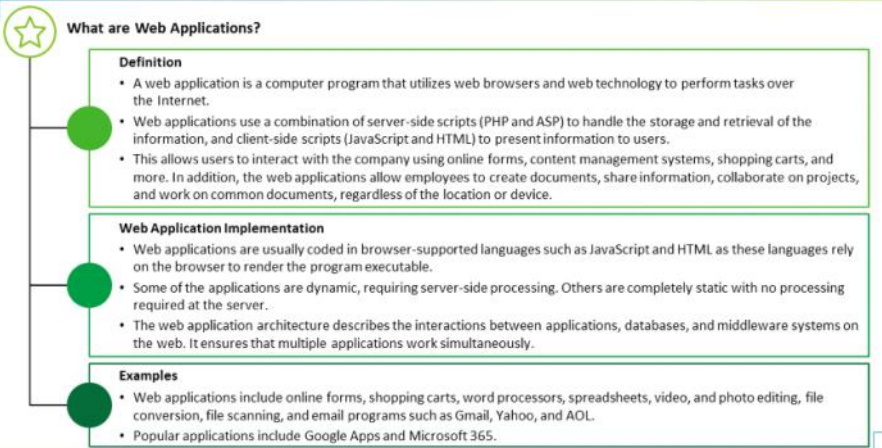
Topic	Descriptions	Duration
Web Application and Terminology	Web Application, Terminology, and Client-Server Architecture	20 hours
Servlets	Introduction to Servlets, Generic vs HttpServlet, HttpServlet Life cycle, HttpServlet, Methods- doGet, doPost, Ways of Creating Servlet, Working of a Servlet, Servlet – Request, Response Objects, Servlet Context & Servlet Config, Servlet Registration, Request Dispatcher: include(), forward(), Web application Deployment process, sendRedirect(), and Session Tracking Mechanism	

Learning Objectives

By the end of this module, you will be able to:

- Explain the concept of web applications along with their real-time uses
- Demonstrate the process of creating a web application using Java servlets
- Describe the lifecycle and working of a servlet explaining request/response objects along with servlet context and config
- Use appropriate technique/approach for creating Java servlets
- Understand the Java web application deployment process, importance of .war and .ear files and their creation
- Discuss the concept of session along with its tracking mechanisms for user authentication
- Illustrate the steps for creating cookies and using HttpSession object to identify session details
- Contrast the characteristics of Generic and Http Servlet classes

Introduction



Process Flow

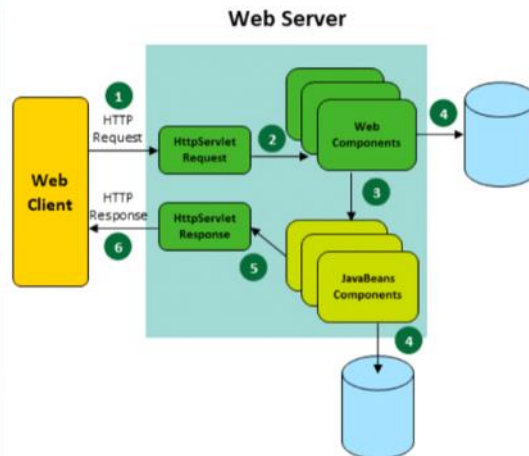
</>

Web Client

- It is the client or user side of the web.
- It typically refers to the web browser on the user's machine.
- It may also refer to plugins and helper applications that enhance the browser to support special services from the site.

HttpServletRequest/Response

- The request and response communications over the internet between the client and server are held using the HTTP protocol. That's why the server is often called an HTTP server.
- When a server answers a request, the server usually sends some type of content to the browser so that the browser can display it.



Web Server

- It is the web server that implements Java Servlet and Java Server pages. The technology converts the request into an HttpServletRequest object.
- This object is delivered to a web component, which can interact with JavaBeans components or a database to generate dynamic content.

Web Component

- The web component can then generate an HttpServletResponse or it can pass the request to another web component.
- Eventually a web component generates an HttpServletResponse object.
- The web server converts this object to an HTTP response and returns it to the client.

Web Terminology

Terms	Details
Web Server	• A web server is a computer that runs websites. It's a computer program that distributes web pages as they are requisitioned. • The basic objective of the web server is to store, process, and deliver web pages to the users.
Static and Dynamic Website	• A static website contains web pages with fixed content. • A dynamic website can have dynamic information based on the user-entered data.
URL	• A Uniform Resource Locator (URL) incorporates the domain name, along with other detailed information, to create a complete address or web address to direct a browser to a specific page online called a web page.
Request/Response	• Request is the data passed from the front end (browser) to the web server for processing. • Response is the data passed from the web server to the front end after processing the request.
Session	• In computer systems, a user session begins when a user logs in to or accesses a particular computer, network, or software service and ends after a certain time period or when the user logs out.
HTTP	• HyperText Transfer Protocol (HTTP) is the protocol used to transfer data over the web. • It is part of the Internet protocol suite and defines commands and services used for transmitting the web page data.

High-Level Steps Involved in Implementing Web Applications

</>

Step 1 Create a directory structure/project as per standards.

Step 2 Create a servlet class extending HttpServlet/GenericServlet classes.

Step 3 Compile the servlet.

Step 4 Add mappings to the web.xml file to register the servlet.

Step 5 Create an HTML/JSP page to access the servlet.

Step 6 Start the server and deploy the project. Access the HTML/JSP through a web browser and check if the response returned by the servlet is as expected.

A JavaBean is a Java class that should follow the following conventions:

- It should have a no-arg constructor.
- It should be Serializable.
- It should provide methods to set and get the values of the properties, known as getter and setter methods.

Why use JavaBean?

According to Java white paper, it is a reusable software component. A bean encapsulates many objects into one object so that we can access this object from multiple places. Moreover, it provides easy maintenance.

Server-Side Programming—Overview

</>

Server-side programming is very useful because it allows us to efficiently deliver information tailored for individual users and thereby create a much better user experience.

Server-side programming allows us to instead store the information in a database and dynamically construct and return HTML and other types of files, such as PDFs and images. It is also possible to simply return data (JSON, XML, etc.) for rendering by appropriate client-side web frameworks. This reduces the processing burden on the server and the amount of data that needs to be sent.

The server is not limited to sending information from databases and might alternatively return the result of software tools or data from the communication services. The content can even be targeted for the type of client device that is receiving it.

Servers can store and use information about clients to provide a convenient and tailored user experience. For example, many sites store credit cards so that the details don't have to be entered again. Sites like Google Maps can use saved or current locations for providing routing information, and search or travel history to highlight local businesses in search results.

Server-side programming allows developers to make use of sessions, or basically, a mechanism that allows a server to store information on the current user of a site and send different responses based on that information.

Features of Server-Side Programming



Knowledge Check 1

Servlets are used to program which component in a web application?

A. Client

B. Server

C. Tomcat

D. Applet

Knowledge Check 2

What type of protocol is HTTP?

A. Stateless

B. Stateful

C. Transfer protocol

D. Information protocol

Knowledge Check 3

A program that accepts requests from a web browser and sends back results in the form of HTML documents is known as _____.

A. Web Host

B. Web Server

C. Web Interface

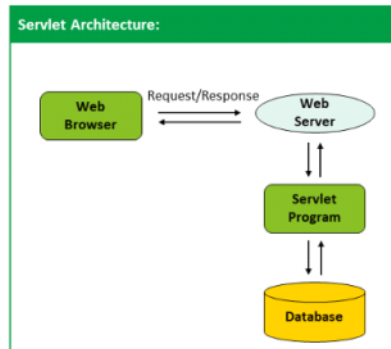
D. Web Application

What are Servlets?

- Handle web server requests. They process the request, produce the response, and then send a response back to the web server
- Work on the server side and send a response to the front end to provide user-specific data
- Connect the front-end to the database
- Are part of the Java Enterprise Edition (Java EE) that was later renamed Jakarta EE. Jakarta EE is a collection of specifications and technologies for building enterprise-level Java applications
- Play a crucial role in web development within the Java EE/Jakarta EE ecosystem

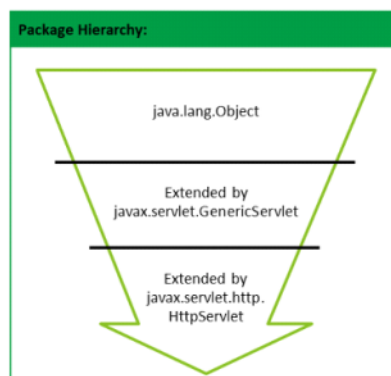
Overview of Servlets

- Servlets are used to handle the requests obtained from the web server, process the request, produce the response, then send a response back to the web server.
- Servlets work on the server side.
- Execution of servlets involves the following basic steps:
 - The client sends the request to the web server.
 - The web server receives the request.
 - The web server passes the request to the corresponding servlet.
 - The servlet processes the request and generates the response in the form of an output.
 - The servlet sends the response back to the web server.
 - The web server sends the response back to the client and the client browser displays it on the screen.



Servlet API

- You need to use Servlet API to create servlets.
- The two packages that you must remember while using API are:
 - The `javax.servlet` package that contains the classes to support generic servlet (protocol-independent servlet)
 - The `javax.servlet.http` package that contains classes to support HTTP servlet
- Some of the many classes in the `javax.servlet.http` package are:
 - `HttpServlet`
 - `Cookie`
 - `HttpServletRequestWrapper`
 - `HttpServletResponseWrapper`
 - `HttpSessionEvent`
 - `HttpSessionBindingEvent`
 - `HttpUtils` (deprecated now)



Servlet Interface

</>

- The Servlet interface provides common behavior to all the servlets by defining methods that all servlets must implement.
- The Servlet interface needs to be implemented for creating any servlet (either directly or indirectly).
- It provides three life cycle methods that are used to initialize the Servlet, to service the requests, and to destroy the Servlet, and two non-lifecycle methods.
- There are five methods in the Servlet interface:
 - public void init(ServletConfig config)
 - public void service(ServletRequest request, ServletResponse response)
 - public void destroy()
 - public ServletConfig getServletConfig()
 - public String getServletInfo()
- The init, service, and destroy are the life cycle methods of a servlet. These are invoked by the web container.

Example:

```
import java.io.*;
import javax.servlet.*;

public class First implements Servlet {
    ServletConfig config=null;

    public void init(ServletConfig config){
        this.config=config;
        System.out.println("servlet is initialized");
    }

    public void service(ServletRequest req, ServletResponse res) |
        throws IOException, ServletException {
        res.setContentType("text/html");

        PrintWriter out=res.getWriter();
        out.print("<html><body>");
        out.print("<b>hello simple servlet</b>");
        out.print("</body></html>");
    }

    public void destroy(){System.out.println("servlet is destroyed");}
    public ServletConfig getServletConfig(){return config;}
    public String getServletInfo(){return "Servlet info";}
}
```

Servlet Interface—API

</>

public void init(ServletConfig config)

Initializes the servlet; is the life cycle method of servlet and is invoked by the web container only once

public void service(ServletRequest request, ServletResponse response)

Responds to the incoming request and is invoked at each request by the web container

public void destroy()

Is invoked only once and indicates that the servlet is being destroyed

public ServletConfig getServletConfig()

Returns the object of ServletConfig

public String getServletInfo()

Returns information about servlet such as writer, copyright, and version

GenericServlet—Class

/>

- **GenericServlet Class**
 - Implements Servlet, ServletConfig, and Serializable interfaces
 - provides the implementation of all the methods of these interfaces except the service method
 - Can handle any type of request so it is protocol-independent
- **Pros of Using GenericServlet:**
 - Is easier to write
 - Has simple life cycle methods
 - Can be written by extending javax.servlet.GenericServlet and overriding the service() method
- **Cons of using GenericServlet:**
 - Working with Generic Servlet is not that easy because we don't have convenience methods, such as doGet(), doPost(), and doHead(), in Generic Servlet that we can use in HTTP Servlet.

Example:

```
import java.io.*;
import javax.servlet.*;

public class First extends GenericServlet{
    public void service(ServletRequest req,ServletResponse res)
    throws IOException,ServletException{

        res.setContentType("text/html");

        PrintWriter out=res.getWriter();
        out.print("<html><body>");
        out.print("<b>hello generic servlet</b>");
        out.print("</body></html>");

    }
}
```

HttpServlet—Class

</>

- Unlike Generic Servlet, the HTTP Servlet doesn't override the service() method. Instead, it overrides the **doGet()** method or **doPost()** method or both.
- The doGet() method is used for getting the information from the server while the doPost() method is used for sending information to the server.
- HttpServlet class is protocol dependent.
- In Http Servlet, there is no need to override the service() method because this method dispatches the HTTP Requests to the correct method handler.
- For example, if it receives HTTP GET Request, it dispatches the request to the doGet() method.

Example:

```
// Import required java libraries
import java.io.*;

// Extend HttpServlet class
public class HelloWorld extends HttpServlet {

    private String message;

    public void init() throws ServletException {
        // Do required initialization
        message = "Hello World";
    }

    public void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {

        // Set response content type
        response.setContentType("text/html");

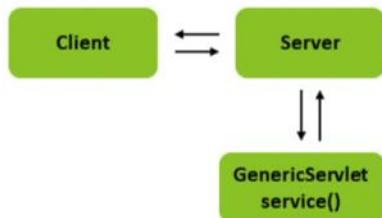
        // Actual logic goes here.
        PrintWriter out = response.getWriter();
        out.println("<h1>" + message + "</h1>");
    }

    public void destroy() {
        // do nothing.
    }
}
```

Process: GenericServlet vs. HttpServlet

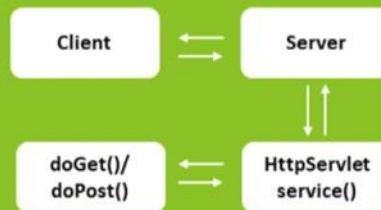
Process—GenericServlet

1. The client sends a request to the server.
2. The server calls the GenericServlet service() method.
3. Service() method processes the request and sends the response object to the server.
4. The server sends a response to the client for a user to see the data.



Process—HttpServlet

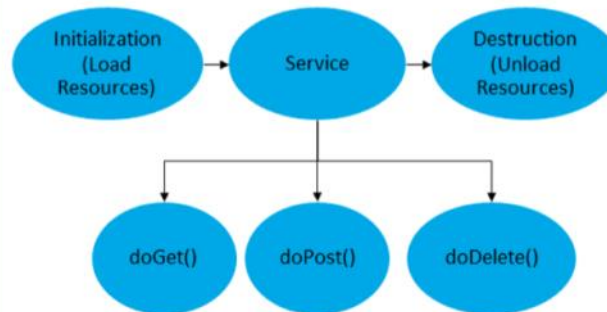
1. The client sends a request to the server.
2. The server calls the HttpServlet service() method.
3. The Service() method then dispatches the request to the doGet() or doPost() method according to the request type.
4. Server sends a response to the client for a user to see the data.



HttpServlet Lifecycle

The lifecycle of HttpServlet begins as soon as it is called by the web server to load into the container.

Flow Diagram:



Three Phases in the HttpServlet Lifecycle

</>

Instantiation and Initialization Phase:

- In the initialization phase, the servlet container starts by locating servlet classes either in the local file system or from a remote location.
- The classes critical to starting the application must be located at startup. The initialization process begins after servlet objects are instantiated.
- It includes reading persistent configuration data, and JDBC connection parameters. The container invokes the `init()` method of the Servlet interface for initialization.

Service Phase:

- The service phase represents the interaction with the request until the duration of the service is destroyed.
- The request and response are mapped to the service method, which in turn delegates the handle to the `doXXX()` methods.
- The request and responses are mapped to the objects of `ServletRequest` and `ServletResponse`, respectively.

Destruction Phase

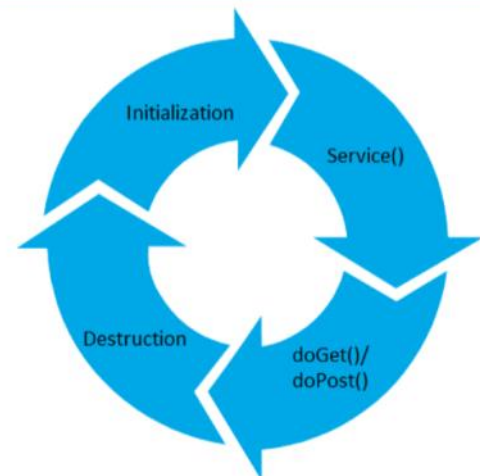
- In this phase, the servlet is removed from the container. The container invokes the `destroy` method of the Servlet interface.
- This `destroy` method ensures that the servlet is terminated gracefully and cleans up all the resources used or created during its lifecycle.

HttpServlet Lifecycle—Conclusion

</>

- The proper use of the lifecycle phases (Initialization, Service, and Destruction) ensures that the servlet manages the application resources efficiently.
- During initialization, the servlet loads up all resources it needs to process requests in the later phase.
- The service phase uses these resources to respond as per requests made.
- In the destruction phase, the memory is freed up with the cleanup process.

Flow Diagram:



HTTP Request Methods—GET vs. POST

</>

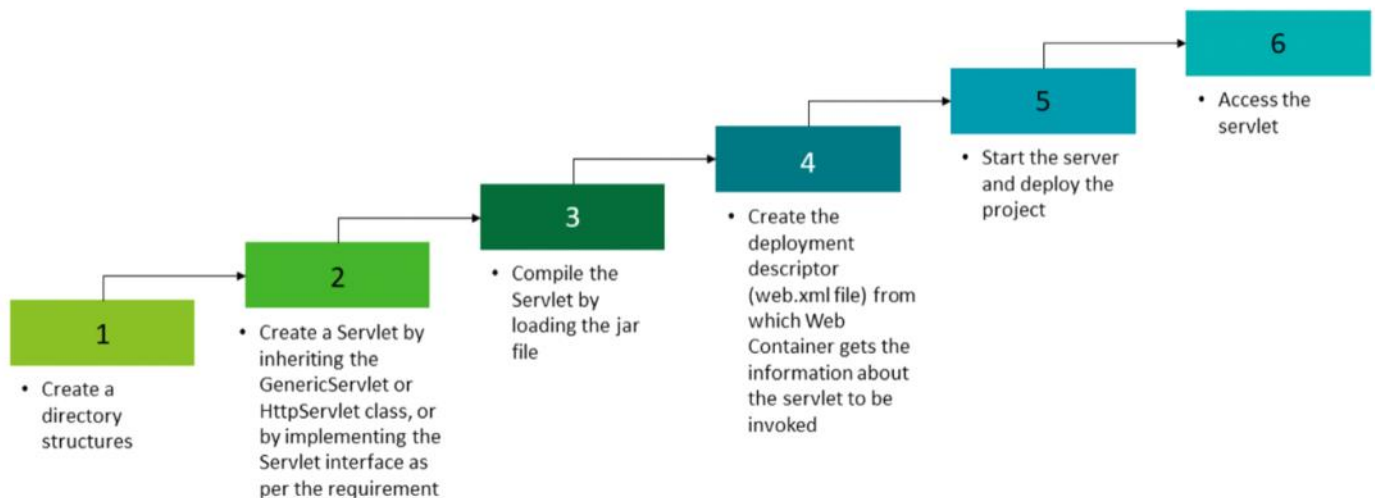
- Retains parameters in the browser history because they are part of the URL
- Can be bookmarked
- Re-executes requests but may not re-submit them to the server, if the HTML is stored in the browser cache
- Can send but the parameter data is limited to what we can stuff into the request line (URL)—the safest is to use less than 2K of parameters, but some servers handle even up to 64K
- Allows only ASCII characters
- Is less secure as compared to Post because data sent is part of the URL and is saved in the browser history and server logs in plaintext
- Is visible to everyone (it will be displayed in the browser's address bar) and has limits on the amount of information to send
- Can be cached

Get vs. Post

- Does not have parameters saved in the browser history
- Cannot be bookmarked
- Usually allows the browser to alert the user that data will need to be resubmitted
- Can send parameters, including uploading files, to the server
- Has no restrictions, and allows binary data
- Is a little safer than GET because the parameters are not stored in the browser history or in web server logs
- Does not display variables in the URL
- Is not cached

Steps to Create a Servlet

</>



Servlet Example

</>

Here is an example of a servlet for saving user input data into the database.

Overview of the mentioned code:

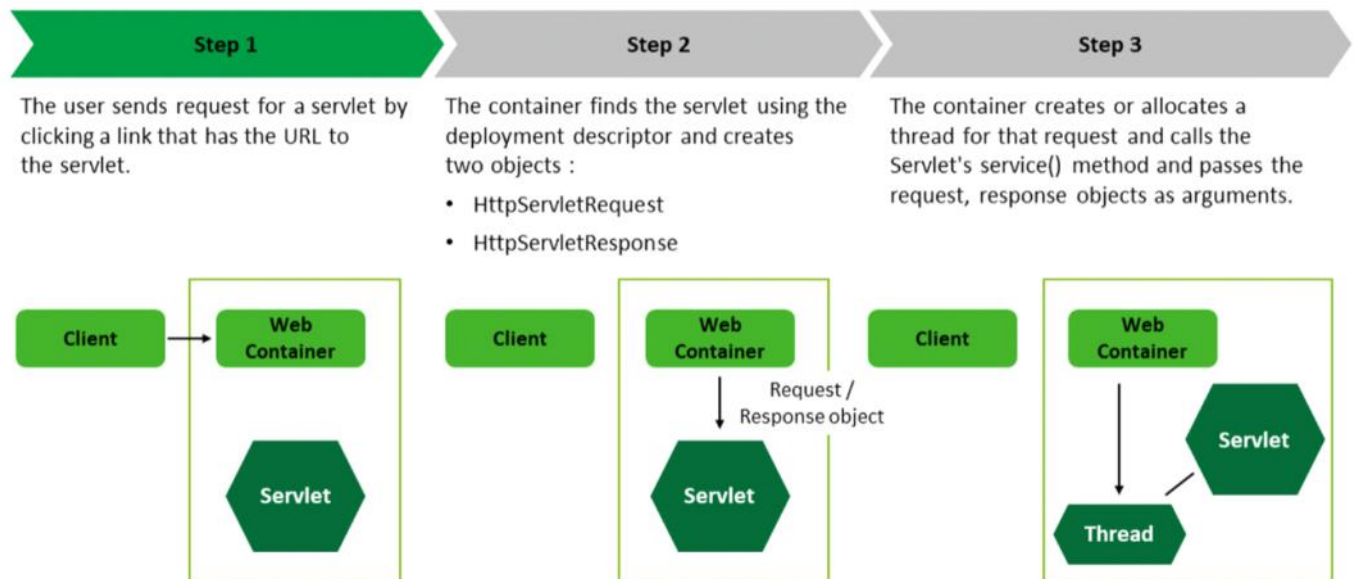
- Fetches data sent from UI (User Interface) through the respective parameters - name, password, email, country
- Creates a Model (Object of Emp class) to hold the data from point one
- Calls the DAO Class API 'save()' and passes the model, where the data will be persisted in DB
- Sends a statement/message to the requestor in response to the original client request—as the outcome of processing

Example:

```
public class SaveServlet extends HttpServlet {  
    protected void doPost(HttpServletRequest request, HttpServletResponse response)  
        throws ServletException, IOException {  
        response.setContentType("text/html");  
        PrintWriter out=response.getWriter();  
  
        String name=request.getParameter("name");  
        String password=request.getParameter("password");  
        String email=request.getParameter("email");  
        String country=request.getParameter("country");  
  
        Emp e=new Emp();  
        e.setName(name);  
        e.setPassword(password);  
        e.setEmail(email);  
        e.setCountry(country);  
  
        int status=EmpDao.save(e);  
        if(status>0){  
            out.print("<p>Record saved successfully</p>");  
            request.getRequestDispatcher("index.html").include(request, response);  
        }else{  
            out.println("Sorry! unable to save record");  
        }  
        out.close();  
    }  
}
```

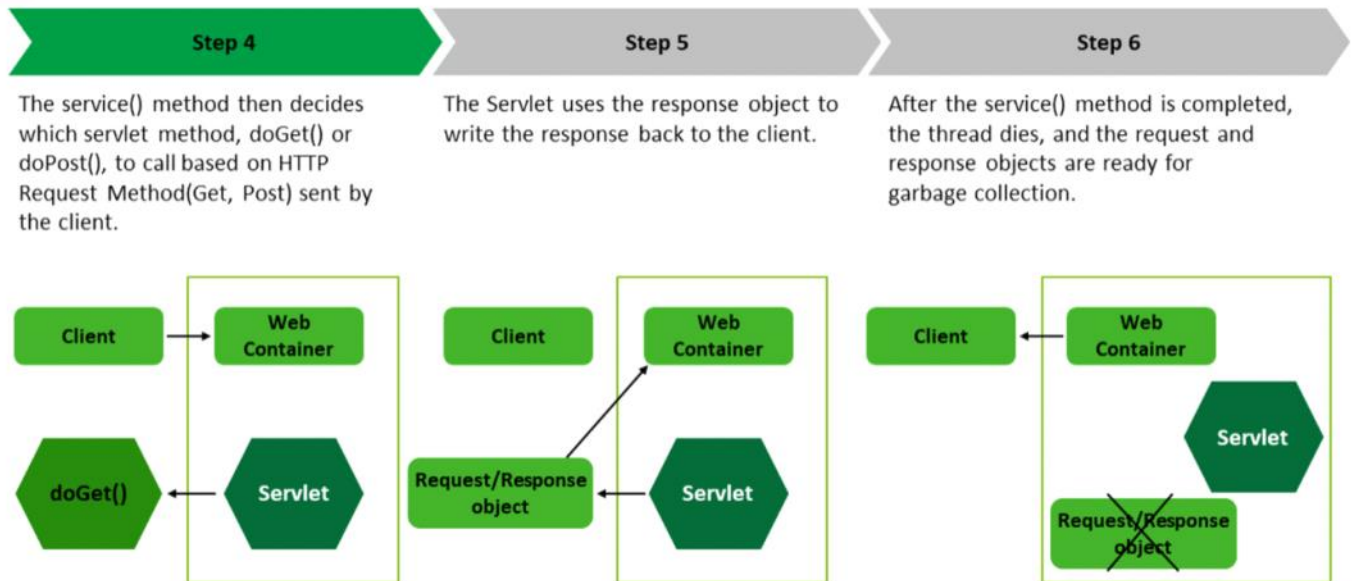
How does a Servlet work?

</>



How a Servlet Works (Cont.)

</>



Ways of Servlet Registration

- Registering servlets in a Java web application can be done using either XML configuration via the 'web.xml' file or annotations.
- Both approaches are commonly used in Java web development, with annotations being more prevalent in modern projects due to their simplicity and integration with the servlet class itself.

XML Approach Using web.xml

- **Define Servlet Configuration:**
 - Create or edit the 'web.xml' file located in the 'WEB-INF' directory of the web application.
 - Use the '<servlet>' element to define the servlet, specifying its name and class.
 - Use the '<servlet-class>' element to specify the fully qualified class name of the servlet.
- **Map Servlet to URL Pattern:**
 - Use the '<servlet-mapping>' element to map the servlet to a URL pattern.
 - Specify the servlet name defined earlier and the URL pattern to which the servlet should respond.
- **Example 'web.xml' Configuration:**

```
<web-app>
  <servlet>
    <servlet-name>HelloServlet</servlet-name>
    <servlet-class>com.example.HelloServlet</servlet-class>
  </servlet>
  <servlet-mapping>
    <servlet-name>HelloServlet</servlet-name>
    <url-pattern>/hello</url-pattern>
  </servlet-mapping>
</web-app>
```

Annotation Approach

- **Use `@WebServlet` Annotation:**

- Annotate the servlet class with `@WebServlet` annotation, specifying the URL pattern(s) to which the servlet should respond.
- This annotation is available since Servlet 3.0 specification.

- **Example Servlet with Annotation:**

```
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import java.io.IOException;

@WebServlet("/hello")
public class HelloServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws IOException {
        response.getWriter().println("Hello, World!");
    }
}
```

RequestDispatcher

- The RequestDispatcher interface provides the facility of dispatching the request to another resource which may be HTML, servlet, or JSP.
- This interface can also be used to include the content of another resource which is one of the ways of servlet collaboration.
- **public void forward(ServletRequest request, ServletResponse response)** forwards a request from a servlet to another resource (servlet, JSP file, or HTML file) on the server.
- **public void include(ServletRequest request, ServletResponse response)** includes the content of a resource (servlet, JSP page, or HTML file) in the response.
- Both methods throw IOException.
- The servlet container creates the RequestDispatcher object, which is used as a wrapper around a server resource located at a particular path or given by a particular name.

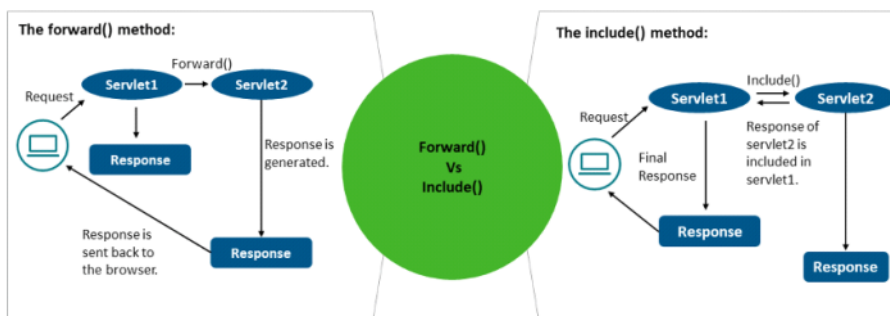
RequestDispatcher

- The RequestDispatcher interface provides the facility of dispatching the request to another resource which may be HTML, servlet, or JSP.
- This interface can also be used to include the content of another resource which is one of the ways of servlet collaboration.
- **public void forward(ServletRequest request, ServletResponse response)** forwards a request from a servlet to another resource (servlet, JSP file, or HTML file) on the server.
- **public void include(ServletRequest request, ServletResponse response)** includes the content of a resource (servlet, JSP page, or HTML file) in the response.
- Both methods throw IOException.
- The servlet container creates the RequestDispatcher object, which is used as a wrapper around a server resource located at a particular path or given by a particular name.



Forward and include(request,response) both throw IOException

RequestDispatcher (Contd.)



forward() vs. sendRedirect()



- The sendRedirect() method of the HttpServletResponse interface can be used to redirect response to another resource which may be a servlet, JSP, or HTML file.
- It accepts relative as well as absolute URLs.
- It works on the client side because it uses the URL bar of the browser to make another request, to be able to work inside and outside of the server.

forward()	sendRedirect()
• The forward() method works on the server side.	• The sendRedirect() method works on the client side.
• It sends the same request and response objects to another servlet.	• It always sends a new request.
• It can work within the server only.	• It can be used within and outside the server.
• Example: request.getRequestDispatcher("servlet2").forward(request, response);	• Example: response.sendRedirect("servlet2");

sendRedirect() method work on the client side, forward() method works on the server side, SendRedirect always sends a new request, forwards() sends the same req and response objects to another servlet. Forward works within the server where as sendRedirect works within and outside the server.

Ex : request.getRequestDispatcher("servlet2").forward(request, response)
Response.sendRedirect("servlet2")

Example

The below Login class extends HttpServlet class and shows an implementation of both forward() and include() methods:

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class Login extends HttpServlet {

    public void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        String n=request.getParameter("userName");
        String p=request.getParameter("userPass");

        if(p.equals("servlet")){
            RequestDispatcher rd=request.getRequestDispatcher("servlet2");
            rd.forward(request, response);
        }
        else{
            out.print("Sorry UserName or Password Error!");
            RequestDispatcher rd=request.getRequestDispatcher("/index.html");
            rd.include(request, response);
        }
    }
}
```

Knowledge Check 4

Which of the following codes is used to get an attribute in an HTTP Session object in servlets?

A. session.getAttribute(String name)

B. session.alterAttribute(String name)

C. session.updateAttribute(String name)

D. session.setAttribute(String name)

Session.getAttribute(String name)

When is the destroy() method of a filter called?

A. Only once at the end of the lifecycle of a filter

B. After the filter has executed the doFilter method

C. Only once at the beginning of the lifecycle of a filter

D. After the filter has executed

Only once at the end of the lifecycle of a filter

Which of the following sentences is true about servlets?

- A. Servlets execute within the address space of web server.
- B. Servlets are platform-independent because they are written in Java.
- C. Servlets can use the full functionality of the Java class libraries.
- D. Servlets execute within the address space of the web server as they are platform-independent and uses the functionality of Java class libraries.

Option d

Which of the following sentences is true about the init() method of servlet?

- A. The init() method simply creates or loads some data that will be used throughout the life of the servlet.
- B. The init() method is not called again and again for each user request.
- C. Both of the above
- D. None of the above

Both

Which of the following sentences is true about the HTTP Get method?

- A. The GET method sends the encoded user information appended to the page request.
- B. The GET method is the default method to pass information from the browser to web server.
- C. Both of the above
- D. None of the above

Both

Which of the following options includes the correct order of filter life cycle phase methods?

- | | |
|--------------------------------------|---|
| A. init(), service(), and destroy() | B. initialize(), service(), and destroy() |
| C. init(), doFilter(), and destroy() | D. init(), service(), and delete() |

Init(), service(), destroy()

Which of the following codes is used to get an HTTP Session object in servlets?

- A. `request.getSession()`
- B. `response.getSession();`
- C. `new Session()`
- D. None of the above

`Request.getSession();`

MVC Design Pattern

- MVC, which stands for Model-View-Controller, is a software architectural pattern commonly used in web development to organize the structure of an application.
- It separates an application into three interconnected components:

1

****Model**:**

- It represents the data and business logic of the application.
- It encapsulates the data and behavior of the application's domain and interacts with the database or other data sources.

2

****View**:**

- It represents the presentation layer of the application.
- It displays the data to the user and handles user interactions, such as input validation and form submission.

3

****Controller**:**

- It acts as an intermediary between the Model and the View.
- It receives user input from the View, processes it using the Model, and updates the View with the results.
- It typically contains application logic, such as request handling, data validation, and business logic.

Servlet as Controller

- In the context of Java web development using Servlets, the Servlet can serve as the Controller in the MVC architecture.
- Using Servlets as Controllers in an MVC architecture helps in separating concerns, promoting code modularity, and making the application easier to maintain and extend.

Here's how it fits into the MVC pattern:

Servlets receive incoming HTTP requests from clients and process them.

- They act as the Controller by handling the request, invoking appropriate methods on the Model to perform business logic, and forwarding the results to the View for presentation.
- Servlets can retrieve data from the Model (e.g., querying a database), perform necessary processing or manipulation, and then pass the data to the View for rendering.
- They are responsible for managing the flow of the application, routing requests to the appropriate components (e.g., forwarding requests to JSP pages or other servlets).

Example of a Servlet Acting as a Controller in an MVC Architecture

```
public class MyServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        // Receive user input from the request
        String username = request.getParameter("username");

        // Perform business logic (Model)
        String greetingMessage = "Hello, " + username + "!";

        // Set data in request attributes
        request.setAttribute("message", greetingMessage);

        // Forward request to a JSP view
        RequestDispatcher dispatcher = request.getRequestDispatcher("/WEB-INF/views/greeting.jsp");
        dispatcher.forward(request, response);
    }
}
```

Servlet Registration Example

Via web.xml:

- The most common way to register a servlet is to add it to your web.xml file by following these two steps:
 - Add the servlet to the servlet tag, making sure to also specify the source path to the class the servlet resides within
 - Specify the URL path that the servlet will be exposed on in the url-pattern tag.
- The web.xml file is usually found in WebContent/WEB-INF.

Example:

```
<welcome-file-list>
  <welcome-file>index.html</welcome-file>
  <welcome-file>index.htm</welcome-file>
  <welcome-file>index.jsp</welcome-file>
</welcome-file-list>
<servlet>
  <servlet-name>Example</servlet-name>
  <servlet-class>com.ExampleServlet</servlet-class>
</servlet>
<servlet-mapping>
  <servlet-name>ExampleServlet</servlet-name>
  <url-pattern>/ExampleServlet</url-pattern>
</servlet-mapping>
```

Servlet Registration Example (Contd.)

Via Annotations:

- Register the servlet using the @WebServlet annotation on the custom servlet class.
- This eliminates the need for servlet mappings in server.xml and registration of the servlet in web.xml.
- The code demonstrates how to add that annotation directly to a servlet.
- The servlet will still be available at the same URL path as before.

Example:

```
import java.io.IOException;

@WebServlet(
    name = "AnnotationExample",
    description = "Example Servlet Using Annotations",
    urlPatterns = {"/AnnotationExample"}
)
public class ExampleServlet extends HttpServlet {

    @Override
    protected void doGet(
        HttpServletRequest request,
        HttpServletResponse response) throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<px>Hello World!</px>");
    }
}
```

Servlet CRUD Example

Use Case:

- Create, update, delete and read a user in an application.

Steps:

1. Create a table in the database to store user information.
2. Create an HTML/JSP page to get user input from the screen.
3. Create a DAO class to perform database operations like create/delete/update.
4. Create Servlets for each operation to read requests from clients and send an appropriate response to clients.

HTML File:

```
<form action="SaveServlet" method="post">
  <table>
    <tr><td>Name:</td><td><input type="text" name="name"/></td></tr>
    <tr><td>Password:</td><td><input type="password" name="password"/></td></tr>
    <tr><td>Email:</td><td><input type="email" name="email"/></td></tr>
    <tr><td>Country:</td><td>
      <select name="country" style="width:150px">
        <option>India</option>
        <option>USA</option>
        <option>UK</option>
        <option>Other</option>
      </select>
    </td></tr>
    <tr><td colspan="2"><input type="submit" value="Save Employee"/></td></tr>
  </table>
</form>

<br/>
<a href="ViewServlet">view employees</a>
```

Servlet CRUD Example (Cont.)

</>

POJO File:

- Create a POJO class with all getter setter methods of the defined variables.

Emp POJO

```
public class Emp {  
    private int id;  
    private String name;  
    private String password;  
    private String email;  
    private String country;  
}
```

DAO File:

- Create a DAO class to perform all CRUD operations. An example of saving is shown.

```
public class EmpDao {  
    public static int save(Emp e){  
        try{ Connection con=EmpDao.getConnection();  
        PreparedStatement ps=con.prepareStatement(  
            "insert into user(name,password,email,country) values  
            (?,?,,?)"); ps.setString(1,e.getName());  
        ps.setString(2,e.getPassword());  
        ps.setString(3,e.getEmail());  
        ps.setString(4,e.getCountry()); ps.executeUpdate();  
        con.close(); }catch(Exception ex){ex.printStackTrace();}  
        return status; } }
```

Servlet CRUD Example (Cont.)

</>

Edit Servlet:

```
import java.io.*;  
import javax.servlet.*;  
  
@WebServlet("/EditServlet")  
public class EditServlet extends HttpServlet {  
    protected void doPost(HttpServletRequest request,  
        HttpServletResponse response)  
        throws ServletException, IOException {  
        response.setContentType("text/html");  
        PrintWriter out=response.getWriter();  
        String sid=request.getParameter("id");  
        int id=Integer.parseInt(sid);  
        String name=request.getParameter("name");  
        String password=request.getParameter("password");  
        String email=request.getParameter("email");  
        String country=request.getParameter("country");  
        Emp e=new Emp();  
        e.setId(id);  
        e.setName(name);  
        e.setPassword(password);  
        e.setEmail(email);  
        e.setCountry(country);  
        int status=EmpDao.update(e);  
        if(status>0){  
            response.sendRedirect("ViewServlet");  
        }else{  
            out.println("Not able to update record");  
        }  
        out.close();  
    }  
}
```

```
String password=request.getParameter("password");  
String email=request.getParameter("email");  
String country=request.getParameter("country");
```

```
Emp e=new Emp();  
e.setId(id);  
e.setName(name);  
e.setPassword(password);  
e.setEmail(email);  
e.setCountry(country);  
int status=EmpDao.update(e);  
if(status>0){  
    response.sendRedirect("ViewServlet");  
}else{  
    out.println("Not able to update record");  
}  
out.close();  
}
```

Servlet CRUD Example (Cont.)

</>

View Servlet:

```
import java.io.*
import javax.servlet.*
@WebServlet("/ViewServlet")
public class ViewServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {
        response.setContentType("text/html");
        PrintWriter out=response.getWriter();
        out.println("<a href='index.html'>Add New
Employee</a>");
        out.println("<h1>Employees List</h1>");
        List<Emp> list=EmpDao.getAllEmployees();
        out.print("<table border='1' width='100%'>");
```

```
out.print("<tr><th>Id</th><th>Name</th><th>Password</th>
<th>Email</th><th>Country</th>

        <th>Edit</th><th>Delete</th></tr>");
        for(Emp e:list){

            out.print("<tr><td>"+e.getId()+"</td><td>"+e.getNam
e()+"</td><td>"+e.getPassword()+"</td>
            <td>"+e.getEmail()+"</td><td>"+e.getCountry()+"</td>
            <td><a
href='EditServlet?id="+e.getId()+"'>edit</a></td>
            <td><a
href='DeleteServlet?id="+e.getId()+"'>delete</a></td>
            </tr>");
        }
        out.print("</table>");
        out.close();
    }
}
```

Servlet CRUD Example (Cont.)

</>

Delete Servlet:

```
import java.io.IOException;
import javax.servlet.*;
@WebServlet("/DeleteServlet")
public class DeleteServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {
        String sid=request.getParameter("id");
        int id=Integer.parseInt(sid);
        EmpDao.delete(id);
        response.sendRedirect("ViewServlet");
    }
}
```

Save Servlet:

```
import java.io.*;
import javax.servlet.*;
@WebServlet("/SaveServlet")
public class SaveServlet extends HttpServlet {
    protected void doPost(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {
        response.setContentType("text/html");
        String name=request.getParameter("name");
        String password=request.getParameter("password");
        String email=request.getParameter("email");
        String country=request.getParameter("country");
        Emp e=new Emp();
        e.setName(name);
        e.setPassword(password);
        e.setEmail(email);
        e.setCountry(country);
        int status=EmpDao.save(e);
    }
}
```

Important Points to Remember!

Servlet Registration	There are two ways to register a Servlet in a web application: • Via web.xml • Via Annotation
Servlet Implementation	Two types of classes can be extended while creating a new Servlet class: • HttpServlet • GenericServlet
Servlet Implementation	• HttpServlet class is preferred over GenericServlet because of its flexibility to dispatch different request types to respective methods.
Servlet Implementation	Servlet has three methods: • init() • service() • destroy()

web.xml is located in which folder in the project?

A. WebContent

B. WebContent/META-INF

C. WebContent/WEB-INF

D. None of the above

<servlet-name> is inside which tag in web.xml?

A. <servlet> tag

B. <servlet-mapping> tag

C. <servlet-properties> tag

D. <servlet-attributes>

Which annotation is used to register Servlets in Java?

A. @Web

B. @Servlet

C. @WebApplication

D. @WebServlet

DAO layer is used to _____.

A. Validate the request.

B. Perform database operations.

C. Create a Servlet.

D. None of these

GO THROUGH THESE DOWN PAGES LATER

Java Web Application Deployment Process

- The deployment process for Java web applications involves several steps to package, configure, and deploy the application to a servlet container like Apache Tomcat.
- Here's a general overview of the deployment process:

Package the Application

- Compile the source code into bytecode (.class files) using a Java compiler.
- Organize the application resources, including HTML, JSP, CSS, JavaScript, and other static files, into appropriate directories.
- Create a directory structure that conforms to the standard layout for a web application, including the `WEB-INF` directory for configuration files and classes.
- Place the compiled `.class` files in the `WEB-INF/classes` directory.
- Place any required JAR files (libraries, dependencies) in the `WEB-INF/lib` directory.
- Optionally, include configuration files such as `web.xml` (deployment descriptor) and any additional context configuration files.

Create the WAR File

- Package the entire directory structure into a `.war` (Web Application Archive) file using a tool like `jar` or any ZIP utility.
- The resulting `.war` file contains all the necessary files and directories required for the web application to run.

v

Java Web Application Deployment Process (Cont.)

Configure Servlet Container	• Install and configure a servlet container such as Apache Tomcat. • Ensure that the servlet container is properly configured with the necessary settings, including network ports, security settings, and resource limits.
Deploy the WAR File	• Copy or upload the <code>.war</code> file to the appropriate directory within the servlet container's deployment directory (e.g., <code>webapps</code> directory in Tomcat). • Alternatively, use the servlet container's management interface (e.g., Tomcat Manager) to deploy the application by uploading the <code>.war</code> file through a web interface.
Start the Servlet Container	• Start or restart the servlet container to initiate the deployment process. • The servlet container will automatically detect the presence of the deployed <code>.war</code> file and proceed to deploy and initialize the web application.
Access the Application	Once the deployment process is complete, the web application should be accessible via its URL, typically in the format <code>http://localhost:8080/application-context</code> , where <code>application-context</code> is the name of the deployed <code>.war</code> file or the context path configured in the servlet container.

Java Web Application Deployment Process (Cont.)

Testing and Monitoring	• Test the deployed web application thoroughly to ensure that it functions correctly and meets the desired requirements. • Monitor the application's performance, resource usage, and error logs to identify any issues or potential improvements.
------------------------	--

By following these steps, you can effectively deploy Java web applications to a servlet container, making them accessible to users over the internet or within an intranet environment.

Manual .war File Creation

Compile Source Code	Compile your Java Servlet source code (<code>.java</code> files) into bytecode (<code>.class</code> files).
Create Directory Structure	Create a directory structure following the standard layout for a web application, including <code>WEB-INF</code> directory for configuration files and classes.
Add Configuration Files	Place <code>web.xml</code> (deployment descriptor) and any other configuration files required by your application in the <code>WEB-INF</code> directory.
Add Compiled Classes	Place compiled <code>.class</code> files in the <code>WEB-INF/classes</code> directory.
Add Libraries (JARs)	Place any required JAR files (libraries, dependencies) in the <code>WEB-INF/lib</code> directory.
Package	Finally, package the directory into a <code>.war</code> file using a tool like <code>jar</code> or any ZIP utility.

.war File Creation Using Maven

Project Setup:

Configure your Maven project with appropriate dependencies and plugins, including the 'maven-war-plugin'.

Project Structure:

Ensure your project follows Maven's standard directory structure for web applications.

Configure 'pom.xml':

Add necessary configurations to the 'pom.xml', such as dependencies, resource directories, and build plugins.

Build:

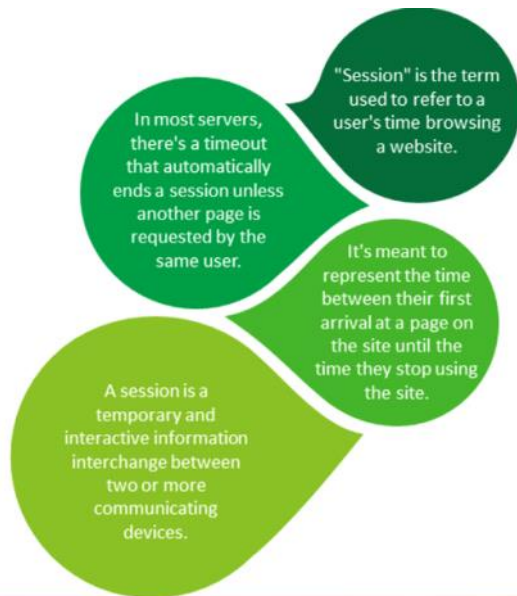
- Run 'mvn package' to build the project.
- This command will compile the source code, gather dependencies, and create the '.war' file in the 'target' directory.

Till here only the next lines are imp

Importance of .war and .ear files

Portability	• These file formats encapsulate the entire web or enterprise application, including all required resources, libraries, and configurations. • This makes it easy to deploy the application to any compatible servlet or application server.
Simplified Deployment	'war' and 'ear' files simplify the deployment process by providing a standard format that can be easily distributed and deployed to various environments without the need for manual configuration.
Isolation	By packaging the application along with its dependencies, these files help in isolating the application from the underlying system, reducing conflicts and ensuring consistency across different environments.
Versioning and Management	'war' and 'ear' files facilitate versioning and management of applications by providing a single artifact that represents a specific version of the application, making it easier to track changes and rollback if necessary.

Overall, '.war' and '.ear' files are essential for deploying Java web applications efficiently, ensuring portability, simplifying deployment, and providing a standardized way to package and distribute applications.



ved.

Session Tracking—What and Why?

</>

Session Tracking

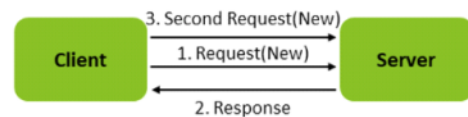
- Is a way to maintain the state (data) of a user and is also known as session management in servlet

HTTP

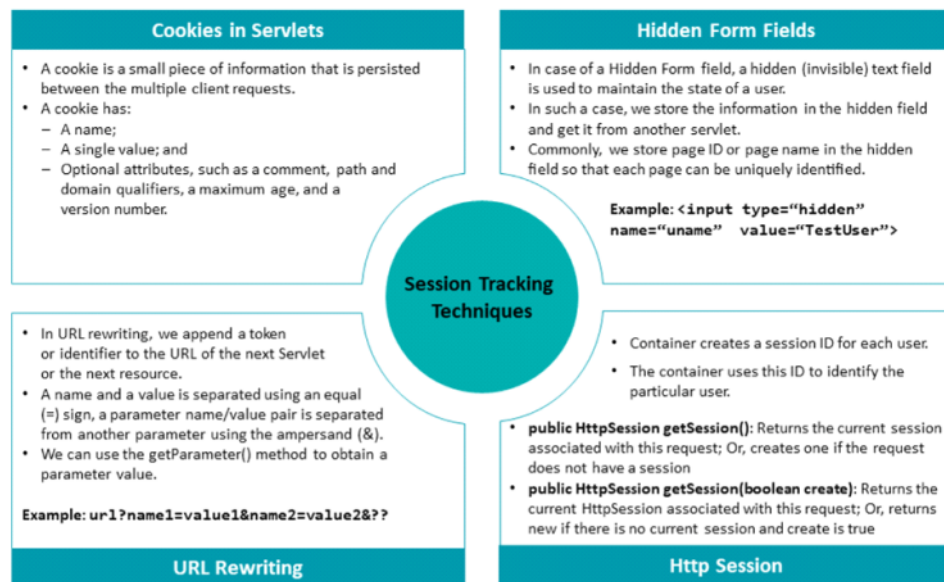
- Is a "stateless" protocol, which means each time a client retrieves a web page, the client opens a separate connection to the web server and the server automatically does not keep any record of the previous client request

Why to Use Session Tracking?

- Is used to recognize the particular user and to show the related user-specific data to the viewer



Session Tracking—Techniques



Session Tracking—Example (Cont.)

```
Date lastAccessed = new Date(session.getLastAccessedTime());
Date now = new Date();
DateFormat formatter = DateFormat.getDateInstance(DateFormat.MEDIUM,DateFormat.MEDIUM);

out.println("<html>");
out.println("<head>");
out.println("<title>Displaying the Session Creation and Last-Accessed Time</title>");
out.println("</head>");
out.println("<body>");
out.println("<h2>Session Creation and Last-Accessed Time</h2>");
out.println("The time and date now is: " + formatter.format(now) + "<br><br>");
out.println("The session creation time: HttpSession.getCreationTime( ): " +
formatter.format(creationTime) + "<br><br>");
out.println("The last time the session was accessed: HttpSession.getLastAccessedTime( ): " +
formatter.format(lastAccessed) );
out.println("</body>");
out.println("</html>");
} }
```

Which of the following options is stored at the client side?

A. URL rewriting

B. Hidden form fields

C. SSL sessions

D. Cookies

Summary

The key learning points of the module:

- A Java servlet is a server-side component that extends the capabilities of a web server and enables the creation of dynamic web applications. Servlets receive and respond to HTTP requests, allowing them to handle client interactions, process data, and generate dynamic content.
- Servlets run inside a servlet container, which is typically a web server or application server. The servlet container manages the lifecycle of servlets, handles the incoming requests, and provides necessary resources and services.
- Servlets are primarily used to handle HTTP requests and responses. They can process various HTTP methods like GET, POST, PUT, DELETE, etc. Servlets receive request parameters, headers, cookies, and other data from the client's HTTP request and can generate dynamic HTML or other content to be sent back as the response.
- Servlets are implemented by creating a Java class that extends the `HttpServlet` class or one of its subclasses. The `HttpServlet` class provides methods such as `doGet()`, `doPost()`, `doPut()`, `doDelete()`, etc., which you can override to handle specific HTTP methods.
- Servlets have a defined lifecycle managed by the servlet container. The container initializes the servlet when the application starts, calls the `init()` method to perform any initialization tasks, and then invokes the appropriate `doXXX()` method to handle each incoming request. Finally, when the application is stopped or restarted, the container calls the `destroy()` method to release any resources held by the servlet.
- Servlets are typically packaged as Java archive (JAR) files or web application archive (WAR) files and deployed within a servlet container. Configuration details, such as URL mappings to servlets, can be specified in a web deployment descriptor (`web.xml`) or using annotations (`@WebServlet`).

83

Summary (Cont.)

Here are the key learning points of the module (cont.):

- Servlets are commonly used to process HTML forms submitted by users. They can extract form parameters, validate user input, and perform business logic accordingly. Servlets can also access databases, perform file operations, and interact with other components to process and store data.
- Servlets can maintain session state for individual clients by using session objects. Sessions allow you to store and retrieve data across multiple requests from the same client. Servlets can create, manage, and destroy sessions, as well as store session attributes for personalized user experiences.
- Servlets are inherently multithreaded. The servlet container handles concurrent requests by spawning multiple threads, each executing a separate instance of the servlet class. As a developer, you should ensure that your servlets are thread-safe and properly handle shared resources.
- Servlets interact with the servlet container using the Servlet API, which provides a set of classes and interfaces for servlet development. The API includes classes like `HttpServletRequest`, `HttpServletResponse`, `HttpSession`, and many others that offer methods to access request parameters, manipulate responses, manage sessions, and more.
- Servlets are part of the Jakarta EE and play a crucial role in web development within the Java EE/Jakarta EE ecosystem.